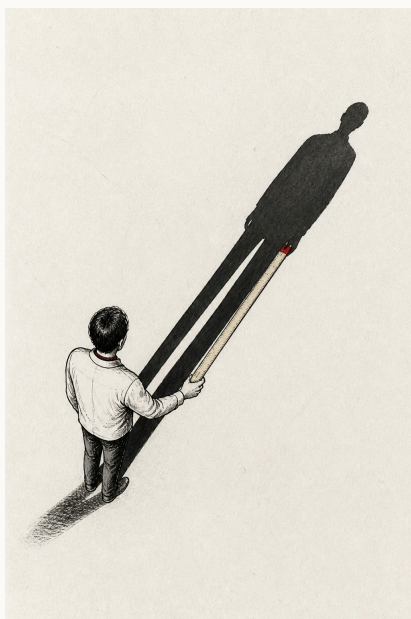


เร็วขึ้นจริง หรือแค่รู้สึกว่า เร็วขึ้น?

บันทึกของคนที่สร้างเกณฑ์ตรวจคนอื่น แล้วถูกเกณฑ์นั้นตรวจกลับ



หลักฐาน ความเชื่อ และการตรวจงานของตัวเอง
ฉบับพิมพ์ครั้งที่หนึ่ง

โครงการพิสูจน์ · THE PROOF PROJECT

เร็วขึ้นจริง หรือแค่รู้สึกว่ารวดขึ้น?

บันทึกของคนที่เราสร้างเกณฑ์ตรวจคนอื่น แล้วถูกเกณฑ์นั้นตรวจกลับ

ข้อเท็จจริงและตัวเลขทั้งหมดในเล่มนี้มาจากโครงการจริง และตรวจสอบกับซอร์สโค้ด
แล้ว ณ วันที่จัดพิมพ์

การทดลองกับผู้เล่นที่กล่าวถึงในบทที่ 7 ยังไม่ได้เก็บข้อมูล เล่มนี้จึงเล่าถึงการออกแบบ
การทดลอง ไม่ใช่ผลลัพธ์

สารบัญ

คำนำ	ทำไมถึงเขียนเล่มนี้	5 นาที
บทที่ 1	ปุ่มที่ไม่มีใครอ่านฉลาก	17 นาที
บทที่ 2	สองร้อยคำแนะนำ กับคำถามเดียวที่ไม่มีใครถาม	19 นาที
บทที่ 3	เกณฑ์ห้ามติ	18 นาที
บทที่ 4	ความปลอดภัยที่ไม่ใช่คำสัญญา	22 นาที
บทที่ 4ก	กระบวนการที่มองไม่เห็น	10 นาที
บทที่ 4ข	เอกสารที่เขียนก่อนโค้ด	10 นาที
บทที่ 4ค	โค้ดอยู่ที่ไหน ทำงานยังไง	11 นาที
บทที่ 5	การทดสอบคือการพยายามทำให้ของตัวเองพัง	21 นาที
บทที่ 6	วันที่ห่าสิบหกกลายเป็นสี่สิบสี่	22 นาที
บทที่ 7	สิ่งที่ยังพิสูจน์ไม่ได้	20 นาที
บทที่ 8	วิธีอ่านคำแนะนำที่คุณจะเจอพ่วงนี้	15 นาที
	อภิธานศัพท์	4 นาที
	คำถามชวนคุย	3 นาที
	กิตติกรรมประกาศ	2 นาที

คำนำ — ทำไมถึงเขียนเล่มนี้

ก่อนจะเริ่มอ่าน

หนังสือเล่มนี้อาจทำให้คุณพูดคำว่า "เร็วขึ้น" ไม่ได้อีกเลย
โดยไม่มีคำถามตามมาในหัวว่า เร็วขึ้นเทียบกับอะไร วัด
ตอนไหน วัดกี่ครั้ง

และถ้าวัดใหม่วันนี้จะได้เลขเดิมไหม

ผมไม่ได้ตั้งใจให้มันเป็นอย่างนั้น มันเกิดกับผมก่อน แล้ว
ผมกลับไปแก้ไขไม่ได้

หนังสือเล่มนี้เริ่มจากคำถามที่ผมตอบไม่ได้

คำถามที่ 01

เรารู้ได้ยังไงว่าสิ่งที่เราคิดว่าดีขึ้น มันดีขึ้นจริง

ตอนนั้นผมกำลังอธิบายให้คนฟัง ว่าเครื่องมือที่ผมทำอยู่ ต่างจากเครื่องมืออื่นยังไง ผมพูดไปได้สักพักว่ามันปลอดภัยกว่า มีหลักฐานรองรับกว่า ย้อนกลับได้จริง

แล้วเขาถามว่า "แล้วรู้ได้ยังไงที่พูดมาจริง"

คำถามนี้ใช้เวลาถามไม่ถึงสิบวินาที

ผมใช้เวลาหลายเดือนกับการตอบ

การสร้างสิ่งที่เร็วกว่า ไม่ยากเท่าการพิสูจน์ว่าสิ่งนั้นเร็วกว่าจริง

ผมตอบไม่ได้ในตอนนั้น ไม่ใช่เพราะสิ่งที่พูดไปไม่จริง แต่เพราะผมไม่มีวิธีพิสูจน์ที่ต่างจากวิธีที่คนอื่นใช้ ซึ่งเป็นวิธีที่ผมเพิ่งพิจารณาไปเมื่อครู่ว่าเชื่อไม่ได้

คำถามนั้นทำให้โครงการเปลี่ยนทิศ จากการสร้างเครื่องมือที่ดีกว่า กลายเป็นการหาว่าคำว่า "ดีกว่า" จะพิสูจน์ได้ยังไง

สิ่งที่ผมพบระหว่างทาง

ผมคิดว่าจะใช้เวลาสองสามสัปดาห์กับการคัดกรอง แล้วกลับไปเขียนโปรแกรมต่อ

ที่จริงมันกินเวลาไปหลายเดือน และเปลี่ยนสิ่งที่ผมคิดว่าตัวเองกำลังทำอยู่

สิ่งที่ผมไม่ได้คาดไว้เลยมีอยู่สองอย่าง

อย่างแรก การพิสูจน์ว่าอะไรได้ผลจริง ยากกว่าการสร้างสิ่งนั้นขึ้นมาหลายเท่า ผมใช้เวลาเขียนโปรแกรมน้อยกว่าเวลาที่ใช้ออกแบบวิธีวัด ว่าโปรแกรมนั้นได้ผลไหม

อย่างที่สอง และเป็นอย่างที่ผมไม่อยากเจอที่สุด คือเกณฑ์ที่ผมสร้างขึ้นเพื่อตรวจงานคนอื่น สุดท้ายมันชักกลับมาทำงานของผมเอง

เรื่องนั้นอยู่ในบทที่ 6



เรากำลังวัดสิ่งที่เป็นจริง หรือวัดภาพที่เราอยากเห็น

หนังสือเล่มนี้เหมาะกับใคร

ถ้าคุณเล่นเกมและเคยกดปุ่มปรับแต่งโดยไม่รู้ว่ามันทำอะไร เล่มนี้เขียนถึงคุณ

ถ้าคุณทำงานสายเทคนิคและต้องตัดสินใจว่าจะเชื่อคำแนะนำไหน เล่มนี้ก็เขียนถึงคุณ

และถ้าคุณกำลังทำโครงการหรืองานวิจัย ที่ต้องพิสูจน์อะไรสักอย่าง ผมคิดว่าบทที่ 6 กับ 7 น่าจะเป็นประโยชน์ที่สุด

สิ่งที่คุณไม่ต้องมีคือความรู้เรื่องระบบปฏิบัติการ สถิติ หรือการเขียนโปรแกรม ทุกอย่างที่ต้องใช้ ผมอธิบายด้วยการเปรียบเทียบกับของที่เห็นได้ในชีวิตประจำวัน

หนังสือเล่มนี้ไม่ใช่อะไร

มันไม่ใช่คู่มือ ผมไม่ได้เขียนว่าให้ไปตั้งค่าอะไรบ้าง เพราะถ้าเขียนแบบนั้น มันจะกลายเป็นคำแนะนำอีกชั้นที่คุณต้องเชื่อ ซึ่งขัดกับทั้งเล่ม

มันไม่ใช่การเชียร์ซอฟต์แวร์ที่ผมทำ ในหนังสือมีทั้งจุดที่มันทำได้ดี และจุดที่มันยังพิสูจน์ไม่ได้ รวมถึงบทหนึ่งที่เล่าว่าผมประกาศตัวเลขเกินจริงไปแล้วต้องแก้

และมันไม่ใช่หนังสือที่จบด้วยผลการทดลอง เพราะการทดลองยังไม่ได้เก็บข้อมูล ผมอธิบายเหตุผลไว้ในบทที่ 7 และบอกไว้ตรงนี้ก่อนเลย เพื่อไม่ให้ใครอ่านไปเรื่อย ๆ แล้วผิดหวังตอนท้าย

วิธีอ่าน

เปิดบทเรียงตามลำดับเวลาที่เกิดขึ้นจริง อ่านเรียงจะได้ภาพครบที่สุด

ถ้ามีเวลาน้อย บทที่ 1 กับบทที่ 8 อ่านคู่กันได้ บทแรกตั้งคำถาม บทสุดท้ายให้เครื่องมือไปใช้ต่อ

ถ้าสนใจเรื่องการตรวจสอบตัวเอง ข้ามไปบทที่ 6 ได้เลย มันอ่านแยกได้

ผมพยายามเขียนให้แต่ละบทมีอะไรติดมือกลับไป ไม่ใช่แค่ข้อมูล แต่เป็นวิธีถามคำถามที่ใช้กับเรื่องอื่นได้

ขอบคุณที่หยิบเล่มนี้ขึ้นมาครับ

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน หนังสือเล่มนี้น่าจะสอนวิธีทำให้คอมพิวเตอร์เร็วขึ้น

หลังอ่าน มันสอนวิธีตรวจว่าคำว่าเร็วขึ้นที่ใครพูด รวมทั้งที่ตัวเองพูด เชื่อได้แค่ไหน

- ✓ รู้ว่าคำถามหลักของเล่มนี้คืออะไร และทำไมมันตอบยากกว่าที่คิด
- ✓ รู้ล่วงหน้าว่าเล่มนี้ไม่ใช่คู่มือ ไม่ใช่การเชียร์ซอฟต์แวร์ และไม่จบด้วยผลการทดลอง

บทที่ 1 — ปุ่มที่ไม่มีใครอ่านฉลาก

คำถามที่ 02

การปรับแต่งระบบที่ทำให้เกมลื่นขึ้นจริงมีอยู่จริงไหม
แล้วจะแยกมันออกจากคำแนะนำที่แค่ฟังดูดี ได้อย่างไร

ติสองครั้ง เพื่อนผมส่งข้อความมาว่าเครื่องเปิดไม่ติด

เขาไม่ได้ทำอะไรแปลก แค่ดูคลิปหนึ่งที่มีคนสอนว่า ถ้าอยากให้
เกมลื่นขึ้น ให้ดาวน์โหลดไฟล์ตัวนี้แล้วกดรัน คลิปนั้นมีคนดูสองแสน
กว่า ในคอมเมนต์มีแต่คนบอกว่าได้ผล เขาเลยกด

สิ่งที่สคริปต์นั้นทำ มีอยู่บรรทัดหนึ่งที่ไปแตะการตั้งค่าการบูตของ
Windows คนเขียนสคริปต์อาจไม่ได้ตั้งใจให้เป็นแบบนี้ อาจจะลอก
มาจากที่อื่นอีกที แต่ผลคือเพื่อนผมต้องนั่งกู้เครื่องจนถึงเช้า

ตอนผมช่วยเขาไล่หาว่าอะไรพัง ผมขอไฟล์นั้นมาเปิดดู มันยาว
สองร้อยกว่าบรรทัด ไม่มีคอมเมนต์อธิบายแม้แต่บรรทัดเดียว ไม่มี
การสำรองค่าเดิมก่อนเขียนทับ และไม่มีทางย้อนกลับ

สิ่งที่ผมคิดไม่ตกในคืนนั้นไม่ใช่ว่าสคริปต์นั้นเขียนแย่แค่ไหน แต่เป็นเรื่องที่เขาไม่มีทางรู้เลยว่ามันจะทำอะไร ไฟล์นั้นเปิดอ่านได้ก็จริง แต่ถ้าอ่านไม่ออกก็เท่ากับกดปุ่มที่ไม่มีฉลาก

และคนที่กดปุ่มแบบนั้นทุกวัน มีเป็นแสน

เงินหลายหมื่นกับการเดา

ลองดูคนเล่นเกมแข่งขันสักคน

เขาจะรู้ว่าจอ 240 เฮิร์ตซ์ต่างจาก 144 ยังไง รู้ว่าเมาส์รุ่นไหนมีความหน่วงต่ำกว่ากันก็มีลิลิวนาติ รู้ว่าการ์ดจอตัวนี้ให้เฟรมเรตมากกว่าตัวนั้นเท่าไรในเกมที่เขาเล่น เขาอ่านรีวิว ดูกราฟ เทียบราคา แล้วจ่ายเงินหลายหมื่นด้วยข้อมูล

แล้วพอมาถึงระบบปฏิบัติการ ซึ่งเป็นขั้นที่คุมความหน่วงจริง เขากลับทำด้วยการเดา

ไม่ใช่เพราะเขาโง่ แต่เพราะมันมองไม่เห็น

จอที่รีเฟรชเร็วขึ้นเห็นได้ด้วยตา เมาส์ที่หน่วงน้อยลงรู้สึกได้ด้วยมือ แต่การตั้งค่าใน registry ที่เปลี่ยนวิธีที่ระบบจัดคิวงาน ไม่มีอะไรให้ดู ไม่มีอะไรให้จับ มีแต่คนบอกว่ามันดีขึ้น

เมื่อไม่มีอะไรวัดได้ คนก็เชื่อคนที่พูดดังที่สุด

ความหน่วงคืออะไร ในภาษาที่ไม่ต้องรู้เรื่องคอมพิวเตอร์

ก่อนจะไปต่อ ผมอยากให้เห็นภาพว่าเรากำลังพูดถึงอะไร

สมมติว่าคุณอยู่ในตึกสำนักงานสูงยี่สิบชั้น มีพนักงานส่งเอกสารคนหนึ่งต้องเอาซองจากชั้นหนึ่งขึ้นไปชั้นยี่สิบ เวลาที่เขาใช้ไม่ได้ขึ้นกับว่าเขาเดินเร็วแค่ไหน แต่ขึ้นกับว่าเขาต้องรอลิฟต์นานเท่าไร และมีใครมาแทรกคิวหน้าเขาบ้าง

ตอนคุณคลิกเมาส์ในเกม สัญญาณนั้นก็ต้องต่อคิวเหมือนกัน มันต้องรอให้ระบบปฏิบัติการหันมาสนใจ รอให้ไดรเวอร์ประมวลผล รอให้การ์ดจอวาดภาพเสร็จ ทุกจุดที่ต้องรอคือความหน่วง

สิ่งที่การปรับแต่งระบบทำ ไม่ใช่การทำให้พนักงานเดินเร็วขึ้น แต่คือการจัดคิวใหม่ ลดคนที่มาแทรก และบอกลิฟต์ว่าใครควรได้ขึ้นก่อน

พอมองแบบนี้ จะเห็นสองอย่างพร้อมกัน

อย่างแรก การจัดคิวใหม่ช่วยได้จริง ถ้าเดิมมีคนแทรกเยอะ การเอาคนที่ไม่จำเป็นออกไปก็ทำให้ช่องถึงเร็วขึ้นจริง นี่คือเหตุผลที่การปรับแต่งบางอย่างได้ผล และเป็นเหตุผลที่โครงการนี้มีอยู่

อย่างที่สอง คนที่ถูกเอาออกจากคิว บางคนอยู่ที่นั่นเพราะมีเหตุผล

คนที่ตรวจบัตรตอนเข้าตึกก็ทำให้ช้าลงจริง แต่เขาอยู่ที่นั่นเพราะมีหน้าที่ การเอาเขาออกจะทำให้ทุกอย่างเร็วขึ้น จนถึงวันที่มีคนที่ไม่ควรเข้ามาเดินเข้ามา

การถอดเบรกกออก

มีคำแนะนำประเภทหนึ่งที่ผมเจอบ่อยมากตอนสำรวจ มันบอกให้ปิดระบบตรวจสอบความปลอดภัยของ Windows เพื่อให้เกมได้ทรัพยากรมากขึ้น

คำแนะนำนั้นไม่ได้โกหก การปิดมันทำให้เครื่องเร็วขึ้นจริง เพราะระบบตรวจสอบก็ใช้ทรัพยากรเหมือนกัน ตัวเลขที่วัดได้ก็ดีขึ้นจริง คนที่ทำตามแล้วบอกว่าได้ผล เขาพูดความจริง

การถอดเบรกรอกจากรถก็ทำให้รถเบาขึ้นจริง และเร็วขึ้นจริงด้วย ปัญหาไม่ได้อยู่ที่ว่าคำแนะนำนั้นผิด ปัญหาอยู่ที่มันบอกไม่ครบ มันบอกสิ่งที่คุณได้ แต่ไม่บอกสิ่งที่คุณเสีย แล้วปล่อยให้คนกดปุ่มโดยคิดว่าได้อย่างเดียว

ผมคิดว่าความต่างระหว่างคำแนะนำที่ดีกับคำแนะนำที่ไม่ดี ไม่ได้อยู่ที่มันแนะนำให้ทำอะไร แต่อยู่ที่มันบอกคุณครบไหมว่าคุณกำลังแลกอะไรกับอะไร

นี่คือสิ่งที่ทำให้ผมเริ่มโครงการนี้ ผมไม่ได้อยากสร้างเครื่องมือที่เร็วที่สุด ผมอยากสร้างเครื่องมือที่บอกได้ว่ามันแลกอะไรไป

สองร้อยเทคนิค

ผมเริ่มจากการไปดูว่าคนแนะนำอะไรกันบ้าง

ผมเปิดโครงการโอเพนซอร์สบน GitHub อ่านฟอรัม ดูคลิป รวบรวมทุกเทคนิคที่มีคนบอกว่าช่วยลดความหน่วงหรือเพิ่มเฟรมเรต ไม่คัดกรองอะไรทั้งนั้นในขั้นนี้ แค่จดว่ามีอะไรบ้าง เทคนิคเดียวกันที่มีคนแนะนำสิบทีก็นับเป็นหนึ่ง

ได้มาสองร้อยเทคนิค

จากนั้นผมตั้งเกณฑ์ขึ้นมาชุดหนึ่ง แล้วเอาทั้งสองร้อยเทคนิคมา ให้คะแนน รายละเอียดของเกณฑ์อยู่ในบทที่ 3 ตอนนี้อยู่ที่ขั้นตอนให้ผลลัพธ์ก่อน

ผ่านสี่สิบห้าเทคนิค

เทียบกับของจริง

สิ่งที่เชื่อ	คำแนะนำปรับแต่งเกมที่ได้ทั่วไปในอินเทอร์เน็ต ส่วนใหญ่น่าเชื่อถือ
สิ่งที่วัดได้	จากสองร้อยเทคนิคที่สำรวจ ผ่านเกณฑ์แค่สี่สิบห้า อีกหนึ่งร้อยห้าสิบห้า คิดเป็น 77.5% ไม่ผ่าน

อีกหนึ่งร้อยห้าสิบห้าเทคนิค คิดเป็น 77.5% ไม่ผ่าน

ตัวเลขนี้ทำให้ผมหยุดคิดอยู่พักหนึ่ง มันไม่ได้แปลว่าคนที่แนะนำเทคนิคเหล่านั้นตั้งใจหลอกใคร ส่วนใหญ่ผมเชื่อว่าเขาเชื่อจริง ๆ ว่ามันได้ผล

แต่มันแปลว่าถ้าคุณหยิบคำแนะนำมาสุ่มหนึ่งอันจากอินเทอร์เน็ต โอกาสที่มันจะผ่านเกณฑ์ด้านความปลอดภัยและหลักฐาน อยู่ที่ประมาณหนึ่งในห้า

เทคนิคที่ไม่ผ่าน หน้าตาเป็นยังไง

หนึ่งร้อยห้าสิบห้าเทคนิคที่ไม่ผ่าน แบ่งได้เป็นสามกลุ่มคร่าว ๆ

กลุ่มแรกคือเทคนิคที่ทำอะไรจริง แต่สิ่งที่มันทำไม่เกี่ยวกับที่โฆษณา ตัวอย่างคลาสสิกคือการ defrag ไดรฟ์ SSD คำแนะนำนี้ยังพบได้ทั่วไป เพราะสมัยฮาร์ดดิสก์แบบจานหมุน การ defrag ช่วยได้จริง มันย้ายไฟล์ที่กระจัดกระจายมาอยู่ติดกัน เพื่อให้หัวอ่านไม่ต้องวิ่งไปมา

แต่ SSD ไม่มีหัวอ่าน มันเข้าถึงทุกตำแหน่งเร็วเท่ากัน การ defrag จึงไม่ช่วยอะไร และเพราะมันเขียนข้อมูลจำนวนมาก มันกินอายุการใช้งานของไดรฟ์ไปฟรี ๆ

คำแนะนำนี้เคยถูกต้อง มันแค่ไม่ได้ตายไปพร้อมกับเทคโนโลยีที่มันเคยถูกต้องกับ

การ defrag ไดรฟ์ SSD เป็นตัวอย่างของคำแนะนำที่ตกยุคแล้วยังเล่าต่อกันไป มันไม่ได้ทำร้ายทันทีเหมือนสคริปต์ที่แะการตั้งค่าการบูต แต่มันกินอายุการใช้งานไดรฟ์ไปเรื่อย ๆ โดยผู้ใช้ไม่รู้ตัว

กลุ่มที่สองคือเทคนิคที่ไม่ทำอะไรเลย มีการตั้งค่าหนึ่งที่ผมเจอในเกือบทุกที่ มันบอกให้ตั้งค่าตัวหนึ่งใน registry เป็นศูนย์ เพื่อให้ระบบให้ความสำคัญกับเกมมากขึ้น

ผมตามหาเอกสารทางเทคนิคของค่านั้น แล้วพบว่าค่าที่กำหนดไว้มีความหมายเฉพาะในบริบทของการเล่นเสียง และวิดีโอพร้อมกัน ในสถานการณ์เล่นเกมทั่วไป การตั้งเป็นศูนย์ไม่ได้เปลี่ยนพฤติกรรมของระบบอย่างที่คำแนะนำอ้าง

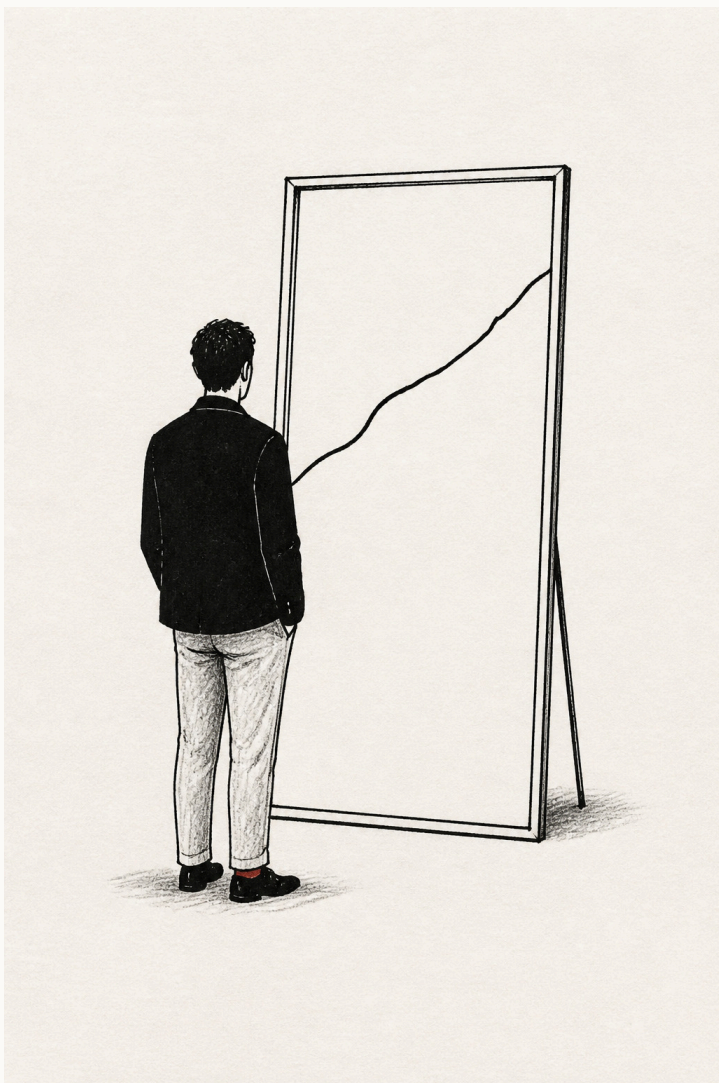
เทคนิคกลุ่มนี้เป็นกลุ่มที่แพร่ง่ายที่สุด เพราะมันไม่ทำอะไรเลย จึงไม่มีใครเจอปัญหา ไม่มีใครมาบ่น มันจึงอยู่รอดในคำแนะนำต่อ ๆ กันไปได้นานเป็นสิบปี

กลุ่มที่สามคือกลุ่มที่ผมกังวลที่สุด มันคือเทคนิคที่ได้ผลจริง แต่แลกด้วยการปิดระบบความปลอดภัย เทคนิคกลุ่มนี้ไม่ได้ผิดในทางเทคนิค มันผิดที่ไม่บอก

ในกลุ่มที่สามนี้ มีอยู่เจ็ดเทคนิคที่ผมตัดสินใจตั้ง แต่ต้นว่าจะไม่ใส่ในเครื่องมือ ไม่ว่าจะเป็นมีคนขอแค่ไหน เพราะผลของมันไม่ได้จำกัดอยู่แค่เครื่องของคนทีกด

การปิดโปรแกรมป้องกันไวรัสของ Windows เป็นหนึ่งในนั้น การปิดระบบถามยืนยันก่อนติดตั้งโปรแกรมก็เป็นอีกหนึ่ง และการปิดการอัปเดตอัตโนมัติแบบถาวรก็เช่นกัน

ผมเข้าใจว่าทำไมคนถึงอยากปิด การอัปเดตมันดีขึ้นมากกลาง
เกม มันรีสตาร์ทเครื่องตอนที่คุณไม่ได้ขอ มันน่ารำคาญจริง แต่การ
ปิดถาวรกับการเลื่อนเวลาไม่เหมือนกัน อย่างแรกคือการยอมอยู่กับ
ช่องโหว่ที่ค้นพบแล้วตลอดไป



คำอ้างที่สะท้อนกลับมาเป็นตัวเลข แต่ตัวเลขไม่ได้สะท้อนตัว
เรา

หนึ่งในนั้นทำสิ่งที่ไม่ได้บอกไว้

ตอนไล่อ่านโค้ดของเครื่องมือตัวหนึ่งที่มีคนใช้เยอะพอสมควร ผมเจอ บรรทัดที่ทำให้หยุดอ่านอยู่พักหนึ่ง

หน้าคำอธิบายของมันบอกว่าปุ่มนั้นเป็นการปรับแต่งด้านการแสดงผล ลดเอฟเฟกต์ที่ไม่จำเป็น เพื่อให้ระบบตอบสนองเร็วขึ้น เป็นเรื่องสมเหตุสมผลและปลอดภัย

แต่ในโค้ดของปุ่มเดียวกันนั้น มีคำสั่งที่ไปปิดฟีเจอร์ความปลอดภัยระดับระบบตัวหนึ่งด้วย และไม่มีที่ไหนในหน้าจอที่บอกผู้ใช้เรื่องนี้เลย

ผมไม่คิดว่าคนเขียนตั้งใจปกปิด ผมเดาว่าเขาเอาชุดคำสั่งมารวมกันเป็นปุ่มเดียว เพื่อความสะดวก แล้วเขียนคำอธิบายตามสิ่งที่เขาคิดว่าปุ่มนั้นทำ

แต่เจตนาไม่ได้เปลี่ยนผลลัพธ์ คนที่กดปุ่มนั้นเชื่อว่าเขากำลังลดเอฟเฟกต์ภาพ เขาไม่ได้เลือกที่จะปิดระบบความปลอดภัย เพราะไม่มีใครบอกเขาวนนั้นคือส่วนหนึ่งของการกด

การเลือกที่ไม่รู้ตัวว่ากำลังเลือก ไม่ใช่การเลือก

ทำไมของที่ไม่จริงถึงแพร่ได้เร็วกว่า

เรื่องนี้มีคำอธิบายที่ผมคิดว่าตรงไปตรงมามาก

คำแนะนำที่ถูกต้องมักจะฟังดูน่าเบื่อ มันมักจะมีเงื่อนไข มีข้อยกเว้น และมักจะบอกว่าผลที่ได้อยู่ในระดับไม่กี่เปอร์เซ็นต์

คำแนะนำที่ผิดไม่มีข้อจำกัดพวกนั้น มันบอกได้เลยว่าเร็วขึ้นสองเท่า บอกได้ว่าใช้ได้กับทุกเครื่อง บอกได้ว่าทำครั้งเดียวจบ

ระหว่างคลิปที่บอกว่า "อาจช่วยได้ 3 ถึง 5 เปอร์เซ็นต์ ขึ้นกับว่าเครื่องคุณคอขวดตรงไหน" กับคลิปที่บอกว่า "ทำตามนี้แล้ว FPS พุ่ง" คลิปไหนมีคนดูมากกว่า

มีอีกเรื่องที่ทำให้แยกยากขึ้นไปอีก คือการปรับแต่งพวกนี้มักจะให้ความรู้รู้สึกว่าได้ผลเสมอ ไม่ว่ามันจะได้ผลจริงหรือไม่

สี่สิบนาทีที่ทำให้ทุกอย่างดูดีขึ้น

ลองนึกภาพลำดับเหตุการณ์นี้

คุณเปิดคลิปปยาวสิบห้านาที กดหยุดกดเล่นตามไปที่ละขั้น เปิด registry แก่ค่าห้าหกจุด ปิด service ที่คลิปบอกว่าไม่จำเป็น ปรับการตั้งค่าการ์ดจอ ใช้เวลาไปสี่สิบนาที แล้วรีสตาร์ทเครื่อง

เครื่องเปิดขึ้นมา คุณเปิดเกม

มันลื่นขึ้น

ตรงนี้มีอย่างน้อยสามอย่างเกิดขึ้นพร้อมกัน และมีแค่อย่างเดียวที่เกี่ยวกับการปรับแต่ง

อย่างแรก เครื่องคุณเพิ่งรีสตาร์ท หน่วยความจำถูกล้าง โปรแกรมที่ค้างอยู่หายไป โปรแกรมพื้นหลังที่ปกติเปิดสะสมมาทั้งวันยังไม่ทันเปิด ถ้าคุณรีสตาร์ทเฉย ๆ โดยไม่ปรับอะไร คุณก็จะได้ผลคล้ายกัน

อย่างที่สอง คุณเพิ่งลงทุนเวลาไปสี่สิบนาที สมอมนมนุษย์ไม่ค่อยเก่งเรื่องการยอมรับว่าเวลาที่ลงไป แล้วไม่ได้ผล

อย่างที่สอง การปรับแต่งบางอย่างอาจได้ผลจริง

ปัญหาคือคุณแยกสามอย่างนี้ออกจากกันด้วยความรู้สึกไม่ได้เลย และความรู้สึกก็เป็นเครื่องวัดเดียวที่คนส่วนใหญ่มี

นี่คือเหตุผลที่งานนี้ต้องมีการทดลองที่ควบคุมตัวแปร ไม่ใช่แค่คำบอกเล่าว่าดีขึ้น และเป็นเหตุผลที่ผมจะวัดค่าที่คนรู้สึกไม่ได้ด้วย เรื่องนั้นอยู่ในบทที่ 7

สิ่งที่หนังสือเล่มนี้จะทำ

ผมใช้เวลากับโครงการนี้อยู่พักใหญ่ สร้างเครื่องมือขึ้นมาตัวหนึ่ง ตั้งเกณฑ์ ทดสอบ แล้วออกแบบการทดลองเพื่อวัดว่ามันได้ผลจริงไหม หนังสือเล่มนี้ไม่ใช่คู่มือใช้เครื่องมือนั้น

สิ่งที่ผมอยากเล่าคือกระบวนการแยกของจริงออกจากความเชื่อ เพราะผมคิดว่ามันใช้ได้กับเรื่องอื่นนอกจากการปรับแต่งคอมพิวเตอร์ และเพราะระหว่างทางผมเจอสิ่งที่ไม่ได้คาดไว้ รวมถึงตอนที่การตรวจสอบชักกลับมาที่ตัวผมเอง

ผมจะเล่าตามลำดับที่มันเกิดขึ้นจริง ไม่ใช่ลำดับที่ทำให้ผมดูฉลาด แปลว่าคุณจะได้อ่านตอนที่ผมคิดผิดด้วย

มีอีกอย่างที่ต้องบอกไว้ตั้งแต่ต้น หนังสือเล่มนี้ไม่มีบทที่บอกว่า "และแล้วผลการทดลองก็ออกมาว่า" เพราะการทดลองกับผู้เล่นจริง ยังไม่ได้เก็บข้อมูล ผมจะอธิบายในบทที่ 7 ว่าทำไม และทำไมผมถึงคิดว่าการเขียนแบบนี้ตรงกว่าการรอให้ครบก่อน แล้วค่อยเล่าย้อนหลัง

ถ้าคุณกำลังหาหนังสือที่บอกว่าทำสิบข้อนี้ แล้วเครื่องเร็วขึ้น เล่มนี้ไม่ใช่ ถ้าคุณอยากรู้ว่าจะแยกคำแนะนำที่เชื่อได้ ออกจากคำแนะนำที่แค่ฟังดูดี ยังไง เล่มนี้เขียนมาให้คุณ

ก่อนไปบทัดไป

ลองนึกถึงคำแนะนำทางเทคนิคขั้นล่าสุดที่คุณทำตาม อาจจะเป็นการตั้งค่าโทรศัพท์ การปรับเราเตอร์ หรือการติดตั้งอะไรสักอย่าง

คุณรู้ใหม่วามันไปเปลี่ยนอะไร

ถ้ารู้ คุณรู้เพราะอ่านเองหรือเพราะมีคนบอก

คำถามพวกนี้ไม่ได้มีไว้ทำให้รู้สึกผิด ทุกคนทำแบบนั้น รวมถึงผมด้วย ตอนเริ่มโครงการนี้ ในเครื่องผมเองก็มีการตั้งค่าหลายอย่าง ที่ทำตามคนอื่นมา โดยไม่รู้ว่ามันทำอะไร ผมพบตอนหลังว่าสองในนั้นไม่ได้ทำอะไรเลย

แต่มันเป็นจุดเริ่มต้นของบทัดไป ตอนที่ผมเปิดโค้ดของเครื่องมือยอดนิยมยี่สิบสามตัว เข้าไปอ่านที่ละบรรทัด ผมเจออะไรบ้าง

ปุ่มที่ผมเล่าไปเมื่อครู่ ไม่ใช่กรณีที่แย่ที่สุดที่ผมเจอ

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน การปรับแต่งระบบเพื่อลดความหน่วงเป็นเรื่องของโชค หรือเป็นความลับที่มีคนรู้บ้างกลุ่มเท่านั้น

หลังอ่าน การปรับแต่งบางอย่างได้ผลจริงและอธิบายได้ แต่ส่วนใหญ่ที่แพร่หลายเป็นคำแนะนำที่ไม่บอกครบว่ากำลังแลกอะไรไป

- ✓ เข้าใจว่าความหน่วงคืออะไร ผ่านภาพเปรียบเทียบกับกรรอลิฟต์ในตึกสูง
- ✓ รู้ว่าจากสองร้อยเทคนิคที่สำรวจ มีแค่สี่สิบห้าเทคนิคที่ผ่านเกณฑ์ด้านความปลอดภัยและหลักฐาน
- ✓ รู้จักสามกลุ่มของเทคนิคที่ไม่ผ่านเกณฑ์ คือเทคนิคที่ตกยุค เทคนิคที่ไม่ทำอะไรเลย และเทคนิคที่ได้ผลแต่แลกด้วยความปลอดภัย

บทที่ 2 — สองร้อยคำแนะนำ กับ คำถามเดียวที่ไม่มีใครถาม

คำถามที่ 03

เครื่องมือปรับแต่งที่มีคนใช้มากที่สุดในโลกโอเพนซอร์ส
ตอบคำถามว่ากดปุ่มนี้แล้วแลกอะไรไป ได้ครบไหม

โครงการแรกที่ผมเปิดอ่าน มีดาวบน GitHub อยู่หลายพัน

หน้าแรกของมันทำมาดี มีภาพหน้าจอ มีคำอธิบายว่าแต่ละปุ่มทำอะไร มีคำเตือนว่าให้สำรองข้อมูลก่อนใช้ ดูเป็นงานที่คนทำตั้งใจ

ผมกดเข้าไปอ่านโค้ดข้างใน เพราะอยากรู้ว่าปุ่มที่เขียนว่า "ปรับแต่งเพื่อประสิทธิภาพสูงสุด" มันไปทำอะไรบ้าง

ใช้เวลาประมาณยี่สิบนาทีถึงจะไล่ครบ ปุ่มเดียวนั้นเรียกฟังก์ชันสิบเจ็ดตัว แต่ละตัวไปแก้ค่าคนละที่ บางตัวแก้สามสี่ค่า รวมแล้วปุ่มเดียวเปลี่ยนการตั้งค่าของเครื่องประมาณสี่สิบจุด

ในสี่สิบจุดนั้น มีสองจุดที่ไม่ได้อยู่ในคำอธิบาย เลย

ทำไมต้องอ่านโค้ด ไม่อ่านคำอธิบาย

ตอนเริ่มผมตั้งใจจะอ่านแค่เอกสาร คิดว่าเครื่องมือที่มีคนใช้เยอะขนาดนี้ คำอธิบายน่าจะตรงกับสิ่งที่มันทำ

ผมเลิกคิดแบบนั้นหลังจากอ่านโครงการที่สาม

ไม่ใช่เพราะเจอใครโกหก แต่เพราะพบว่าคำอธิบายกับโค้ดมันแยกกันเดิบโต คนเขียนคำอธิบายตอนเวอร์ชันแรก แล้วเพิ่มฟีเจอร์เข้าไปเรื่อย ๆ โดยไม่ได้กลับไปแก้คำอธิบาย

พอผ่านไปสองปี เอกสารก็เล่าเรื่องเครื่องมือคนละตัวกับที่โค้ดทำอยู่

เรื่องนี้ไม่ได้เกิดเฉพาะกับโครงการเล็ก ๆ มันเกิดกับซอฟต์แวร์แทบทุกตัว ที่ไม่มีกระบวนการบังคับให้เอกสารกับโค้ดเดินไปด้วยกัน ผมจะกลับมาเรื่องนี้อีกทีในบทที่ 6 ตอนที่มันเกิดขึ้นกับผมเอง

ดังนั้นเกณฑ์แรกที่ผมตั้งขึ้นก่อนจะเริ่มสำรวจจริง คือถ้าอ่านโค้ดไม่ได้ ผมจะไม่นับ

ยี่สิบแปดโครงการ เหลือยี่สิบสาม

ผมเริ่มจากรายชื่อโครงการที่คนแนะนำกันบ่อยที่สุด ได้มายี่สิบแปดโครงการ

ห้าโครงการถูกตัดออกตั้งแต่ต้น ด้วยเหตุผลสามข้อ

โครงการแรกที่ตัดออกเพราะเผยแพร่เฉพาะไฟล์ที่รันได้ ไม่มีโค้ดให้ดู ผมไม่ได้บอกว่ามันไม่ดี แต่ผมประเมินสิ่งที่ผมอ่านไม่ได้ไม่ได้

อีกสองโครงการมีโค้ดก็จริง แต่เป็นโค้ดที่ผ่านการแปลงให้อ่านไม่รู้เรื่อง ชื่อตัวแปรถูกเปลี่ยนเป็นตัวอักษรสุ่ม เจตนาอาจเป็นการป้องกันการลอก แต่ผลคืออ่านไม่ได้เหมือนกัน

อีกสองโครงการหยุดพัฒนาไปตั้งแต่ก่อน Windows 10 ค่าที่มันเก็บบางตัวไม่มีอยู่ในระบบปฏิบัติการรุ่นปัจจุบันแล้ว การเอามาเทียบจึงไม่ยุติธรรมกับทั้งสองฝ่าย

เหลืออีกลีบสามโครงการที่ผมอ่านโค้ดได้ทุกบรรทัด

ผมเขียนเกณฑ์การคัดออกนี้ไว้ก่อนเริ่มอ่าน ไม่ใช่หลังจากอ่านแล้วพบว่าบางตัวคะแนนแย่ เรื่องนี้ฟังดูเป็นรายละเอียดเล็กน้อย แต่มันคือความต่างระหว่างการคัดกรองกับการเลือกเฉพาะที่เข้าทาง

สิ่งที่ผมทำกับโค้ดอีลีสบสามชุด

วิธีอ่านของผมไม่ได้ซับซ้อน แต่ใช้เวลามาก

สำหรับแต่ละโครงการ ผมไล่หาจุดที่มันแตะระบบจริง ในทางปฏิบัติมีอยู่ไม่กี่แบบ การเขียนค่าลง registry การเปิดปิด service การแก้การตั้งค่าการบูต และการเรียกโปรแกรมของ Windows เอง

ทุกครั้งที่เจอจุดแบบนั้น ผมจดสามอย่าง มันแก้อะไร คำอธิบายบอกว่ามันแก้อะไร และถ้าต้องย้อนกลับต้องทำยังไง

ช่องที่สามคือช่องที่ว่างบ่อยที่สุด

หลายโครงการเขียนค่าใหม่ทับลงไปเลย โดยไม่อ่านค่าเดิมเก็บไว้ก่อน แปลว่าต่อให้อยากย้อนกลับ ก็ไม่รู้จะย้อนไปเป็นอะไร บางค่าเดาได้จากค่าเริ่มต้นของ Windows แต่ถ้าผู้ใช้เคยแก้เองมาก่อน ค่าเดิม

ของเขาก็หายไปแล้ว

สามสัปดาห์นั้นน่าเบื่อกว่าที่คิด

งานส่วนใหญ่คือการเปิดไฟล์ กวาดตาหาบรรทัดที่แต่ละระบบ แล้วจดลงตาราง ไม่มีอะไรน่าตื่นเต้น ไม่มีวินาทีที่ค้นพบอะไรยิ่งใหญ่ วันที่ดีคือวันที่ไล่ได้สองโครงการ

สิ่งที่เปลี่ยนไปในหัวผมช่วงนั้น ไม่ใช่ความรู้เรื่องเทคนิค แต่เป็นความเคยชิน

พออ่านโค้ดที่เขียนค่าทับลงไปโดยไม่เก็บค่าเดิมเป็นอันที่สิบ ผมเริ่มไม่แปลกใจกับมัน แล้วผมก็จับได้ว่าตัวเองเริ่มเลื่อนผ่านเร็วขึ้น

นั่นคือตอนที่ผมกลับไปนับใหม่ ว่าที่ไล่มาแล้วมีกี่โครงการที่ผมอ่านเร็วกว่าโครงการแรก

ผมใช้เวลากับส่วนนี้ประมาณสามสัปดาห์ สิ่งที่ได้ออกมาไม่ใช่แค่รายการเทคนิค แต่เป็นความเข้าใจว่าเครื่องมือกลุ่มนี้คิดยังไง

มันคิดแบบคนที่มั่นใจว่าตัวเองทำถูก

ตรงนี้ไม่ใช่ความผิด คนเราต้องมั่นใจระดับหนึ่งถึงจะกล้าเผยแพร่อะไรให้คนอื่นใช้ แต่ความมั่นใจกับการออกแบบเพื่อกรณีที่ตัวเองผิดเป็นคนละเรื่องกัน

ผมเก็บเรื่องหนึ่งไว้เล่าตรงนี้ เพราะตอนนั้นมันยังไม่สำคัญ แต่ต่อมามันสำคัญมาก

ระหว่างที่จดตารางอยู่ ผมมีเอกสารอีกไฟล์ที่เขียนไว้ก่อนเริ่มโครงการ ในนั้นมีตัวเลขที่ผมประกาศไว้ว่าเครื่องมือของผมมีเทคนิคกี่รายการ

ผมไม่ได้กลับไปตรวจตัวเลขนั้นเลยตลอดสามสัปดาห์ ทั้งที่ผมกำลังนับของคนอื่นอยู่ทุกวัน

เรื่องนั้นตามมาทวงในบทที่ 6

สิ่งที่พบ

หลังอ่านครบยี่สิบสามโครงการ ผมได้ภาพที่ชัดกว่าที่คาดไว้

สิบเอ็ดโครงการ คิดเป็น 47.8% ไม่มีระบบบอกระดับความเสี่ยง
เลยแม้แต่หนึ่งเดียว

เทียบกับของจริง

สิ่งที่เชื่อ	เครื่องมือปรับแต่งที่มีคนใช้เยอะ ย่อมมีระบบแยกระดับ ความเสี่ยงของแต่ละตัวเลือกให้ผู้ใช้เห็น
สิ่งที่วัดได้	จากยี่สิบสามโครงการที่อ่านโค้ดครบ สิบเอ็ดโครงการ คิดเป็น 47.8% ไม่มีระบบบอกระดับความเสี่ยงเลย แม้แต่หนึ่งเดียว

ไม่ใช่ว่ามันจัดหมวดหมู่หายากไป แต่คือไม่มีเลย ทุกการปรับแต่งถูก
วางไว้ในระนาบเดียวกันหมด การปิดเอฟเฟกต์ภาพกับการปิดฟีเจอร์
ความปลอดภัยของระบบ อยู่ในรายการเดียวกัน มีหน้าตาเหมือนกัน
กอดด้วยวิธีเดียวกัน

**ผู้ใช้ที่มองหน้าจอนั้นไม่มีทางรู้ว่าสองอย่างนี้ต่าง
กันคนละโลก**

เร็วขึ้นจริง หรือแค่รู้สึกว่เร็วขึ้น?

ภาพ 2.1 คำแนะนำสองร้อยรายการที่สำรวจมา เหลือสี่สิบห้ารายการหลังคัดกรอง และเหลือสี่สิบสี่หลังตรวจสอบซ้ำอีกรอบ เรื่องนั้นอยู่ในบทที่ 6 ส่วนที่หายไป ระหว่างทางไม่ได้หายไปเพราะใช้ไม่ได้ แต่เพราะหาเหตุผลมารองรับไม่ได้

200

คำแนะนำที่สำรวจ

จาก 23 แหล่งบนอินเทอร์เน็ต

45

ผ่านการคัดกรอง

มีเหตุผลทางเทคนิครองรับ

44

อยู่ในเครื่องมือจริง

หลังตรวจสอบซ้ำตัดออกหนึ่งรายการ

ที่มา: การคัดกรองใน 23 แหล่ง กับผลตรวจสอบซ้ำ D-03

ตัวอย่างที่ทำให้เห็นภาพชัดที่สุด

ในโครงการหนึ่ง ผมเจอรายการปรับแต่งเรียงกันอยู่ในหน้าเดียว มีช่องให้ติ๊กข้างหน้าทุกอัน

อันหนึ่งเขียนว่าปิดแอนิเมชันตอนย่อขยายหน้าต่าง เป็นการปรับที่ผลกระทบจำกัดอยู่แค่ความสวยงาม ถ้าไม่ชอบก็เปิดกลับได้ในสามวินาที

อีกอันที่อยู่ห่างกันสองบรรทัด เขียนว่าปิดการป้องกันหน่วยความจำระดับฮาร์ดแวร์ ตัวนี้เป็นกลไกที่กันโปรแกรมอันตรายไม่ให้รันโค้ดในพื้นที่ที่ไม่ควรรัน

สองอันนี้มีช่องติ๊กแบบเดียวกัน ตัวหนังสือขนาดเดียวกัน สีเดียวกัน และอยู่ในกลุ่มเดียวกันที่ชื่อว่า "ประสิทธิภาพ"

ถ้าผู้ใช้กดปุ่ม "เลือกทั้งหมด" อย่างที่คนส่วนใหญ่ทำ เขาก็ได้ทั้งสองอย่างพร้อมกัน

ผมไม่คิดว่าคนออกแบบหน้าจอจะตั้งใจให้เป็นแบบนี้ ผมคิดว่าเขาแค่ไม่ได้คิดว่ามันต่างกัน เพราะในมุมมองของคนเขียนโค้ด ทั้งสองอย่างคือการเขียนค่าลง registry เหมือนกัน ต่างกันแค่ค่าที่เขียน

แต่ในมุมมองของผลลัพธ์ อันหนึ่งทำให้หน้าต่างแดงเร็วขึ้น อีกอันทำให้เครื่องเปิดช่องให้โปรแกรมอันตรายทำงานได้ง่ายขึ้น

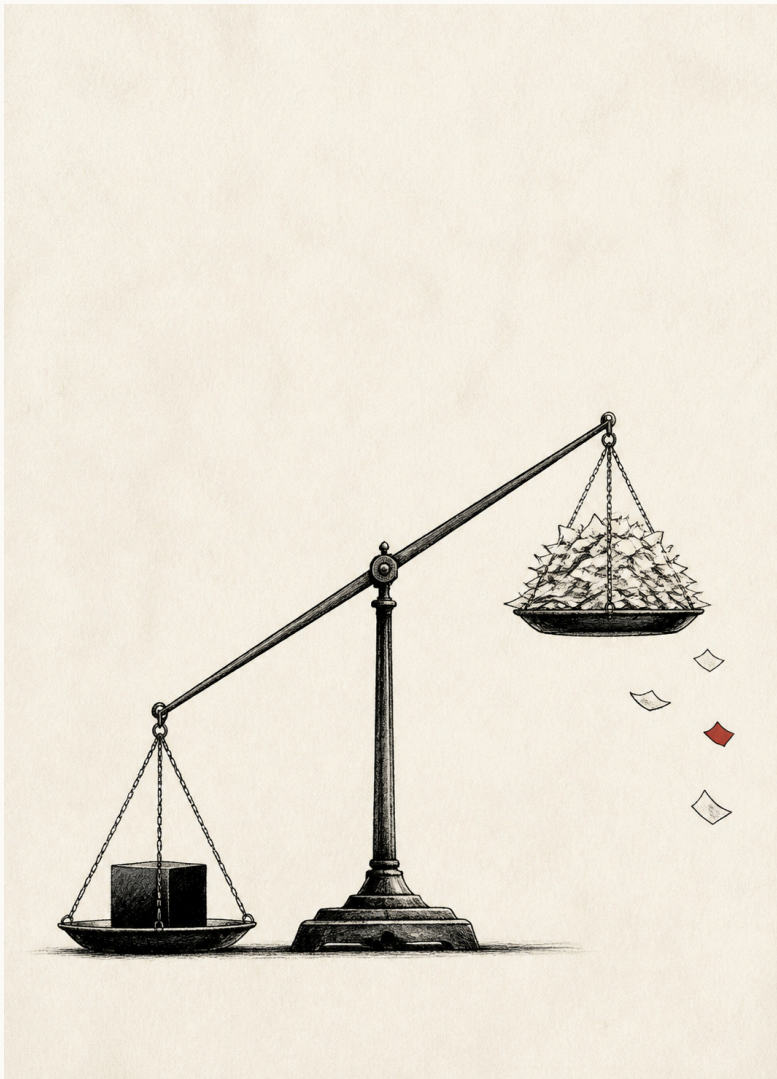
กาน้ำร้อนที่ไม่มีสัญลักษณ์เตือน

ผมนึกถึงเรื่องนี้ตอนที่กำลังจดบันทึกอยู่

ถ้าคุณเดินเข้าครัวในร้านอาหาร คุณจะเห็นสัญลักษณ์เตือนติดอยู่บนอุปกรณ์ที่ร้อน มีดคมจะมีปลอก เครื่องที่มีใบมีดหมุนจะมีฝาครอบที่ต้องปิดก่อนจะกดสวิตช์ได้

สิ่งเหล่านี้ไม่ได้มีไว้ เพราะเขาไม่รู้ว่าอะไรอันตราย เขารู้ดี มันมีไว้เพราะในวันที่เหนื่อยและรีบ ความรู้ไม่พอ

เครื่องมือปรับแต่งส่วนใหญ่ที่ผมอ่าน เหมือนครัวที่ไม่มีอะไรพวกนั้นเลย ทุกอย่างวางเรียงกันบนโต๊ะ กาน้ำร้อนอยู่ข้างแก้วน้ำ และคนใช้ต้องรู้เองว่าอันไหนคืออันไหน



หลักฐานหนึ่งชิ้นที่วัดได้ หนักกว่าคำยืนยันเป็นกอง

ข้อสองและข้อสาม

นอกจากเรื่องการจัดแนกความเสี่ยง ผมยังพบอีกสองรูปแบบที่ซ้ำกันเกือบทุกโครงการ

เรื่องแรกคือการย้อนกลับ เกือบทุกตัวบอกว่าให้สร้างจุดคืนค่าระบบก่อนใช้ บางตัวสร้างให้อัตโนมัติด้วยซ้ำ ฟังดูดี

แต่การสร้างจุดคืนค่ากับการรู้ว่าตัวเองเปลี่ยนอะไรไป เป็นคนละเรื่องกัน

จุดคืนค่าเหมือนการถ่ายรูปห้องทั้งห้องไว้ก่อนจัดของ ถ้าจัดเสร็จแล้วไม่ชอบ คุณย้อนกลับไปรูปรูปนั้นได้ แต่ถ้าคุณชอบเก้าอี้และไมชอบอย่างเดียว คุณจะแก้เฉพาะอย่างเดียวนั้นไม่ได้เลย ต้องย้อนทั้งห้อง

และในทางปฏิบัติ จุดคืนค่าของ Windows ก็ล้มเหลวได้ มันถูกปิดโดยค่าเริ่มต้นในบางเครื่อง มันถูกลบทิ้งเมื่อพื้นที่ไม่พอ และมันกู้ไฟล์บางประเภทไม่ได้

การพึ่งมันอย่างเดียว จึงเป็นการฝากความปลอดภัยทั้งหมด ไว้กับกลไกที่คุณไม่ได้ควบคุม

สิ่งที่ผมอยากได้แทน คือบันทึกว่าเปลี่ยนอะไรไปบ้างทีละรายการ พร้อมค่าเดิมของแต่ละอัน แบบที่ย้อนกลับเฉพาะอันที่ไม่ชอบได้ โดยไม่ต้องย้อนทั้งหมด เหมือนใบเสร็จที่แจกแจงรายการ ไม่ใช่แค่ยอดรวม

ในยี่สิบสามโครงการที่อ่าน มีสองโครงการที่ทำแบบนี้ อีกยี่สิบเอ็ดโครงการพึ่งจุดคืนค่าอย่างเดียว

เรื่องที่สองคือการวัดผล ตัวเลขที่โครงการเหล่านี้อ้างว่าเร็วขึ้นเท่าไร? แทบทั้งหมดมาจากการวัดของคนทำเอง บนเครื่องของเขาเอง โดยไม่มีการควบคุมตัวแปร

ผมไม่ได้บอกว่าเขาโกหก ผมเชื่อว่าเขาวัดได้ตัวเลขนั้นจริง แต่การวัดสองครั้งบนเครื่องเดียวกันแล้วเทียบกัน ไม่ได้บอกว่าความต่างมาจากการปรับแต่ง มันอาจมาจากอย่างอื่นอีกสิบอย่างที่แตกต่างกันระหว่างสองครั้งนั้น

ตัวอย่างที่ชัดที่สุดคือคำอ้างแบบ "ก่อนปรับได้ 180 เฟรม หลังปรับได้ 195 เฟรม" ฟังดูเป็นหลักฐานที่ดี จนกว่าคุณจะถามว่าวัดกี่ครั้ง วัดตอนไหน เครื่องร้อนเท่ากันไหม เกมอัปเดตระหว่างนั้นหรือเปล่า และมีโปรแกรมพื้นหลังตัวไหนเปิดอยู่บ้างในแต่ละครั้ง

คำถามพวกนี้ไม่ใช่การจับผิด มันคือคำถามที่ต้องตอบให้ได้ก่อนจะเรียกตัวเลขนั้นว่าหลักฐาน

ผมจะเล่าเรื่องนี้ให้ละเอียดในบทที่ 7 เพราะพอผมมาออกแบบการวัดของตัวเอง ผมถึงเข้าใจว่ามันยากกว่าที่คิดมาก

คำถามเดียวที่ไม่มีใครถาม

ถ้าให้สรุปสิ่งที่พบจากยี่สิบสามโครงการเป็นประโยคเดียว

เครื่องมือเหล่านี้ตอบคำถามว่า "ทำยังไงให้เร็วขึ้น" ได้ดีมาก แต่ไม่มีตัวไหนตอบคำถามว่า "การกดปุ่มนี้แลगेอะไรไป"

ผมคิดอยู่นานว่าทำไม แล้วได้ข้อสรุปที่ไม่ค่อยน่าพอใจนัก คือเพราะไม่มีใครถาม

คนที่ดาวน์โหลดเครื่องมือพวกนี้มา เขามาด้วยความต้องการเดียว คืออยากให้เกมลื่นขึ้น เขาไม่ได้มาถามว่ามีอะไรต้องแลगेบ้าง คนทำเครื่องมือก็ตอบสิ่งที่ถูกถาม

เครื่องมือพวกนี้ไม่ได้โกหก มันแค่ไม่เคยถูกถาม

ความแตกต่างระหว่างเครื่องมือที่ไม่ดีกับเครื่องมือที่ไม่มีใครถามคำถามที่ถูกคือคนละปัญหากันเลย ข้อแรกแก้ด้วยการเขียนโค้ดใหม่ ข้อที่สองแก้ด้วยการเปลี่ยนคำถามที่ตั้งไว้ตั้งแต่ต้น

พอผมคิดถึงจุดนี้ โจทย์ของผมก็เปลี่ยนไป จากเดิมที่คิดว่าจะสร้างเครื่องมือที่ดีกว่า กลายเป็นการสร้างเครื่องมือที่ตอบคำถามที่ยังไม่มีใครตอบ

ทำไมคำถามนั้นถึงสำคัญกว่าที่ฟัง

คำถามว่า "แลกอะไรไป" ฟังดูเหมือนเรื่องมารยาท เหมือนการติดฉลากคำเตือนไว้เฉย ๆ เพื่อความสบายใจ

แต่มันเปลี่ยนวิธีสร้างเครื่องมือทั้งหมด

ถ้าคุณต้องตอบคำถามนั้นให้ได้ทุกรายการ คุณจะใส่การปรับแต่งที่คุณไม่เข้าใจไม่ได้ เพราะคุณจะเขียนคำอธิบายไม่ออก

คุณจะลกรายการจากที่อื่นมาทั้งหมดไม่ได้ เพราะคุณต้องรู้ว่าแต่ละอันแลกอะไร

คุณจะรวมหลายอย่างไว้ในปุ่มเดียวไม่ได้ ถ้าแต่ละอย่างแลกไม่เหมือนกัน

และคุณจะบอกว่า "ปลอดภัย" ลอย ๆ ไม่ได้ เพราะคำถามบังคับให้ระบุว่าปลอดภัยในแง่ไหน และไม่ปลอดภัยในแง่ไหน

**คำถามเดียวจึงกลายเป็นข้อกำหนดของทั้งระบบ
ไม่ใช่แค่ข้อความบนหน้าจอ**

นี่คือสิ่งที่ผมเอาไปใช้ในบทถัดไปตอนสร้างเกณฑ์ ผมไม่ได้เริ่มจากคำถามว่าเทคนิคไหนดี ผมเริ่มจากคำถามว่าเทคนิคไหนที่ผมอธิบายได้ครบว่ามันแลกอะไร

หกความเชื่อที่ไม่ผ่าน

ระหว่างที่อ่าน ผมเก็บรายการเทคนิคที่พบบ่อยที่สุดไว้ แล้วไล่ตรวจทีละอันว่ามีหลักฐานอะไรรองรับบ้าง

หกอันในนั้นไปไม่ถึงไหนเลย

เรื่อง defrag ไดรฟ์ SSD ผมเล่าไปแล้วในบทที่แล้ว

อีกอันคือการตั้งค่าที่อ้างว่าทำให้ระบบให้ความสำคัญกับเกมมากขึ้น แต่เอกสารทางเทคนิคระบุว่าค่านั้นมีผลเฉพาะในบริบทของการเล่นสโตร์พร้อมกัน ไม่ใช่บริบทที่คำแนะนำอ้างถึง

อีกอันคือการปรับค่าที่เกี่ยวกับการจัดการแบนด์วิดท์เครือข่าย ค่าแนะนำบอกว่าปลดล็อกความเร็วที่ระบบสงวนไว้ 20% ความจริงคือค่านั้นกำหนดเพดานที่แอปพลิเคชันบางประเภทขอได้เองได้ ไม่ใช่ปริมาณที่ระบบกันไว้ไม่ให้ใครใช้ ถ้าไม่มีใครขอ แบนด์วิดท์นั้นก็ว่างให้ใช้อยู่แล้ว

อีกสามอันเป็นทำนองเดียวกัน คือเคยจริงในอดีต หรือจริงในเงื่อนไขที่แคบมาก แล้วถูกเล่าต่อจนกลายเป็นคำแนะนำทั่วไป

จุดร่วมของทั้งหกอันคือไม่มีอันไหนที่พิสูจน์ว่าผิดได้ง่าย ถ้าคุณทำตามแล้วเครื่องไม่เร็วขึ้น คุณจะสรุปว่าเครื่องคุณต่างจากคนอื่น ไม่ใช่ว่าคำแนะนำผิด

คาร์ล ป็อบเปอร์ นักปรัชญาวิทยาศาสตร์ เรียกคุณสมบัตินี้ว่าการพิสูจน์เท็จไม่ได้ เขาเสนอว่าสิ่งที่แยกทฤษฎีวิทยาศาสตร์ออกจากความเชื่อ ไม่ใช่การที่มันพิสูจน์ว่าถูกได้ แต่คือการที่มันเปิดช่องให้พิสูจน์ว่าผิดได้

ข้ออ้างที่ไม่มีทางถูกพิสูจน์ว่าผิด ก็ไม่มีทางถูกพิสูจน์ว่าถูกเหมือนกัน มันแค่ลอยอยู่

และนี่คือเหตุผลที่ผมคิดว่า ความเชื่อกลุ่มนี้อยู่ทนกว่าเทคนิคที่ได้ผลจริงเสียอีก เทคนิคที่ได้ผลจริงมีตัวเลขให้เทียบ มีคนวัดแล้วไม่ตรงแล้วมาโต้กันได้ ส่วนความเชื่อที่วัดไม่ได้ ไม่มีอะไรให้เทียบ มันจึงไม่เคยตาย

ตอนผมตัดท่อนนี้ออก ผมไม่ได้ตัดเพราะพิสูจน์ได้ว่ามันไม่ได้ผล ผมตัดเพราะไม่มีใครพิสูจน์ได้ว่ามันได้ผล สองอย่างนี้เป็นเหตุผลที่ต่างกัน และผมคิดว่าต้องเขียนให้ชัดว่าเป็นแบบไหน

ก่อนไปบทถัดไป

ตอนนี้ผมมีสองอย่างอยู่ในมือ

อย่างแรกคือรายการเทคนิคสองร้อยอัน ที่รู้แล้วว่าแต่ละอันทำอะไรจริง เพราะไล่อ่านไต่ตอมมาแล้ว

อย่างที่สองคือความมั่นใจว่าปัญหาไม่ได้อยู่ที่คนทำเครื่องมือไม่เก่ง แต่อยู่ที่ไม่มีใครวางแผนทว่าอะไรควรอยู่ในเครื่องมือ และอะไรไม่ควร

ปัญหาคือผมยังไม่รู้ว่าเกณฑ์นั้นควรหน้าตายังไง

ตอนแรกผมคิดว่าง่าย แคถามว่า "ปลอดภัยไหม" กับ "ได้ผลไหม" ก็พอ ผมลองใช้สองคำถามนี้กับเทคนิคยีสันแรก แล้วพบว่ามันพังทันที

เทคนิคหลายอันปลอดภัยในเครื่องหนึ่ง แต่ไม่ปลอดภัยในอีกเครื่อง หลายอันได้ผลแต่ได้ผลน้อยจนไม่คุ้มความเสี่ยง และมีอันหนึ่งที่ผมให้คะแนนไปแล้ววันหนึ่ง พอกลับมาดูอีกวันกลับให้คะแนนไม่เหมือนเดิม

ถ้าคนคนเดียวกันให้คะแนนสิ่งเดียวกันไม่ตรงกันในสองวัน เกณฑ์นั้นก็ไม่ใช่เกณฑ์

บทถัดไปคือเรื่องของกาการแก้ปัญหา นั้น

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน การประเมินว่าเครื่องมือปรับแต่งไหนน่าเชื่อถือ ดูจากจำนวนดาวบน GitHub หรือจำนวนคนใช้ก็พอ

หลังอ่าน ต้องอ่านโค้ดจริงถึงจะรู้ว่ามันแลกอะไรไป เพราะคำอธิบายกับโค้ดแยกกันเติบโตได้ และคำแนะนำที่ผิดพิสูจน์เท็จไม่ได้จึงอยู่ทนกว่า

- ✓ รู้ว่าจากยีสันแปดโครงการที่สำรวจ เหลือยีสันสามโครงการที่อ่านโค้ดได้ครบ และยีสันเฮ็ดโครงการในนั้นไม่มีระบบแยกระดับความเสี่ยงเลย
- ✓ เห็นคำถามเดียวที่เปลี่ยนทั้งระบบ คือ การกดปุ่มนั้นแลกอะไรไป และทำไมไม่มีใครถามคำถามนั้นมาก่อน
- ✓ รู้จักแนวคิดการพิสูจน์เท็จไม่ได้ของคาร์ล ป็อบเปอร์ และทำไมความเชื่อที่วัดไม่ได้ถึงอยู่ทนกว่าเทคนิคที่ได้ผลจริง

บทที่ 3 — เกณฑ์ห้ามติ

คำถามที่ 04

จะตัดสินว่าเทคนิคหนึ่งผ่านหรือไม่ผ่าน โดยไม่ให้อารมณ์ของวันนั้นเป็นคนตัดสินแทนได้อย่างไร

ผมให้คะแนนเทคนิคเดียวกันสองครั้ง ห่างกันหนึ่งวัน

วันแรกผมให้ผ่าน มันเป็นการปรับค่าที่เกี่ยวกับการจัดคิวหน่วยความจำ อ่านแล้วดูสมเหตุสมผล มีคนอ้างถึงเยอะ ผมนั่งพักหน้าคนเดียวแล้วดีที่ว่าผ่าน

วันรุ่งขึ้นผมเปิดสมุดจดกลับมาดูรายการเดิมอีกที ไม่ได้ตั้งใจจะทวนซ้ำ แคंबังเอิญเลื่อนผ่าน แล้วผมอ่านข้อมูลเดียวกัน ให้คะแนนไม่ผ่าน

ไม่มีอะไรเปลี่ยน ไม่มีข้อมูลใหม่เข้ามา คนที่ให้คะแนนก็เป็นคนเดียว สิ่งเดียวที่เปลี่ยนคืออารมณ์ตอนนั้น วันแรกผมกำลังรีบเก็บรายการให้ครบ วันที่สองผมกำลังเหนื่อยกับการแก้บั๊กอีกตัว และหงุดหงิดง่ายกว่าปกติ

**ถ้าคนคนเดียวกันให้คะแนนสิ่งเดียวกันไม่ตรงกัน
ในสองวัน นั้นไม่ใช่เกณฑ์ นั่นคือความรู้สึกที่
แปลงตัวมาเป็นตัวเลข**

ทำไมความรู้สึกใช้ไม่ได้

ปัญหาไม่ได้อยู่ที่ผมไม่มีความรู้พอ ผมอ่านเอกสารของค่านั้นมาแล้ว รู้ว่ามันทำอะไรจริง ๆ

ปัญหาคือความรู้สึกอย่างเดียวนั้นพอจะตัดสินว่า "ผ่าน" หรือ "ไม่ผ่าน" เพราะการตัดสินแบบนี้ต้องเทียบหลายมิติเข้าด้วยกันในหัว ต้องชั่งว่าความเสี่ยงกับผลที่ได้ อันไหนหนักกว่า ต้องชั่งว่าหลักฐานที่มีพอไหม แล้วชั่งทั้งหมดนี้พร้อมกันในเวลาไม่กี่วินาที

สมองคนไม่ได้ถูกออกแบบมาให้ชั่งหลายอย่างพร้อมกัน แล้วได้ผลเท่ากันทุกครั้ง วันที่เหนื่อยเรามากให้น้ำหนักกับความเสีียงน้อยลง เพราะอยากให้เรื่องจบเร็ว ๆ วันที่ระแวงเรามากให้น้ำหนักกับความเสีียงมากขึ้น เพราะกลัวว่าจะพลาด

ผมไม่ได้ค้นพบเรื่องนี้เอง มีงานศึกษาของดานชีเกอร์และคณะ ตีพิมพ์ในวารสาร PNAS ปี 2011 ที่ตามดูการตัดสินคำร้องขอทุนที่บน 1,112 คำร้อง แล้วพบรูปแบบที่ชัดเจนน่าอัศจรรย์ อัตราการอนุมัติอยู่ราวหกสิบห้าเปอร์เซ็นต์ตอนเริ่มรอบพิจารณา แล้วลดลงเรื่อย ๆ จนเกือบเป็นศูนย์เมื่อพิจารณาไปหลายคำร้อง และเต่งกลับไปที่หกสิบห้าเปอร์เซ็นต์ทันที หลังผู้พิจารณาได้พักกินอาหาร

คำร้องแบบเดียวกัน ผู้ตัดสินคนเดียวกัน ผลต่างกันตามว่าเพิ่งพักมาหรือยัง

เทียบกับของจริง

สิ่งที่เชื่อ	คนที่มีความรู้และความตั้งใจดี จะตัดสินเรื่องเดิมได้ผลตรงกันทุกครั้ง
สิ่งที่วัดได้	อัตราอนุมัติทบทวนอยู่ราว 65% ตอนเริ่มรอบ ลดลงจนเกือบศูนย์เมื่อพิจารณาไปหลายคำร้อง แล้วดึงกลับไป 65% หลังพักกินอาหาร จากการตามดู 1,112 คำร้อง

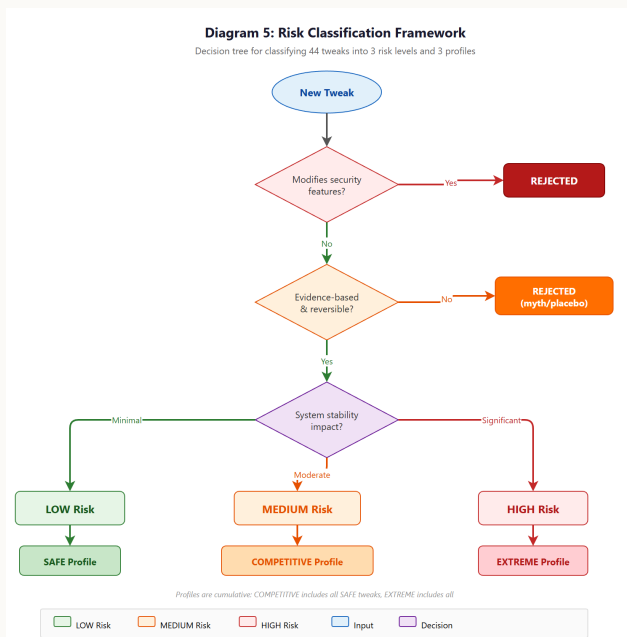
ทางแก้ไขการใช้การพยายามมีสมาธิมากขึ้นหรือพยายามไม่เหนื่อย ทางแก้คือเขียนสิ่งที่ต้องชั่งลงบนกระดาษ แยกออกจากตัวเทคนิคที่กำลังตัดสิน แล้วให้กระดาษนั้นชี้แทนสมองในวันที่เหนื่อย

นั่นคือจุดเริ่มของเกณฑ์ห้ามติ

ห้ามติที่แยกจากกัน

ผมแบ่งการตัดสินออกเป็นห้าเรื่องที่ไม่ทับกัน แต่ละเรื่องตอบคำถามคนละแบบ และแต่ละเรื่องจับสิ่งที่เรื่องอื่นจับไม่ได้

ภาพ 3.1 ต้นไม้ตัดสินใจที่ใช้จัดระดับความเสี่ยงของแต่ละเทคนิค คำถามแรกคือ เทคนิคนี้ไปแตะกลไกป้องกันของระบบหรือเปล่า ถ้าใช่คือจบ ไม่ต้องถามข้ออื่นต่อ ที่วางข้อนี้ไว้บนสุดเพราะไม่มีคะแนนด้านความเร็วเท่าไรที่แลกกับมันได้ ปลายทางทั้งสามคือระดับความเสี่ยงที่กำหนดว่าเทคนิคนั้นจะอยู่ในโปรไฟล์ไหนได้บ้าง



ที่มา: ไดอะแกรมกรอบจำแนกความเสี่ยงของโครงการ ฉบับแก้ตัวเลขแล้ว

ความปลอดภัยตอบคำถามว่า ถ้าเทคนิคนี้พลาดไป ผลเสียที่ตามมากระทบแค่ไหน เทคนิคที่แย่ที่สุดในมิตินี้ คือเทคนิคที่เปิดช่องให้เครื่องเสียหายแบบกู้กลับไม่ได้ หรือเปิดช่องให้โปรแกรมอันตรายเข้ามาได้ง่ายขึ้น

วิศวกรรมตอบคำถามว่า เทคนิคนี้ทำงานตามหลักการที่อธิบายได้จริงไหม หรือเป็นการเดาที่บังเอิญได้ผลในบางเครื่อง มิตินี้จับเทคนิคที่ปลอดภัย แต่ไม่มีเหตุผลรองรับว่าทำไมมันควรได้ผล

หลักฐานตอบคำถามว่า มีใครวัดผลของมันด้วยวิธีที่เชื่อถือได้บ้างไหม หรือมีแต่คนบอกว่ารู้สึกดีขึ้น มิตินี้คือมิติที่ทรงความเชื่อหกข้อที่ผมเล่าไปในบทที่แล้ว ออกไปได้

ความโปร่งใสตอบคำถามว่า ผู้ใช้รู้ครบไหมว่ากำลังแลกอะไร นี่คือคำถามเดียวที่ผมเจอในบทที่แล้วว่าไม่มีใครถาม มิตินี้บังคับให้ทุกเทคนิคต้องมีคำอธิบาย ที่ตรงกับสิ่งที่มันทำจริง

ย้อนกลับได้ตอบคำถามว่า ถ้าทำแล้วไม่ชอบ กลับไปเป็นเดิมได้ง่ายแค่ไหน มิตินี้จับเทคนิคที่เขียนค่าทับลงไป โดยไม่เก็บค่าเดิม แบบที่ผมเจอเยอะมากตอนอ่านยี่สิบสามโครงการ

หำมิตินี้ทับกันบางส่วนก็จริง เทคนิคที่ย้อนกลับไม่ได้ก็มักจะโปร่งใสน้อยไปด้วย แต่จุดสำคัญคือมันไม่ทับกันทั้งหมด มีเทคนิคที่ปลอดภัยสูงแต่ย้อนกลับไม่ได้เลย และมีเทคนิคที่ย้อนกลับง่ายแต่ไม่ปลอดภัยเลย ถ้าใช้แค่มิติเดียว จะพลาดอีกสี่มิติทันที

ลองให้คะแนนจริงหนึ่งเทคนิค

เพื่อให้เห็นว่าหำมิตินี้ทำงานยังไงจริง ๆ ผมขอยกตัวอย่างเทคนิคหนึ่งที่ผมให้คะแนนตอนคัดกรองรอบแรก คือการปรับลำดับความสำคัญของโปรเซสเกมให้สูงกว่าปกติ

มิติความปลอดภัยได้คะแนนสูง เพราะการปรับลำดับความสำคัญไม่ได้แก้ไขไฟร์ระบบ ไม่ได้ปิดฟิเจอร์ป้องกันอะไร ถ้าพลาดที่สุดก็แค่โปรแกรมอื่นตอบสนองช้าลงชั่วคราว

มิติวิศวกรรมก็ได้คะแนนสูง เพราะระบบปฏิบัติการมีเอกสารอธิบายไว้ชัดเจน ว่าลำดับความสำคัญมีผลต่อการจัดคิวยังไง เหตุผลตรวจสอบได้ ไม่ใช่การเดา

มิติหลักฐานได้คะแนนกลาง เพราะมีคนวัดผลจริงในบางกรณี แต่ผลที่ได้ต่างกันมากตามประเภทของเกม และสเปกเครื่อง บางเครื่องเห็นผลชัด บางเครื่องแทบไม่ต่าง

มิติความโปร่งใสได้คะแนนสูง เพราะอธิบายได้ตรงไปตรงมาว่ามันทำอะไร ไม่มีผลข้างเคียงที่ซ่อนอยู่

มิตีย้อนกลับได้คะแนนสูงสุด เพราะการปรับลำดับความสำคัญนี้ใช้ได้เฉพาะตอนโปรเซสยังทำงานอยู่ พอปิดโปรแกรมหรือรีสตาร์ทเครื่อง ค่ากลับไปเป็นปกติเองโดยไม่ต้องทำอะไร

รวมคะแนนทั้งห้ามิติแล้วผ่านเส้นตัด เทคนิคนี้จึงอยู่ในสี่สิบห้าอันที่ผ่าน แต่จุดที่ผมอยากชี้คือ ถ้าใช้ความรู้สึกอย่างเดียวโดยไม่แยกมิติ ผมอาจตัดคะแนนเทคนิคนี้ตกไป เพราะติดใจกับมิติหลักฐานที่ไม่ชัดเจน ทั้งที่อีกสี่มิติสนับสนุนมันอย่างมาก การแยกมิติทำให้เห็นว่าจุดอ่อนหนึ่งจุดไม่ควรจุดทั้งภาพรวมลง ถ้ามิติอื่นแข็งพอ

การทดลองที่ 01

คำถาม	การปรับลำดับความสำคัญของโปรเซสเกมให้สูงกว่าปกติ ผ่านเกณฑ์ห้ามิติไหม
วิธี	ให้คะแนนแยกทีละมิติ ความปลอดภัย วิศวกรรม หลักฐาน ความโปร่งใส ย้อนกลับได้ แล้วรวมตามน้ำหนักที่ตั้งไว้ก่อนเห็นข้อมูล
ผล	ผ่านเส้นตัด เป็น 1 ใน 45 เทคนิคจาก 200 ที่ผ่าน แม้มิติหลักฐานได้คะแนนกลางเพียงมิติเดียว
บทเรียน	จุดอ่อนหนึ่งมิติไม่ควรจุดภาพรวมลง ถ้าอีกสี่มิติสนับสนุนมันมากพอ

น้ำหนักที่ไม่เท่ากัน และคนที่ตัดสิน

ผมให้น้ำหนักความปลอดภัยสามสิบ วิศวกรรมยี่สิบห้า หลักฐานยี่สิบ ความโปร่งใสสิบห้า ย้อนกลับได้สิบ รวมเป็นหนึ่งร้อย

เหตุผลที่ความปลอดภัยหนักที่สุดตรงไปตรงมา ความเสียหายจากมิตินี้กู้กลับไม่ได้บ่อยที่สุด เครื่องพังทั้งเครื่อง ข้อมูลหาย หรือช่องโหว่ที่เปิดไว้ถูกใช้ประโยชน์โดยคนอื่น เทียบกับมิตินอื่นที่ถ้าพลาดก็แค่ไม่ได้ผลตามที่คาด ไม่มีผลเสียเพิ่มเติม

เหตุผลที่ย้อนกลับได้เบาที่สุดก็ตรงไปตรงมาเหมือนกัน ถ้าย้อนกลับไม่ได้แต่เทคนิคนั้นปลอดภัย และมีหลักฐานรองรับ ความเสียหายจากการย้อนกลับไม่ได้ ก็ยังจำกัดกว่าความเสียหายจากมิตินอื่น

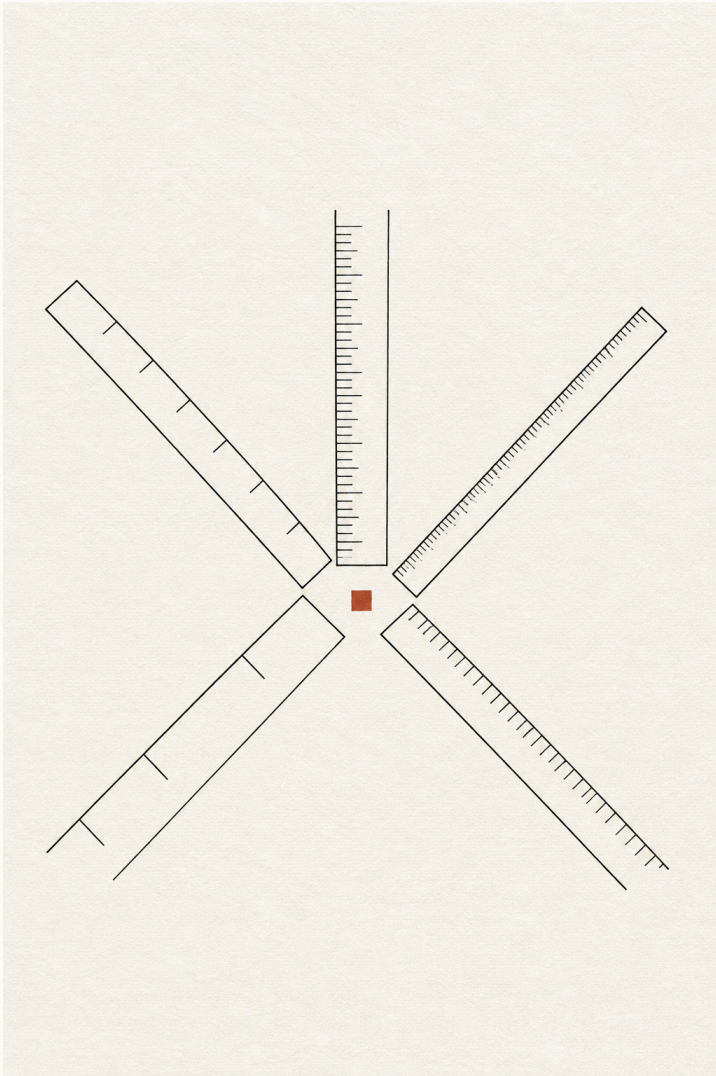
แต่ตัวเลขที่แม่นยำ สามสิบยี่สิบห้ายี่สิบสิบห้าสิบ ผมเป็นคนตั้งเองคนเดียว ไม่มีคนกลุ่มที่สองมาตรวจทานสัดส่วนนี้ ไม่มีการทดสอบว่าถ้าเปลี่ยนน้ำหนักเล็กน้อย ผลจะเปลี่ยนไปมากแค่ไหน

ผมต้องบอกตรง ๆ ว่านี่คือข้อจำกัดของงานนี้ ไม่ใช่เรื่องที่ควรซ่อน ตัวเลขน้ำหนักสะท้อนสิ่งที่ผมให้ความสำคัญ ไม่ใช่ค่าที่พิสูจน์ได้ว่าถูกต้องที่สุด คนอื่นที่ให้ความสำคัญกับมิติต่างออกไป อาจตั้งน้ำหนักต่างจากนี้ แล้วได้รายการเทคนิคที่ผ่านต่างจากที่ผมได้

น้ำหนักสามสิบยี่สิบห้ายี่สิบสิบห้าสิบ เป็นค่าที่ผมตั้งเองคนเดียว ไม่มีคนกลุ่มที่สองตรวจทานสัดส่วนนี้

ถ้าเปลี่ยนน้ำหนักเล็กน้อยแล้วผลเปลี่ยนไปมาก นั่นคือจุดที่งานต่อไปควรทดสอบ

สิ่งที่ผมทำได้คือเขียนน้ำหนักนี้ไว้ให้เห็นชัด ก่อนที่จะเอาไปให้คะแนนเทคนิคใด ๆ เพื่อให้คนอื่นตรวจสอบย้อนกลับได้ ว่าผลที่ออกมาสมเหตุสมผลกับน้ำหนักที่ตั้งไว้หรือเปล่า นี่คือการต่างระหว่างการมีความเห็นกับการปิดบังว่านั่นคือความเห็น



ของชิ้นเดียว วัดด้วยเกณฑ์ต่างกัน ได้คำตอบต่างกันห้าแบบ

เขียนก่อนดูข้อมูล

ส่วนที่ผมยึดมั่นที่สุดในกระบวนการนี้ไม่ใช่ตัวเลขน้ำหนัก แต่คือลำดับของการทำงาน

ผมเขียนเกณฑ์ห้ามติและน้ำหนักทั้งหมดให้เสร็จก่อน แล้วเก็บไว้เป็นเอกสารแยก จากนั้นถึงหยิบสองร้อยเทคนิคที่รวบรวมมาในบทที่แล้ว ขึ้นมาให้คะแนนทีละอัน

ลองนึกภาพถ้าทำสลับกัน สมมติผมดูเทคนิคทั้งสองร้อยอันก่อน รู้อยู่แล้วว่าอันไหนที่ผมชอบ เพราะเคยใช้แล้วรู้สึกว่ดี แล้วค่อยมาเขียนเกณฑ์ทีหลัง ผมจะเขียนเกณฑ์ที่ให้คะแนนเทคนิคที่ผมชอบผ่าน โดยไม่รู้ตัว โดยไม่ได้ตั้งใจ

นี่คือความต่างระหว่างเกณฑ์กับข้ออ้าง

เกณฑ์ที่เขียนหลังเห็นคำตอบ ไม่ใช่เกณฑ์ มันคือข้ออ้าง

การเขียนก่อนไม่ได้แปลว่าผมไม่มีอคติเลย ผมมีอคติแน่นอน ทุกคนมี แต่การเขียนก่อน บังคับให้อคตินั้นแสดงตัวเป็นตัวเลขน้ำหนักที่คนอื่นตรวจได้ ไม่ใช่ซ่อนอยู่ในการเลือกทีละเคส ที่ตรวจไม่ได้ว่าทำไมอันนี้ผ่าน อันนั้นไม่ผ่าน

จากสองร้อยเหลือสี่สิบห้า

พอมิเกนท์และน้ำหนักรบ ผมเอาสองร้อยเทคนิคมาให้คะแนนตามห้ามิติ แต่ละมิติให้คะแนนเป็นสัดส่วนของน้ำหนักเต็ม แล้วรวมกันเป็นคะแนนเดียว

ผมตั้งเส้นตัดไว้ก่อนเริ่มให้คะแนนเหมือนกัน ไม่ใช่ตั้งทีหลังตามที่ทำให้ตัวเลขออกมาสวย เทคนิคที่คะแนนรวมไม่ถึงเส้นตัด ไม่ผ่าน ไม่มีข้อยกเว้นเพราะเทคนิคนั้นดูน่าสนใจหรือมีคนขอมาเยอะ

พอรวมคะแนนครบทุกรายการแล้วนับดู ได้สัดส่วนผ่าน 22.5% ซึ่งตรงกับ 77.5% ที่ไม่ผ่าน ตามที่เล่าไปในบทที่แล้ว

สี่สิบห้าเทคนิคผ่าน จากสองร้อย คิดเป็น 22.5%

ตอนเห็นตัวเลขนี้ครั้งแรก ผมกลับไปนั่งดูรายการที่ไม่ผ่านอีกรอบ ไม่ใช่เพราะสงสัยว่าเกนท์ผิด แต่อยากรู้ว่ามีอันไหนที่น่าเสียดยใหม่ เทคนิคหลายอันที่ไม่ผ่านไม่ใช่เพราะมันแย่ แต่เพราะไม่มีหลักฐานพอ ซึ่งต่างจาก "พิสูจน์แล้วว่าไม่ได้ผล" คนละเรื่องกัน

ผมเขียนสถานะนี้ไว้แยกกันในเอกสาร ไม่ปนกับเทคนิคที่พิสูจน์แล้วว่าไม่ได้ผลจริง เพราะถ้าวันหนึ่งมีหลักฐานใหม่เข้ามา สำหรับเทคนิคกลุ่มที่ขาดหลักฐาน มันควรมีทางกลับมาอยู่ในรายการได้ ในขณะที่กลุ่มที่พิสูจน์แล้วว่าไม่ได้ผลไม่ควรกลับมาอีก

เจ็ดข้อที่ห้ามถาวร

ในสี่สิบห้าเทคนิคที่ผ่านเกณฑ์รวม ยังมีอีกชั้นหนึ่งที่ผมแยกออกมาต่างหาก คือเจ็ดเทคนิคที่ผมตัดสินใจแบนถาวร ไม่ให้เครื่องมือทำได้เลย ไม่ว่าผู้ใช้จะยืนยันกี่รอบก็ตาม

เจ็ดข้อนั้นคือการปิด Windows Defender การปิด UAC การปิดการอัปเดตของ Windows แบบถาวร การปิด DEP การปิด ASLR และ CFG การเปิด testsigning และการเปิด nointegritychecks

ทุกข้อในเจ็ดข้อนี้มีจุดร่วมเดียวกัน คือมันไม่ได้เพิ่มความเสี่ยงให้เครื่องเครื่องเดียว มันเปลี่ยนกฎพื้นฐานที่ระบบใช้ แยกโค้ดที่เชื่อถือได้ออกจากโค้ดที่ไม่ควรรัน เมื่อกฎนั้นถูกปิด ทุกอย่างต่อจากนั้นเชื่อถือไม่ได้อีกต่อไป ไม่ใช่เฉพาะส่วนที่เกี่ยวกับเกม

ผมลองนึกภาพเปรียบเทียบง่าย ๆ กฎอื่นในเครื่องมือเหมือนการล็อกประตูห้องแต่ละห้อง ถ้าล็อกห้องหนึ่งพลาด ห้องอื่นยังปลอดภัยอยู่ แต่เจ็ดข้อนี้เหมือนสัญญาณเตือนไฟไหม้ของทั้งอาคาร ถ้าปิดตัวนี้อาคารทั้งตึกไม่มีทางรู้เลยว่ากำลังไหม้อยู่หรือไม่

ทำไมต้องปฏิเสธแม้ผู้ใช้ขอเอง

ผมคาดไว้ก่อนว่าจะมีคนถามหาเหตุผล ว่าทำไมเครื่องมือของผมไม่ยอมทำตามที่ใช้ต้องการ ในเมื่อเป็นเครื่องของเขา

คำถามนี้ยุติธรรม และผมไม่มีคำตอบที่ทำให้ทุกคนพอใจ แต่ผมมีคำตอบที่ผมยืนอยู่ได้ เครื่องมือนี้ไม่ได้ปฏิเสธ เพราะไม่เชื่อว่าผู้ใช้มีสิทธิ์เลือก ผู้ใช้มีสิทธิ์เต็มที่จะไปปิดค่าเหล่านั้นเองผ่านช่องทางของ Windows โดยตรง ผมไม่มีทางห้ามได้ และไม่ได้พยายามห้าม

สิ่งที่เครื่องมือนี้ปฏิเสธ คือการเป็นตัวกลางที่ทำให้การปิดนั้นง่ายขึ้น เพราะการทำให้มันง่ายขึ้นในหน้าจอเดียวกับที่มีปุ่มปลดถักอื่น ๆ อยู่ เท่ากับเอาเจ็ตชื่อนี้ไปวางเรียงข้างเทคนิคทั่วไป ให้ความรู้สึกว่ามันเป็นระดับเดียวกัน

นี่คือปัญหาเดียวกับที่ผมเจอในบทที่แล้ว ตอนที่ปุ่มปิดแอนิเมชันกับปุ่มปิดการป้องกันหน่วยความจำ อยู่ในรายการเดียวกัน ถ้าผมเอาเจ็ตชื่อนี้มาไว้ในเครื่องมือ แม้จะมีคำเตือนกำกับ ผมก็เป็นส่วนหนึ่งของปัญหา ที่ผมเพิ่งวิจารณ์คนอื่นไปเมื่อบทที่แล้ว

ผมเลยเลือกทางที่ตรงไปตรงมากกว่า ตัดเจ็ตชื่อนี้ออกจากขอบเขตของเครื่องมือทั้งหมด ไม่ใช่ซ่อนไว้ลึก ไม่ใช่ต้องกดหลายชั้นถึงจะเจอ แต่ไม่มีอยู่เลย

เกณฑ์ที่ยังไม่จบ

ผมอยากยอมรับอีกเรื่องหนึ่งก่อนปิดบทนี้ เกณฑ์ห้ามมิติที่ผมเล่ามาทั้งหมด ไม่ใช่เกณฑ์ที่สมบูรณ์แบบและไม่มีวันเปลี่ยน

มันคือเกณฑ์ที่ดีที่สุดที่ผมเขียนได้ในตอนที่ผมยังไม่รู้คำตอบ ตรงนี้แหละที่ผมคิดว่าสำคัญที่สุดของเกณฑ์ ไม่ใช่ความสมบูรณ์แบบ ถ้าวันหนึ่งมีคนพบว่าน้ำหนักไม่ควรเป็นแบบนี้ หรือมีมิติที่หกที่ผมมองไม่เห็น เกณฑ์นี้ก็แก้ได้ ตราบใดที่การแก้ันเกิดก่อนดูข้อมูลชุดใหม่ ไม่ใช่แก้เพื่อให้ผลลัพธ์ที่อยากได้ผ่าน

การมีเกณฑ์แบบนี้เป็นเรื่องหนึ่ง แต่การสร้างเครื่องมือที่ทำตามเกณฑ์นั้นได้จริงทุกครั้ง โดยไม่มีวันที่คนเขียนโค้ดเพลอสิม เป็นอีกเรื่องหนึ่งที่ยากกว่ามาก และนั่นคือสิ่งที่ผมจะเล่าต่อในบทถัดไป

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน การให้คะแนนเทคนิคปรับแต่งคอมพิวเตอร์เป็นเรื่องของความรู้
และความรู้สึกของคนที่ตรวจ

หลังอ่าน ต้องแยกออกเป็นหลายมิติที่ตั้งน้ำหนักไว้ก่อนเห็นข้อมูล ไม่ให้
อารมณ์วันนั้นมาตัดสินแทน

- ✓ รู้จักห้ามติที่ใช้ตัดสินเทคนิค ความปลอดภัย วิศวกรรม หลักฐาน ความ
โปร่งใส ย้อนกลับได้
- ✓ เห็นว่าการเขียนเกณฑ์ก่อนดูข้อมูล ต่างจากการเขียนเกณฑ์หลังเห็นคำตอบ
อย่างไร
- ✓ รู้ว่าเจ็ดเทคนิคถูกแบนถาวรเพราะเหตุผลอะไร แม้ผู้ใช้จะยืนยันก็รอบกก็ตาม

บทที่ 4 — ความปลอดภัยที่ไม่ใช่คำสัญญา

คำถามที่ 05

ออกแบบระบบให้ปลอดภัยไว้แล้ว พอจะเรียกว่า
ปลอดภัยจริงได้เมื่อไหร่

ผมพบข้อบกพร่องนี้ตอนไล่ทดสอบเส้นทางที่ไม่มีใครอยากให้เกิดขึ้น
สั่งให้การสำรองข้อมูลล้มเหลวโดยตั้งใจ แล้วดูว่าโปรแกรมทำอะไรต่อ

สิ่งที่มันควรทำคือหยุดทันที ไม่แตะระบบเลยถ้าไม่มีทางย้อนกลับ

สิ่งที่มันทำจริงคือเดินหน้าต่อ มันเขียน log ไว้ว่าการสำรองข้อมูล
ล้มเหลว แล้วก็ไปแก้ registry ต่อเหมือนไม่มีอะไรเกิดขึ้น

ผมนั่งอึ้งอยู่หน้าจอพักหนึ่ง เพราะนี่ไม่ใช่บั๊กเล็ก ๆ ที่ทำให้ปุ่มดู
เพี้ยนหรือข้อความสะกดผิด นี่คือจุดที่คำสัญญาข้อสำคัญที่สุดของ
เครื่องมือทั้งตัวพังลง คำสัญญาที่ว่า "ทุกการเปลี่ยนแปลงย้อนกลับได้
เสมอ" ใช้ไม่ได้จริงในเส้นทางที่สำคัญที่สุด คือเส้นทางที่ตัวมันเองล้ม
เหลว

ผมเขียนโค้ดส่วนนี้ขึ้นมาเอง ผ่านมันมาหลายรอบ อ่านทวนหลายครั้ง แต่ก็ยังพลาดจุดนี้ไปได้ นี่คือเหตุผลที่บทนนี้ไม่ได้เล่าถึงระบบที่ปลอดภัย เพราะออกแบบมาดี บทนนี้เล่าถึงระบบที่ปลอดภัย เพราะถูกทดสอบจนจับผิดตัวเองได้

ความปลอดภัยที่พึ่งความตั้งใจ หมดอายุเมื่อไหร่

ก่อนจะเล่าว่าผมแก้งังใจ ผมอยากอธิบายว่าทำไมข้อบกพร่องแบบนี้ถึงเกิดขึ้นได้ ทั้งที่ผมตั้งใจสร้างระบบที่ปลอดภัยตั้งแต่ต้น

ปัญหาไม่ได้อยู่ที่ผมไม่รู้ว่ต้องหยุดเมื่อสำรองข้อมูลล้มเหลว ผมรู้ผมเขียนกฎนี้ไว้ในเอกสารตั้งแต่วันแรก ปัญหาคือความรู้ที่อยู่ในหัวคนเขียนโค้ด ไม่ได้แปลว่โค้ดทุกจุดจะทำตามความรู้นั้นเสมอ

คนเขียนโค้ดมีวันที่เหนื่อย มีวันที่รีบส่งงาน มีวันที่แก้ไฟล์หนึ่งแล้วลืมว่มีไฟล์อื่นที่ต้องแก้ตามไปด้วย ถ้าความปลอดภัยของระบบขึ้นอยู่กับว่ คนเขียนโค้ดจำได้ทุกครั้งไหม ว่ต้องเช็คอะไรก่อนทำอะไร วันหนึ่งที่คนเขียนโค้ดเหนื่อยหรือรีบ ความปลอดภัยนั้นก็หายไปพร้อมกัน โดยไม่มีใครรู้ตัวจนกว่าจะสายไปแล้ว

ความปลอดภัยที่พึ่งความตั้งใจของคนเขียน คือ ความปลอดภัยที่หมดอายุตอนคนเขียนเหนื่อย

ทางแก้ที่ผมเลือกไม่ใช้การพยายามไม่เหนื่อยหรือรอบคอบมากขึ้น เพราะนั่นเป็นทางแก้ที่ใช้ไม่ได้ยาว ทางแก้คือสร้างโครงสร้างที่บังคับให้ระบบทำตัวปลอดภัย แม้คนเขียนโค้ดจะเหนื่อยหรือลืมหกก็ตาม ให้โครงสร้างจับผิดแทนสมองของคนคนเดียว

เส้นทางของการเปลี่ยนค่าหนึ่งค่า

ลองนึกภาพผู้ใช้กดปุ่มเดียว ปุ่มที่บอกว่าจะแก้ค่าหนึ่งค่าใน registry เพื่อลดความหน่วงของระบบ ผมจะเล่าว่าค่าสิ่งเดียวนี้อ่านอะไรมาบ้าง ก่อนจะไปถึงจุดที่ค่าถูกเปลี่ยนจริง

ขั้นแรกคือการกรองคำสั่งอันตราย ก่อนจะยอมรันบรรทัดใด ๆ ระบบตรวจสอบข้อความของคำสั่งนั้นก่อน มีสามสิบเก้ารูปแบบที่ขึ้นบัญชีไว้ ว่าเป็นสัญญาณของคำสั่งอันตราย เช่นคำสั่งที่ลบทั้งไดรฟ์ หรือคำสั่งที่ไปวุ่นวายกับหัวเส้นทางในระบบที่ถือว่าอ่อนไหวที่สุด ถ้าคำสั่งตรงกับรูปแบบเหล่านี้ ระบบปฏิเสธทันที ไม่ถามผู้ใช้ ไม่ถามใครทั้งนั้น

สามสิบเก้ารูปแบบอันตรายนี้อยู่ในขั้นแรกสุด ก่อนจะไปถึงขั้นตอนตรวจสอบเข้ากันได้หรือการสำรองข้อมูลเลย

คำสั่งที่ตรงรูปแบบถูกปฏิเสธทันทีโดยไม่ถามผู้ใช้ก่อน

ขั้นที่สองคือการตรวจสอบความเข้ากันได้ ระบบเชื่อว่าเครื่องนี้เป็น Windows รุ่นไหน มีฮาร์ดแวร์แบบไหน ค่าที่จะเปลี่ยนใช้ได้กับเครื่องนี้จริงไหม เพราะบางค่ามีความหมายต่างกันในแต่ละรุ่น Windows หรือไม่มีอยู่เลยในบางรุ่น การเปลี่ยนค่าที่ไม่มีจริงไม่ได้ทำให้เกิดอันตรายทันที แต่มันคือสัญญาณว่าเครื่องมือไม่ได้ตรวจสอบอะไรเลยก่อนลงมือทำ

ขั้นที่สามคือการสำรองข้อมูล ก่อนจะเขียนค่าใหม่ ระบบต้องอ่านค่าเดิมเก็บไว้ก่อน และนี่คือขั้นที่ผมเจอข้อบกพร่องที่เล่าไปตอนต้นบท ผมจะเล่าเรื่องนี้ละเอียดในหัวข้อถัดไป

ขั้นที่สี่คือ Flight Recorder ระบบบันทึกค่าก่อนและหลังการเปลี่ยนแปลงไว้เป็นรายการ พร้อมสร้างสคริปต์ย้อนกลับสำหรับแต่ละรายการโดยอัตโนมัติ ไม่ใช่จุดคืนค่าทั้งระบบแบบที่ผมเจอในโครงการอื่นตอนสำรวจ แต่เป็นบันทึกแบบใบเสร็จรายการต่อรายการย้อนกลับเฉพาะอันที่ต้องการได้ โดยไม่กระทบอันอื่น

ขั้นที่ห้าคือผู้ใช้ยืนยัน ก่อนค่าจะถูกเปลี่ยนจริง หน้าจอต้องแสดงให้ผู้ใช้เห็นว่ากำลังจะเกิดอะไรขึ้น และรอการยืนยันจากผู้ใช้ก่อน ไม่มีการเปลี่ยนแปลงใดที่เกิดขึ้นโดยผู้ใช้ไม่รู้ตัว

ห้าขั้นนี้เรียงกันเหมือนด่านตรวจห้าด่านก่อนขึ้นเครื่องบิน แต่ละด่านตรวจคนละเรื่อง ด่านแรกตรวจว่าไม่ได้พกของอันตราย ด่านที่สองตรวจว่าตัวตรงกับเที่ยวบินจริง ด่านที่สามคือขั้นตอนที่บันทึกว่าใครขึ้นเครื่องบ้าง เพื่อต้องตามหาทีหลัง ด่านที่สี่คือการจดที่นั่งไว้ให้แน่ชัด ด่านที่ห้าคือการเรียกชื่อยืนยันก่อนปิดประตู ถ้าด่านใดด่านหนึ่งไม่ทำงาน ด่านอื่นก็ยังทำหน้าที่ของมันต่อไปได้ นี่คือจุดสำคัญของการออกแบบเป็นขั้น ไม่ใช่การพึ่งจุดเดียวจุดเดียว แล้วถ้าพังก็พังทั้งระบบ

สองภาษา คนละหน้าที่

เครื่องมือนี้เขียนด้วยสองภาษา ไม่ใช่เพราะสะเปะสะปะ แต่เพราะแต่ละภาษาทำหน้าที่ที่อีกภาษาทำได้ไม่ดีเท่า

ส่วนที่แต่ละระบบจริง คือการเขียนค่าลง registry การเปิดปิด service และการแก้การตั้งค่าการบูต เขียนด้วยไฟล์ batch ทั้งหมด ยี่สิบเก้าไฟล์ เหตุผลที่เลือก batch ไม่ใช่เพราะมันทันสมัยหรือมี

พีเจียร์เยอะ ตรงกันข้ามเลย เหตุผลคือมันเรียบงายจนอ่านได้ด้วยตาเปล่า ไม่ต้องติดตั้ง runtime อะไรเพิ่ม เปิดไฟล์ notepad ธรรมดา ก็อ่านได้ทุกบรรทัดว่ามันจะไปแก้อะไร

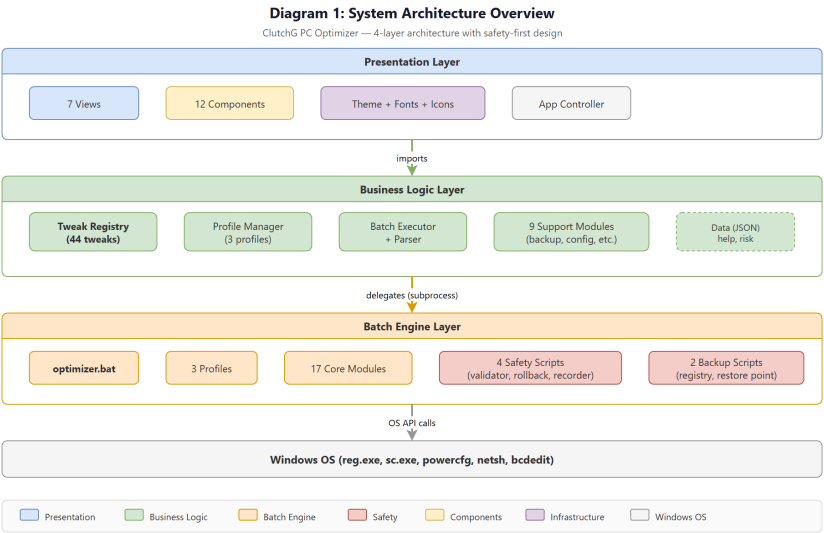
นี่คือคำตอบต่อปัญหาที่ผมเจอตอนสำรวจในบทที่ 2 ตอนที่พบว่าอ่านโค้ดของเครื่องมืออื่นไม่ได้ เพราะมันถูกแปลงให้อ่านไม่รู้เรื่อง ผมไม่อยากให้เครื่องมือของผมเป็นแบบนั้น ใครก็ตามที่สงสัยว่าปุ่มหนึ่งทำอะไรจริง เปิดไฟล์ batch ที่เกี่ยวข้องขึ้นมาอ่านได้เลย ไม่ต้องเชื่อคำอธิบายบนหน้าจออย่างเดียว

ส่วนที่ตัดสินใจว่าจะรันอะไร ตรวจสอบอะไร และแสดงผลยังงัย เขียนด้วย Python หนึ่งร้อยสิบสี่ไฟล์ แบ่งเป็นโค้ดจริงหกสิบไฟล์ และไฟล์ทดสอบห้าสิบสี่ไฟล์ เหตุผลที่เลือก Python ตรงข้ามกับ batch เลย คือ Python ทดสอบได้ง่าย เขียนโค้ดจำลองสถานการณ์แปลก ๆ ได้ เช่นจำลองว่าการสำรองข้อมูลล้มเหลว แล้วเช็คว่ามีโปรแกรมทำตัวถูกไหม

การแบ่งงานแบบนี้แปลว่าชั้นที่แต่ละระบบจริงตรวจสอบได้ด้วยตาคนทั่วไป และชั้นที่ตัดสินใจตรวจสอบได้ด้วยการทดสอบอัตโนมัติซ้ำ ๆ สองอย่างนี้ทำงานคนละแบบ และต้องใช้เครื่องมือคนละชนิด การเอาทั้งสองอย่างมารวมเป็นภาษาเดียว จะได้ทั้งโค้ดที่อ่านยากและทดสอบยากไปพร้อมกัน

เร็วขึ้นจริง หรือแค่รู้สึกว่าร็วขึ้น?

ภาพ 4.2 โครงของระบบทั้งหมด ชั้นบนสุดคือหน้าจอที่ผู้ใช้เห็น ชั้นกลางคือส่วนที่ตัดสินใจว่าจะไร่นได้ ชั้นล่างคือส่วนที่แตะระบบจริง ถูกครเดินทางเดียวจากบนลงล่างเสมอ ชั้นล่างเรียกชั้นบนไม่ได้ ซึ่งเป็นข้อบังคับที่ทำให้ทดสอบชั้นกลางได้โดยไม่ต้องเปิดหน้าจอ



ที่มา: ไดอะแกรมสถาปัตยกรรมของโครงการ ฉบับแก้ตัวเลขแล้ว

ขั้นที่พัง

กลับมาที่ข้อบกพร่องตอนต้นบท ขั้นที่สามคือชั้นสำรวจข้อมูล และเป็นขั้นที่ผมเขียนไว้ว่าถ้าล้มเหลว โปรแกรมต้องหยุด

ปัญหาอยู่ที่ตรงกลางของโค้ด ฟังก์ชันที่เรียกให้สำรวจข้อมูลคืนค่ากลับมาว่าสำเร็จหรือไม่สำเร็จ แต่โค้ดที่เรียกฟังก์ชันนั้นไม่ได้เช็คค่าที่คืนมาก่อนจะเดินหน้าต่อ มันเรียกฟังก์ชันแล้วบันทึก log ตามค่าที่ได้ แต่ไม่ได้ใส่เงื่อนไขว่าถ้าค่าที่ได้คือล้มเหลว ให้หยุดตรงนั้นทันที

นี่คือความต่างระหว่างการมีระบบบันทึกว่าอะไรเกิดขึ้น กับการมีระบบที่บังคับพฤติกรรมตามสิ่งที่เกิดขึ้น log บอกความจริงได้ถูกต้องทุกครั้ง ปัญหาคือความจริงนั้นไม่มีใครทำอะไรกับมัน จนกว่าจะมีคน

มาเปิดอ่าน

เทียบกับของจริง

สิ่งที่เชื่อ	ถ้าโปรแกรมเขียน log บอกว่าการสำรองข้อมูลล้มเหลว ก็แปลว่าโปรแกรมจัดการกับความล้มเหลวนั้นถูกต้องแล้ว
สิ่งที่วัดได้	ทั้งห้าชั้นความปลอดภัยมีจริง แต่ชั้นที่ 3 คือชั้นสำรองข้อมูลบันทึก log ตามค่าที่คืนมาจริง โดยไม่มีเงื่อนไขหยุดเมื่อค่าที่ได้คือล้มเหลว จึงเดินทางไปแก้ registry ต่อ

log บอกความจริงได้ถูกต้องทุกครั้ง ปัญหาคือความจริงนั้นไม่มีใครทำอะไรกับมัน จนกว่าจะมีคนมาเปิดอ่าน

ผมเจอข้อบกพร่องนี้ผ่านการทดสอบที่จงใจบีบให้การสำรองข้อมูลล้มเหลว ไม่ใช่ผ่านการอ่านโค้ดทวนแล้วนี่บอก ผมอ่านโค้ดชุดนี้มาแล้วหลายรอบก่อนหน้านี้ และทุกรอบก็ผ่านไปโดยไม่เห็นปัญหา เพราะโค้ดที่เขียนผิดแบบนี้อ่านแล้วดูเหมือนถูก มันมีการเรียกฟังก์ชันสำรองข้อมูลอยู่จริง มันมี log อยู่จริง สิ่งที่ขาดไปคือเงื่อนไขหยุดที่ไม่มีใครสังเกตว่าไม่มี

ผมแก้ด้วยการเพิ่มเงื่อนไขให้ตรวจค่าที่คืนมาจากฟังก์ชันสำรองข้อมูลทุกครั้ง ถ้าล้มเหลว ระบบยกเลิกการทำงานทั้งหมดทันที ไม่แต่ค่าใด ๆ ในระบบต่อ แล้วแจ้งผู้ใช้งานว่าทำไม่ได้ เพราะสำรองข้อมูลไม่สำเร็จ

สิ่งที่ทำให้แน่ใจว่าจะไม่พังซ้ำ

การแก้จุดเดียวไม่พอ เพราะถ้าผมพลาดจุดนี้ได้ครั้งหนึ่ง ก็มีโอกาสปวดจุดคล้ายกันได้อีกในอนาคต ตอนแก้โค้ดที่อื่น หรือตอนมีคนอื่นมาแก้ต่อ

ผมเขียนการทดสอบสืตัวขึ้นมาเผื่อเรื่องนี้โดยเฉพาะ แต่ละตัวจำลองสถานการณ์ที่การสำรองข้อมูลล้มเหลวในรูปแบบต่างกัน เช่น ล้มเหลวเพราะพื้นที่ดิสก์ไม่พอ ล้มเหลวเพราะไม่มีสิทธิ์เขียนไฟล์ ล้มเหลวเพราะไฟล์ปลายทางถูกใช้งานอยู่ ทุกตัวเช็คอย่างเดียวกันคือ ถ้าสำรองข้อมูลล้มเหลว ต้องไม่มีการเขียนค่าใด ๆ ลงระบบเกิดขึ้นเลย

การทดสอบพวกนี้รันอัตโนมัติทุกครั้งที่มีการแก้โค้ด ถ้าใครมาแก้โค้ดในอนาคต แล้วบังเอิญทำให้เงื่อนไขหยุดนี้หลุดไปอีก การทดสอบจะแจ้งทันทีก่อนที่โค้ดนั้นจะถูกใช้จริง

นี่คือความต่างที่ผมอยากเน้น ระหว่างการแก้บั๊กเฉพาะจุด กับการสร้างเครื่องเผื่อที่จับความผิดพลาดแบบเดียวกันได้ในอนาคตด้วย

**การแก้จุดเดียวแก้ปัญหาที่เจอไปแล้ว การมี
เครื่องเผื่อแก้ปัญหาที่ยังไม่เกิด**

ยี่สิบสี่ service ที่เตะไม่ได้

นอกจากห้าขั้นความปลอดภัยและการแบ่งภาษา ยังมีการป้องกันอีกชั้นหนึ่งที่ผมออกแบบไว้เฉพาะสำหรับ service ของ Windows

Service คือโปรแกรมที่ทำงานอยู่เบื้องหลังตลอดเวลา โดยไม่มีหน้าต่างให้เห็น บางตัวจำเป็นต่อการทำงานพื้นฐานของระบบ บางตัวไม่จำเป็นสำหรับคนเล่นเกม และปิดได้โดยไม่กระทบอะไร คำแนะนำที่ผมเจอตอนสำรวจในบทที่ 2 มักจะรวมทั้งสองกลุ่มนี้ไว้ในรายการเดียวกันให้ปิดหมด

ผมเลือกทางตรงข้าม สร้างรายการ service ยี่สิบสี่ตัวที่เครื่องมือนี้ปฏิเสธจะปิดไม่ว่ากรณีใด ไม่ว่าโปรไฟล์ที่เลือกจะเข้มข้นแค่ไหน รายการนี้ครอบคลุม service ที่เกี่ยวกับความปลอดภัยพื้นฐาน การอัปเดตระบบ และการทำงานที่ Windows เองต้องพึ่งพา

การป้องกันนี้อยู่ในโค้ดโดยตรง ไม่ใช่แค่คำเตือนบนหน้าจอที่ผู้ใช้กดข้ามได้ ถ้าโปรไฟล์ใดพยายามส่งคำสั่งปิด service ที่อยู่ในยี่สิบสี่รายการนี้ คำสั่งนั้นถูกปฏิเสธตั้ง แต่ขั้นแรกที่กรองคำสั่งอันตรายก่อนจะไปถึงขั้นตอนใดต่อไป

ชั้นเสื้อผ้าที่ถอดสวนกันไม่ได้

โปรไฟล์ทั้งสามของเครื่องมือนี้ ไม่ใช่รายการที่แยกจากกันสามชุด แต่เป็นการสะสมกัน SAFE มีสิบสามการปรับแต่ง COMPETITIVE มีสามสิบแปด รวมสิบสามของ SAFE ไว้ในนั้นด้วย แล้วเพิ่มอีกยี่สิบห้า EXTREME มีสี่สิบสี่ รวมสามสิบแปดของ COMPETITIVE แล้วเพิ่มอีกหก

ผมนึกถึงการใส่เสื้อผ้าซ้อนกันหลายชั้นตอนหน้าหนาว เริ่มจากเสื้อในสุดที่บางที่สุด แล้วใส่เสื้อกันหนาวทับ แล้วใส่เสื้อโค้ตทับอีกที ถ้ายากถอดเสื้อในสุดออก ต้องถอดเสื้อโค้ตกับเสื้อกันหนาวก่อน ถอดตัวในสุดโดยไม่แตะตัวนอกไม่ได้ เพราะมันซ้อนกันอยู่

การปรับแต่งของเครื่องมือนี้ก็เป็นแบบนั้น ถ้าเลือก EXTREME ระบบไม่ได้ทำสลิปส์การปรับแต่งแบบสุ่มลำดับ มันทำตามลำดับที่ SAFE ก่อน แล้ว COMPETITIVE แล้วค่อยส่วนเพิ่มของ EXTREME เพราะการปรับแต่งบางอันในขั้นที่ลึกกว่า อ้างอิงถึงค่าที่ขั้นก่อนหน้า ตั้งไว้แล้ว ถ้าสลับลำดับ อาจไปแก้ค่าที่ยังไม่ถูกเตรียมไว้

การออกแบบแบบสะสมนี้มีข้อดีอีกอย่างที่ผมตั้งใจไว้ ผู้ใช้ที่อยากลดความเข้มข้นลง จาก EXTREME กลับมาที่ COMPETITIVE ระบบรู้ว่าต้องย้อนกลับเฉพาะหกการปรับแต่งที่เพิ่มเข้ามาในขั้นนอกสุด ไม่ต้องย้อนกลับทั้งสลิปส์แล้วเริ่มใหม่ นี่คือประโยชน์ของ Flight Recorder ที่เล่าไปก่อนหน้านี้ มันรู้ว่าอะไรถูกเปลี่ยนตอนไหน จึงย้อนกลับเฉพาะส่วนที่ต้องย้อนได้

ความเสี่ยงที่จำแนกไว้ล่วงหน้า

สลิปส์การปรับแต่งที่ execute ได้จริงในเครื่องมือนี้ ทุกอันถูกจำแนกระดับความเสี่ยงไว้ล่วงหน้า ไม่ใช่ปล่อยให้ผู้ใช้เดาเอง สามสลิปส์อยู่ในระดับต่ำ เก้าอยู่ในระดับกลาง และมีเพียงหนึ่งในระดับสูง

ภาพ 4.1 สัดส่วนความเสี่ยงของทั้งสลิปส์รายการ แถบที่ยาวที่สุดคือรายการที่ย้อนกลับได้ง่ายและไม่แตะกลไกป้องกัน ส่วนรายการเดียวที่อยู่ระดับสูงเล็กจนเกือบมองไม่เห็นในภาพ ตรงกับที่ตั้งใจไว้

34

ความเสียดำ
34 รายการ

9

ปานกลาง
9 รายการ

— สูง 1 รายการ

รายการเสี่ยงสูงมีรายการเดียว และอยู่ในโปรไฟล์ที่ต้องกดยืนยันซ้ำ

ที่มา: นับ `risk_level` ใน `tweak_registry.py`

ตัวที่อยู่ในระดับสูงคือการปิด Hypervisor ซึ่งกระทบระบบที่เรียกว่า VBS หรือ Virtualization Based Security เป็นกลไกที่ Windows ใช้แยกส่วนความปลอดภัยสำคัญออกจากส่วนอื่นของระบบ ด้วยเทคโนโลยีจำลองฮาร์ดแวร์ การปิดมันอาจช่วยลดความหน่วงได้จริงในบางสถานการณ์ แต่แลกกับการเปิดช่องให้การป้องกันบางชั้นทำงานได้น้อยลง

เทียบกับของจริง

สิ่งที่เชื่อ	การปรับแต่งทุกอย่างในเครื่องมือปรับแต่งเกม มีความเสี่ยงระดับใกล้เคียงกัน
สิ่งที่วัดได้	จาก 44 การปรับแต่งที่ execute ได้จริง 34 อยู่ระดับต่ำ 9 อยู่ระดับกลาง และมีเพียง 1 ที่อยู่ระดับสูง คือการปิด Hypervisor

การปรับแต่งนี้ไม่ได้อยู่ในเจดข้อที่แบนถาวร ตามที่เล่าไปในบทที่แล้ว เพราะมันย้อนกลับได้ และผลกระทบไม่ได้กว้างเท่าเจดข้อนั้น แต่มันถูกจำกัดไว้เฉพาะโปรไฟล์ EXTREME เท่านั้น พร้อมคำเตือนที่ชัดเจนกว่าการปรับแต่งอื่น และต้องผ่านการยืนยันจากผู้ใช้อย่างเจาะจง แยกจากการยืนยันโปรไฟล์โดยรวม

นี่คือตัวอย่างของการที่ความโปร่งใสไม่ใช่แค่คำอธิบายบนหน้าจอ แต่คือการออกแบบว่าอะไรควรทำให้ผู้ใช้ต้องหยุดคิดนานกว่าปกติก่อนกด

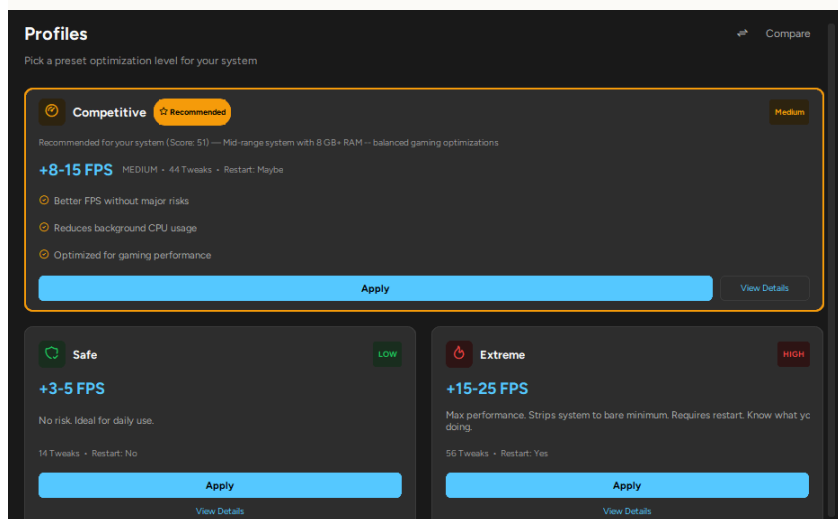
หน้าจอที่ตัดสินใจแทนผู้ใช้ไปแล้วครึ่งหนึ่ง

เรื่องที่ผมยังไม่ได้เล่าคือความปลอดภัยส่วนหนึ่งไม่ได้อยู่ในโค้ด มันอยู่ในหน้าจอ

ผมพบเรื่องนี้จากการนั่งดูคนอื่นใช้เครื่องมือแบบเดียวกัน คนที่ไม่รู้ว่าปุ่มไหนอันตราย จะกดปุ่มที่อยู่ใกล้มือที่สุด ปุ่มที่ใหญ่ที่สุด หรือปุ่มที่มีคำว่าแนะนำติดอยู่

ซึ่งแปลว่าคนออกแบบหน้าจอเป็นคนเลือกให้เขาไปแล้ว ตั้งแต่ก่อนเขาจะกด

ภาพ 4.3 หน้าเลือกโปรไฟล์ ตัวเลือกกลางที่มีกรอบสีและป้ายแนะนำคือตัวที่ระบบเลือกให้ตามคะแนนเครื่อง ส่วนตัวเลือกที่แรงที่สุดวางอยู่ขวาล่างในกรอบเดียวกับตัวที่ปลอดภัยที่สุด ไม่ได้เด่นกว่ากัน ภาพนี้ถ่ายก่อนการตรวจสอบซ้ำในบทที่ 6 ตัวเลขบนหัวจอจึงยังเป็นห้าสิบหก



ที่มา: ภาพหน้าจอจากโปรแกรมจริง ถ่ายเมื่อ 2026-06-16 ครอบเฉพาะพื้นที่การ์ดโปรไฟล์

สิ่งที่ผมตัดสินใจไว้ในหน้านี้มีสามข้อ

ข้อแรก ตัวเลือกที่ระบบแนะนำถูกเลือกจากคะแนนของเครื่องผู้ใช้เอง ไม่ใช่ตัวที่แรงที่สุดเสมอไป เครื่องที่คะแนนต่ำจะได้รับคำแนะนำที่เบากว่า

ข้อสอง ตัวเลือกที่แรงที่สุดไม่ได้ถูกทำให้ดูน่ากดกว่าตัวอื่น มันมีขนาดเท่ากับตัวที่ปลอดภัยที่สุด อยู่ในแถวเดียวกัน ต่างกันแค่ป้ายระดับความเสี่ยงมุมขวามือ

ข้อสาม ทุกตัวเลือกบอกจำนวนรายการที่จะถูกเปลี่ยน และบอกว่าต้องรีสตาร์ทเครื่องหรือไม่ ก่อนกด ไม่ใช่หลังกด

ข้อที่สามฟังดูเล็กที่สุด แต่เป็นข้อที่ผมคิดว่าสำคัญที่สุด เพราะคำถามว่าต้องรีสตาร์ทไหม คือคำถามที่ตัดสินว่าผู้ใช้จะกดตอนนี้หรือรอ และการรู้คำตอบหลังกดไปแล้วไม่มีประโยชน์อะไรเลย

ปุ่มเดียวที่ทำหลายอย่าง กับปุ่มหลายปุ่มที่ทำอย่างเดียว

มีการตัดสินใจอีกข้อที่ผมลังเลอยู่นาน

ผู้ใช้ส่วนใหญ่ไม่อยากจะเลือกทีละรายการ เขาอยากกดปุ่มเดียวจบ แต่ปุ่มเดียวที่ทำหลายอย่างคือปุ่มที่ผู้ใช้ไม่รู้ว่ามันทำอะไรบ้าง ขัดกับทุกอย่างที่ผมเขียนมาในบทนี้

ทางออกที่ผมเลือกคือมีทั้งสองแบบ แต่แยกกันคนละหน้า

หน้าหนึ่งเป็นชุดสำเร็จรูปที่กดครั้งเดียวจบ อีกหน้าเป็นรายการทั้งหมดที่เลือกทีละอันได้ และชุดสำเร็จรูปทุกชุดบอกไว้ตรงหน้าปุ่มว่าประกอบด้วยกี่รายการ และอยู่ระดับความเสี่ยงไหน

คนที่ไม่อยากรู้ ไม่ต้องรู้ แต่คนที่อยากรู้ ไม่ต้องหา

ความปลอดภัยที่พิสูจน์ ไม่ใช่ความปลอดภัยที่ประกาศ

ผมเล่าเรื่องข้อบกพร่องในบทนี้ไม่ใช่ เพื่อให้เครื่องมือดูแย่ ตรงข้ามเลย ผมคิดว่าการที่ระบบจับข้อบกพร่องแบบนี้ได้ก่อนถึงมือผู้ใช้ คือหลักฐานว่ากระบวนการทดสอบทำงานตามที่ควร

แต่มันก็เตือนผมอีกเรื่องหนึ่งที่สำคัญกว่า การที่ผมออกแบบห้าชั้นความปลอดภัยไว้อย่างรอบคอบ ไม่ได้แปลว่าทั้งห้าชั้นทำงานถูกต้องจริงตั้งแต่แรก ชั้นที่สามพังไปแล้วครั้งหนึ่งโดยที่ผมไม่รู้ตัว จนกว่าจะทดสอบมันโดยตรง

การออกแบบที่ดีคือจุดเริ่มต้น ไม่ใช่จุดจบ สิ่งที่พิสูจน์ว่าการออกแบบนั้นใช้ได้จริง คือการพยายามทำให้มันพังด้วยมือตัวเอง ก่อนที่คนอื่นจะเจอ

นี่คือสิ่งที่ผมจะเล่าต่อในบทถัดไป เพราะออกแบบไว้ดีแค่ไหน ก็ยังต้องพิสูจน์ว่ามันทำงานจริงอยู่ดี และการพิสูจน์นั้นไม่ใช่แค่ดรีนแล้วดูว่ามันไม่พัง มันคือการพยายามทำให้ของตัวเองพังให้ได้มากที่สุด ก่อนที่จะยอมเชื่อว่ามันไม่พัง

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน ระบบที่ออกแบบมาปลอดภัยตั้งแต่แรก น่าจะปลอดภัยจริงตามที่ออกแบบไว้

หลังอ่าน การออกแบบที่ดีเป็นแค่จุดเริ่มต้น ต้องทดสอบให้พังด้วยมือตัวเองก่อน จึงจะเรียกว่าปลอดภัยจริง

- ✓ รู้จักห้าชั้นความปลอดภัยที่คำสั่งหนึ่งต้องผ่านก่อนแก้ registry จริง
- ✓ เห็นว่าข้อบกพร่องในชั้นสำรองข้อมูลถูกจับได้จากการทดสอบ ไม่ใช่จากการอ่านโค้ดทวน
- ✓ รู้ว่าทำไมยี่สิบสี่ service และการปิด Hypervisor ถูกจัดการต่างระดับกัน

บทที่ 4ก — กระบวนการที่มองไม่เห็น

คำถามที่ 06

ทำไมงานคนเดียวถึงต้องมีเอกสารเยอะขนาดนั้น

มีคำถามหนึ่งที่ผมโดนถามบ่อยกว่าที่คิด และทุกครั้งผมตอบได้ไม่ค่อยดี

คำถามคือ ทำไมงานคนเดียวถึงต้องมีเอกสารเยอะขนาดนั้น

คำถามนี้ฟังดูเหมือนคำถามเรื่องความขี้เกียจ แต่จริง ๆ มันเป็นคำถามที่ดีมาก เพราะคนที่ถามกำลังชี้ไปที่สิ่งที่ดูเหมือนงานส่วนเกิน ไม่มีใครใช้ ไม่มีใครอ่าน และไม่ได้ทำให้โปรแกรมทำงานได้ดีขึ้นแม้แต่นิดเดียว

ผมเคยคิดแบบนี้เหมือนกัน

เรื่องที่เปลี่ยนความคิดผมไม่ใช่ตำรา แต่เป็นเช้าวันหนึ่งที่ผมเปิดโค้ดของตัวเอง ที่เขียนไว้เมื่อสัปดาห์ก่อน แล้วอ่านไม่ออกว่าทำไมถึงเขียนแบบนี้

เขียนให้ตัวเองในอีกหกเดือน

โค้ดบรรทัดนั้นตรวจสอบเงื่อนไขอยู่สามข้อก่อนจะยอมรับคำสั่ง ผมจำได้ว่าเขียนเอง แต่จำไม่ได้ว่าข้อที่สามมีไว้ทำไม มันดูเหมือนซ้ำกับข้อที่สอง

ผมนั่งดูอยู่ยี่สิบนาที คิดว่าจะลบทิ้ง

แล้วผมไปเจอบันทึกที่เขียนไว้ตอนออกแบบ ในนั้นมีประโยคเดียวที่อธิบายว่าข้อที่สามกันอะไร มันกันกรณีที่ผู้ใช้กดปุ่มสองครั้งซ้อนเร็วๆ ซึ่งข้อที่สองจับไม่ได้เพราะมันตรวจคนละจังหวะ

ถ้าไม่มีบันทึกนั้น ผมจะลบมันทิ้ง แล้วบั๊กที่เคยแก้ไปแล้วจะกลับมา

นี่คือคำตอบแรกของผมสำหรับคำถามว่าทำไมคนเดียวถึงต้องมีเอกสาร

**คนที่คุณเขียนเอกสารให้ ไม่ใช่คนอื่น มันคือตัว
คุณเองในอีกหกเดือน**

มาตรฐานสำหรับคนที่ไม่มีก็คน

พอผมยอมรับว่าต้องเขียนเอกสาร คำถามถัดมาคือเขียนอะไรบ้าง

มาตรฐานที่สอนกันในวิชาชีพวิศวกรรมซอฟต์แวร์ส่วนใหญ่ ออกแบบมาสำหรับองค์กรที่มีคนหลายสิบคน มีทีมทดสอบแยก มีคนดูแลเอกสารโดยเฉพาะ การเอามาใช้กับงานคนเดียวจะได้เอกสารกองใหญ่ที่ไม่มีใครอ่าน รวมทั้งคนเขียน

มีมาตรฐานอยู่ตัวหนึ่งชื่อ ISO/IEC 29110 เขียนขึ้นสำหรับสิ่งที่เรียกว่าองค์กรขนาดเล็กมาก หมายถึงหน่วยงานที่มีคนไม่เกินยี่สิบห้าคน และครอบคลุมถึงงานที่มีคนเดียวด้วย

สิ่งที่ผมชอบเกี่ยวกับมาตรฐานตัวนี้ ไม่ใช่ว่ามันบังคับอะไร แต่คือมันบอกว่าอะไรที่ไม่ต้องทำ

มันไม่บังคับให้มีการประชุมทบทวนแบบเป็นทางการ ไม่บังคับให้มีคนตรวจสอบอิสระ ไม่บังคับให้มีระบบติดตามข้อบกพร่องแยกต่างหาก สิ่งเหล่านี้มีค่าในองค์กรใหญ่ แต่ในงานคนเดียวมันกลายเป็นพิธีกรรมที่ทำ เพื่อให้มีหลักฐานว่าทำ

สิ่งที่มันบังคับคือของที่ต้องได้ออกมาจากแต่ละขั้น เรียกว่า work product

ภาพ 4ก.1 เจ็ดขั้นของการพัฒนาที่ใช้จริงในโครงการนี้ กับเอกสารที่ต้องได้จากแต่ละขั้น จุดสำคัญไม่ได้อยู่ที่ลำดับ เพราะทุกตำราเรียงคล้ายกันหมด แต่อยู่ที่คอลัมน์ขวา ขั้นที่ไม่มีของออกมา คือขั้นที่พิสูจน์ทีหลังไม่ได้ว่าทำไปแล้วจริง

- 1 **เก็บความต้องการ**
SRS — ข้อกำหนดที่ตรวจสอบได้
- 2 **วางแผน**
Project Plan — ขอบเขต เวลา ความเสี่ยง
- 3 **ออกแบบ**
SDD — โครงสร้าง และเหตุผลที่แบ่งแบบนี้
- 4 **แต่งงาน**
Task list — งานย่อยที่ผูกกับข้อกำหนด
- 5 **เขียนโค้ด**
Source code + unit test ที่เขียนคู่กัน
- 6 **ทดสอบ**
Test Plan, Test Record — ผลรันที่บันทึกไว้
- 7 **ปิดงาน**
Closure Report, Traceability Record

ขั้นที่ไม่ได้ออกเอกสารออกมา คือขั้นที่พิสูจน์ทีหลังไม่ได้ว่าทำแล้ว

ที่มา: ISO/IEC 29110 VSE Basic Profile กับ work product ที่มีจริงในโครงการ

ทำไมต้องเป็นของที่จับต้องได้

ความต่างระหว่าง "ทำแล้ว" กับ "มีของออกมา" ฟังดูเหมือนเรื่องเล็ก แต่มันคือความต่างทั้งหมด

คำว่า work product ในบทนี้ หมายถึงของที่จับต้องได้ ตรวจสอบได้ ไม่ใช่คำพูดที่บอกว่าทำแล้ว

ความต่างระหว่าง "ทำแล้ว" กับ "มีของออกมา" ฟังดูเหมือนเรื่องเล็ก แต่มันคือความต่างทั้งหมด

ถ้าผมบอกว่าออกแบบระบบแล้ว ไม่มีใครตรวจสอบคำพูดนั้นได้ ถ้าผมบอกว่าออกแบบแล้ว และนี่คือเอกสารออกแบบที่ระบุว่าระบบมีกี่ชั้น แต่ละชั้นทำอะไร และทำไมถึงแบ่งแบบนั้น คนอ่านตรวจได้ทันทีว่าเหตุผลที่ผมให้ฟังขึ้นหรือเปล่า

ผมชอบเปรียบเทียบกับการทำอาหารในครัวร้าน เชฟที่บอกว่าเตรียมของครบแล้ว กับเชฟที่ชี้ให้ดูวัตถุดิบที่หั่นเรียบร้อยวางเรียงไว้ สองอย่างนี้ต่างกันตรงที่อย่างหลังตรวจสอบได้ โดยไม่ต้องเชื่อใคร และตรวจได้ตอนที่ยังแก้ทัน ไม่ใช่ตอนที่ลูกค้าสั่งแล้ว

ในโครงการนี้ เอกสารที่ต้องมีตามมาตรฐานมีสิบฉบับ ผมเขียนครบทั้งสิบ

เทียบกับของจริง

สิ่งที่เชื่อ	งานคนเดียวไม่จำเป็นต้องมีเอกสารมากมายเหมือนโครงการใหญ่
สิ่งที่ทำได้	โครงการนี้เขียนเอกสารครบ 10 ฉบับตามมาตรฐาน และกลับไปอ่านเอกสารออกแบบของตัวเองอย่างน้อย 10 ครั้งระหว่างพัฒนา

แต่ตัวเลขสิบไม่ใช่ประเด็น ประเด็นคือแต่ละฉบับตอบคำถามที่ต่างกัน คนละคำถาม และคำถามเหล่านั้นเป็นคำถามที่ผมต้องตอบอยู่ดี ไม่ว่าจะมีมาตรฐานหรือไม่

แผนโครงการตอบว่างานนี้จะเสร็จเมื่อไหร่ และมีอะไรบ้างที่อาจทำให้ไม่เสร็จ ข้อกำหนดตอบว่าโปรแกรมนี้ต้องทำอะไรได้ และจะรู้ได้ยังไงว่ามันทำได้แล้ว เอกสารออกแบบตอบว่าโครงข่ายในเป็นยังไง และทำไม แผนทดสอบตอบว่าจะพิสูจน์ยังไงว่ามันทำงาน บันทึกผลทดสอบตอบว่าพิสูจน์แล้วได้ผลยังไง

สิ่งที่มาตรฐานไม่ได้ให้

ผมอยากพูดถึงด้านที่คนมักไม่เล่า

การทำตามมาตรฐานไม่ได้ทำให้โปรแกรมดีขึ้นโดยอัตโนมัติ มันไม่ได้จี้บักให้ ไม่ได้ทำให้โค้ดเร็วขึ้น และไม่ได้บอกว่าการออกแบบของคุณฉลาดหรือโง่

สิ่งที่มันทำคือทำให้ความผิดพลาดถูกจับได้เร็วขึ้น เพราะทุกชั้นมีของออกมาให้ตรวจ

กระดาศที่ไม่ได้ทำให้ช้าลง

มีความเข้าใจผิดข้อหนึ่งที่ผมอยากแก้

คนมักคิดว่าการทำตามกระบวนการทำให้งานช้าลง ผมเคยคิดแบบนั้น และผมพบว่ามันจริงครั้งเดียว

มันช้าลงจริงในสัปดาห์แรก เพราะต้องเขียนสิ่งที่ปกติเก็บไว้ในหัว

แต่มันเร็วขึ้นในเดือนที่สาม เพราะตอนนั้นสิ่งที่เก็บไว้ในหัวเริ่มล้น และคนที่ไม่ได้เขียนไว้จะเริ่มตัดสินใจเข้าเรื่องที่เคยตัดสินใจไปแล้ว โดยไม่รู้ว่เคยตัดสินใจ

ผมนับได้ว่าตัวเองกลับไปอ่านเอกสารออกแบบของตัวเอง อย่างน้อยสิบครั้งระหว่างเขียนโปรแกรม ทุกครั้งเป็นตอนที่กำลังจะเพิ่มอะไรบางอย่าง แล้วไม่แน่ใจว่มันควรอยู่ชั้นไหน

ถ้าไม่มีเอกสารนั้น ผมจะเดา และการเดาสิบครั้งจะได้คำตอบที่ไม่เหมือนกันสิบแบบ สุดท้ายก็จะกลายเป็นโปรแกรมที่ไม่มีโครงสร้าง แม้จะเริ่มต้นด้วยการออกแบบที่ดี

บทเรียนที่เอาไปใช้ได้

ถ้าให้ผมสรุปเรื่องนี้เป็นสิ่งที่คนอ่านเอาไปใช้ได้จริง ผมจะสรุปแบบนี้

อย่าเริ่มจากการถามว่าต้องเขียนเอกสารกี่ฉบับ ให้เริ่มจากการถามว่ามีคำถามอะไรบ้างที่คุณจะต้องตอบซ้ำ ๆ ตลอดโครงการ แล้วเขียนคำตอบของคำถามเหล่านั้นลงที่ที่หาเจอ

คำถามพวกนั้นมักเป็นคำถามเดิม ๆ ทุกโครงการ งานนี้จะเสร็จเมื่อไหร่ ต้องทำอะไรได้บ้าง ช่างในเป็นยังไง จะรู้ได้ยังไงว่ทำงานจริงเปลี่ยนอะไรไปบ้างแล้ว

พอเขียนคำตอบครบ คุณจะพบว่าคุณมีเอกสารตามมาตรฐานอยู่แล้วโดยไม่ได้ตั้งใจ

และถ้าวันหนึ่งมีคนถามว่าทำไมงานคนเดียวถึงต้องมีเอกสารเยอะขนาดนั้น คำตอบที่ผมจะให้คือ มันไม่ได้เยอะ มันแค่เป็นคำตอบของคำถามที่ยังไงก็ต้องตอบ ที่เขียนไว้ก่อนจะลืมนั่นเอง

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน เอกสารในงานคนเดียวเป็นภาระที่ไม่มีใครอ่าน

หลังอ่าน เอกสารคือคำตอบของคำถามที่ต้องตอบซ้ำอยู่ดี เขียนไว้ก่อนจะลืมน

- ✓ รู้ว่ามาตรฐาน ISO/IEC 29110 ไม่บังคับพิธีกรรม แต่บังคับของที่ต้องได้ออกมา
- ✓ รู้ว่าข้อกำหนดที่วัดไม่ได้ ตรวจสอบไม่ได้ว่าทำเสร็จหรือยัง
- ✓ รู้ว่าเอกสารสืบค้นคำตอบคำถามคนละคำถามที่ยังไงก็ต้องตอบ

บทที่ 4ข — เอกสารที่เขียนก่อนโค้ด

คำถามที่ 07

เอกสารตามมาตรฐานที่เขียนไว้จริง หน้าตาเป็นยังไง และพิสูจน์อะไรได้บ้าง

บทที่แล้วเล่าว่าทำไมต้องมีเอกสาร บทนี้เล่าว่าเอกสารพวกนั้นหน้าตาเป็นยังไงจริง ๆ

ผมตั้งใจเล่าแบบเปิดของให้ดู เพราะตอนผมเริ่มต้น สิ่งที่ยากที่สุดไม่ใช่คำอธิบายว่าเอกสารแต่ละชนิดคืออะไร แต่คือตัวอย่างจริงที่ไม่ใช่แบบฝึกหัดในตำรา

ข้อกำหนดที่ตรวจสอบได้ กับข้อกำหนดที่ฟังดูดี

เอกสารฉบับแรกที่ต้องเขียนคือข้อกำหนด มันตอบคำถามว่าโปรแกรมนี้ต้องทำอะไรได้บ้าง

ในโครงการนี้มีข้อกำหนดหกสิบเก้าข้อ แต่ละข้อมีรหัสของตัวเอง

ผมจะยกตัวอย่างข้อหนึ่งมาให้ดูทั้งข้อ ข้อที่ชื่อว่า FR-SD-01 เขียนว่า ระบบต้องตรวจจับหน่วยประมวลผลได้ ประกอบด้วยชื่อรุ่น ผู้ผลิต จำนวนแกน จำนวนเธรด และความเร็วพื้นฐาน โดยข้อมูลที่แสดงต้องคลาดเคลื่อนจากค่าจริง ไม่เกินห้าเปอร์เซ็นต์

สังเกตประโยคสุดท้าย ตัวเลขห้าเปอร์เซ็นต์คือสิ่งที่ทำให้ข้อกำหนดนี้ ต่างจากข้อกำหนดที่เขียนว่า "ระบบต้องตรวจจับฮาร์ดแวร์ได้ถูกต้อง"

ข้อกำหนดแบบหลังฟังดูดีกว่า เขียนง่ายกว่า และไม่มีใครเถียงได้ แต่มันมีปัญหาข้อเดียวคือไม่มีวันรู้ว่ทำเสร็จหรือยัง

ถูกต้องแค่ไหนถึงเรียกว่าถูกต้อง ถ้าโปรแกรมอ่านความเร็วซีพียูได้ 3.59 กิกะเฮิร์ตซ์ แต่ของจริงคือ 3.60 นี่คือถูกหรือผิด

ถ้าไม่มีเลขกำกับ คำตอบจะขึ้นกับอารมณ์ของคนตอบในวันนั้น

รหัสที่ทำให้ตามรอยได้

เรื่องที่ผมคิดว่าน่าสนใจกว่าตัวข้อกำหนด คือสิ่งที่เกิดขึ้นเมื่อทุกข้อมีรหัส

พอทุกข้อกำหนดมีรหัส มันเชื่อมกับของอื่นได้ ผมทำตารางหนึ่งที่ผูกส่อย่างเข้าด้วยกัน ข้อกำหนด ส่วนของระบบที่รับผิดชอบ ไฟล์และบรรทัดของโค้ดจริง และรหัสของการทดสอบที่พิสูจน์ว่มันทำงาน

แถวหนึ่งในตารางนั้นหน้าตาแบบนี้ ข้อกำหนด FR-SD-01 เรื่องการตรวจจับหน่วยประมวลผล รับผิดชอบโดยส่วนที่ชื่อ SystemDetector อยู่ในไฟล์ system_info.py บรรทัดที่ 172 ถึง

222 พิสูจน์ด้วยการทดสอบรหัส UT-SD-01

อีกแถวหนึ่ง ข้อกำหนด FR-PM-01 เรื่องไฟล์สำเร็จรูปสามชุด
รับผิดชอบโดยส่วนที่ชื่อ ProfileManager อยู่ในไฟล์
profile_manager.py บรรทัดที่ 61 ถึง 128 พิสูจน์ด้วยการทดสอบ
รหัส UT-PM-01

ตารางแบบนี้ตอบคำถามได้สองทาง

ทางแรก ถ้ามีคนถามว่าข้อกำหนดข้อนี้ทำแล้วหรือยัง ผมชี้ไปที่
บรรทัดของโค้ดกับผลการทดสอบได้ทันที ไม่ต้องเปิดหาทั้งโปรเจกต์
ทางที่สอง ซึ่งเป็นทางที่มีประโยชน์กว่า ถ้าผมกำลังจะแก้โค้ด
บรรทัดนั้น ผมย้อนกลับไปดูได้ว่ามันรับผิดชอบข้อกำหนดข้อไหนอยู่
และการแก้ไขของผมจะทำให้ข้อกำหนดนั้นเสียหายไหม

ช่องว่างที่ไม่ปิด

ในหลักสิบเก้าข้อกำหนดนั้น มีหลักสิบสองข้อที่มีการทดสอบอัตโนมัติ
รองรับ

เหลืออีกเจ็ดข้อที่ไม่มี

เทียบกับของจริง

สิ่งที่เชื่อ เอกสารข้อกำหนดที่เขียนไว้ครบตามมาตรฐาน แปลว่า
ทุกข้อพิสูจน์ได้จริงแล้ว

สิ่งที่วัดได้ จาก 69 ข้อกำหนด มีเพียง 62 ข้อที่มีการทดสอบ
อัตโนมัติรองรับ เหลืออีก 7 ข้อที่ทำไม่ได้ ถูกบันทึก
เหตุผลไว้แยกไม่ปล่อยให้หายไปในตัวเลขรวม

เจ็ดข้อที่ไม่มีการทดสอบอัตโนมัติ ไม่ได้แปลว่าไม่ถูกทดสอบ แต่ถูกทดสอบด้วยมือแล้วบันทึกผลไว้ต่างหาก

ผมเขียนเจ็ดข้อนั้นไว้ในตารางแยกพร้อมเหตุผล ไม่ใช่ปล่อยให้มันหายไปเงียบ ๆ ในตัวเลขรวม

เหตุผลของแต่ละข้อไม่เหมือนกัน ข้อที่เกี่ยวกับการตรวจจับพื้นที่จัดเก็บข้อมูล ทดสอบอัตโนมัติไม่ได้ เพราะผลลัพธ์ขึ้นกับฮาร์ดแวร์ของเครื่องที่รัน เครื่องที่มีดิสก์แบบหนึ่ง จะได้คำตอบไม่เหมือนเครื่องที่มีอีกแบบ การเขียนเทสต์ที่ผ่านบนทุกเครื่อง จึงต้องเขียนให้หลวมจนไม่ได้พิสูจน์อะไร

ข้อที่เกี่ยวกับการตรวจจับแบบไม่ขวางหน้าจอ ทดสอบอัตโนมัติไม่ได้ เพราะมันเป็นเรื่องจังหวะเวลาระหว่างสองส่วนที่ทำงานพร้อมกัน เทสต์แบบนี้เขียนได้ แต่จะเป็นเทสต์ที่บางครั้งผ่านบางครั้งไม่ผ่าน โดยที่โค้ดไม่ได้เปลี่ยน ซึ่งแย่กว่าไม่มีเทสต์ เพราะมันสอนให้คนเลิกเชื่อผลการทดสอบ

ผมเลือกทดสอบทั้งสองข้อนี้ด้วยมือแทน แล้วบันทึกไว้ว่าทดสอบเมื่อไหร่ ได้ผลยังไง

สิ่งที่ผมอยากให้สังเกตคือ ตัวเลขที่เขียนไว้ในเอกสารคือหกสิบสองจากหกสิบเก้า ไม่ใช่หกสิบเก้าจากหกสิบเก้า ทั้งที่ผมเขียนเอกสารเองและไม่มีใครมาตรวจ

การเขียนตัวเลขที่ไม่เต็มลงในเอกสารของตัวเอง เป็นสิ่งที่ทำได้ง่ายกว่าที่คิด ถ้ายอมรับตั้งแต่ต้นว่าเอกสารมีไว้ให้ตัวเองใช้ ไม่ได้มีไว้ให้ตัวเองดูดี

แผนที่มีความเสี่ยงเขียนไว้

เอกสารอีกฉบับที่ผมคิดว่าคนมองข้ามบ่อยคือแผนโครงการ

คนส่วนใหญ่คิดว่าแผนคือตาราง Gantt ที่มีแท่งสีสวย ๆ บอกว่างานไหนทำเดือนไหน ก็เป็นส่วนหนึ่งจริง แต่เป็นส่วนที่ผมใช้น้อยที่สุด ส่วนที่ผมใช้จริงคือตารางตารางงาน มันแตกโครงการทั้งหมดออกเป็นก้อนงานที่มีของส่งมอบชัดเจน แต่ละก้อนบอกว่าใช้เวลาเท่าไร ได้อะไรออกมา และต้องทำอะไรเสร็จก่อน

ยกตัวอย่างสองก้อนจากโครงการนี้ ก้อนที่ชื่อ Batch Optimizer ใช้เวลาสองเดือน ได้ออกมาเป็นสคริปต์กับโปรไฟล์สามชุด และต้องออกแบบสถาปัตยกรรมเสร็จก่อน ก้อนถัดมาชื่อ Python GUI ใช้เวลาหกเดือน ได้ออกมาเป็นตัวโปรแกรม และต้องรอก้อนแรกเสร็จก่อน

คอลัมน์สุดท้ายคือคอลัมน์ที่มีค่าที่สุด เพราะมันบอกว่าถ้าก้อนแรกช้า ก้อนหลังจะช้าตามแน่นอน ไม่ใช่อาจจะช้า

และส่วนที่ผมคิดว่าสำคัญไม่แพ้กันคือตารางความเสี่ยง เขียนไว้ตั้งแต่ต้นว่าอะไรบ้างที่อาจทำให้โครงการนี้พัง

การเขียนความเสี่ยงล่วงหน้ามีผลข้างเคียงที่ผมไม่ได้คาดไว้ พอเขียนแล้วอ่านซ้ำ ผมพบว่าความเสี่ยงบางข้อป้องกันได้ตั้งแต่ตอนออกแบบ ไม่ต้องรอให้มันเกิด

ความเสี่ยงข้อที่ว่าการทดลองกับผู้เล่นจริง อาจหาคนไม่ได้ตามเป้า ทำให้ผมออกแบบการวิเคราะห์ให้รองรับกรณีที่ได้คนน้อยตั้งแต่แรก แทนที่จะออกแบบไว้แบบเดียว แล้วมาแก้ตอนที่หาคนไม่ครบ

เอกสารที่บันทึกว่าเปลี่ยนอะไรไปแล้ว

เอกสารฉบับสุดท้ายที่ผมอยากเล่าคือบันทึกคำขอเปลี่ยนแปลง

ข้อฟังดูเป็นทางการเกินไปสำหรับงานคนเดียว แต่สิ่งที่มันทำงานมาก มันบันทึกว่ระหว่างทางมีอะไรที่เปลี่ยนไปจากแผนเดิมบ้าง เปลี่ยนเพราะอะไร และผลของการเปลี่ยนคืออะไร

ในโครงการนี้มีสี่รายการ

รายการหนึ่งคือการรวมหน้าสารองข้อมูลกับหน้ากู้คืน เข้าเป็นหน้าเดียว ไม่ได้อยู่ในแผนเดิม ผมเปลี่ยนเพราะพบว่าผู้ใช้ที่กดสารองข้อมูล กับผู้ใช้ที่กดกู้คืน คือคนเดียวกันในคนละเวลา การแยกเป็นสองหน้าทำให้คนที่ตกใจอยากกู้คืน ต้องหาว่าหน้านั้นอยู่ไหน ในจังหวะที่ไม่ควรต้องหาอะไรทั้งนั้น

อีกรายการคือการเพิ่มการตรวจสอบด้านความปลอดภัย กับการขยายชุดทดสอบ เกิดหลังจากผมตรวจโค้ดตัวเอง แล้วเจอช่องโหว่ที่ไม่ได้คิดถึงตอนออกแบบ

การเขียนสิ่งเหล่านี้ไว้ ทำให้ตอบคำถามได้ ว่าทำไมโปรแกรมสุดท้ายไม่เหมือนที่ออกแบบไว้ตอนแรก คำตอบไม่ใช่ว่าออกแบบผิด แต่เป็นว่ระหว่างทางได้รู้อะไรเพิ่ม

ลำดับที่กลับกันไม่ได้

ผมอยากปิดบทนี้ด้วยข้อสังเกตข้อเดียว

เอกสารทั้งหมดที่เล่ามา ถ้าเขียนหลังจากโค้ดเสร็จแล้ว มันจะกลายเป็นคำอธิบายว่โค้ดทำอะไร ซึ่งอ่านโค้ดเขาก็ได้

ค่าของมันอยู่ที่การเขียนก่อน เพราะการเขียนก่อนบังคับให้ตัดสินใจ ตอนที่ยังเปลี่ยนใจได้ถูก

การเปลี่ยนประโยคในเอกสารข้อกำหนดใช้เวลาสามนาที การเปลี่ยนสิ่งเดียวกันหลังเขียนโค้ดไปแล้วสามเดือน ใช้เวลาเป็นสัปดาห์ และมักจะจบลงด้วยการไม่เปลี่ยน แล้วปล่อยให้โปรแกรมทำสิ่งที่ไม่ควรทำต่อไป เพราะแก้ตอนนี้แพงเกิน

นี่คือเหตุผลที่บทนี้ชื่อว่าเอกสารที่เขียนก่อนโค้ด ไม่ใช่เอกสารที่ต้องมี

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน เอกสารข้อกำหนดคือรายการสิ่งที่อยากให้โปรแกรมทำ

หลังอ่าน ข้อกำหนดที่ดีต้องมีตัวเลขกำกับว่าตรวจสอบผ่านได้ยังไง ไม่ใช่แค่ฟังดูดี

- ✓ รู้จักตารางที่ผูกข้อกำหนด โค้ด และการทดสอบเข้าด้วยกัน เพื่อตามรอยได้
- ✓ รู้ว่าช่องว่างที่ทดสอบอัตโนมัติไม่ได้ ต้องบันทึกเหตุผลไว้ ไม่ใช่ปล่อยให้หายไปในตัวเลขนรวม
- ✓ รู้ว่าทำไมเอกสารต้องเขียนก่อนโค้ด ไม่ใช่เขียนอธิบายทีหลัง

บทที่ 4ค — โค้ดอยู่ที่ไหน ทำงานยังไง

คำถามที่ 08

จะรู้ได้ยังไงว่าตอนนี้ของอยู่ในสภาพไหน และใครมีสิทธิ์
แตะอะไรได้บ้าง

บทนี้เป็นบทที่ผมลังเลอยู่นานว่าจะเขียนไหม

เพราะมันเล่าเรื่องที่คุณไม่มีอะไร โฟลเดอร์ ชื่อไฟล์ ข้อความตอน
บันทึกงาน สิ่งเหล่านี้ไม่มีใครโชว์ในงานนำเสนอ ไม่มีใครถามถึง และ
ไม่ได้ทำให้โปรแกรมทำงานได้ดีขึ้น

แต่ผมพบว่ามันเป็นเรื่องที่คุณถามผมบ่อยที่สุด เวลานั่งคุยกันจริง
ๆ มากกว่าคำถามเรื่องอัลกอริทึมเสียอีก

แบ่งโฟลเดอร์ตามคำถามว่าใครแตะอะไรได้

โครงการนี้แยกออกเป็นสามส่วนใหญ่ที่อยู่ในโฟลเดอร์คนละที่

ส่วนแรกคือสคริปต์ที่แตะระบบจริง อยู่ในโฟลเดอร์ที่ชื่อ src ส่วน
ที่สองคือตัวโปรแกรมที่มีหน้าจอ อยู่ในโฟลเดอร์ที่ชื่อ clutchg ส่วนที่
สามคือเอกสารทั้งหมด อยู่ในโฟลเดอร์ docs

การแบ่งแบบนี้ไม่ได้แบ่งตามภาษาที่ใช้เขียน แม้ว่าผลลัพธ์จะออกมาเหมือนแบ่งตามภาษาก็ตาม

มันแบ่งตามว่าไฟล์นั้นแต่ละเครื่องของผู้ใช้ได้แค่ไหน

ไฟล์ในโฟลเดอร์แรกแต่ละได้โดยตรง มันเขียนค่าลงในระบบ ปิดเปิดบริการ แก่การตั้งค่าการบูต ไฟล์กลุ่มนี้ทุกไฟล์ ต้องตรวจสอบสิทธิ์ผู้ดูแลระบบก่อนทำอะไร และต้องบันทึกทุกอย่างที่ทำ

ไฟล์ในโฟลเดอร์ที่สองแต่ละไม่ได้เลยด้วยตัวเอง มันตัดสินใจว่าจะสั่งให้ไฟล์กลุ่มแรกทำอะไร แต่ตัวมันเองไม่มีคำสั่งที่เปลี่ยนค่าในระบบ แม้แต่บรรทัดเดียว

ความยากนี้ทำให้ตอบคำถามความปลอดภัยได้ง่ายขึ้นมาก ถ้ามีคนถามว่าโปรแกรมนี้ทำอะไรในเครื่องบ้าง ผมชี้ไปที่โฟลเดอร์เดียวได้ ไม่ต้องบอกว่าต้องดูทั้งโปรเจกต์

ข้อความตอนบันทึกงาน ที่เขียนให้ตัวเองอ่าน

ทุกครั้งที่บันทึกงานลงระบบเก็บประวัติ ต้องเขียนข้อความกำกับว่าครั้งนี้ทำอะไร

ตอนเริ่มโครงการ ข้อความของผมหน้าตาแบบนี้ "แก้บั๊ก"

"อัปเดต" "เพิ่มฟีเจอร์"

หกเดือนต่อมา ผมกลับไปหาว่าตอนไหนที่ทำให้การสำรองข้อมูลเปลี่ยนพฤติกรรม ผมไล่ดูประวัติแล้วเจอข้อความว่า "แก้บั๊ก" สิบสี่ครั้ง

ไม่มีทางรู้ว่าครั้งไหนใช่ ต้องเปิดดูทีละครั้งว่าแก้อะไร

หลังจากนั้นผมเปลี่ยนวิธี ข้อความทุกอันขึ้นต้นด้วยประเภทของงานในวงเล็บ ว่าเป็นการแก้ไขข้อบกพร่อง การเพิ่มการทดสอบ หรือการแก้ไขเอกสาร ตามด้วยส่วนของระบบที่แตะ แล้วตามด้วยคำอธิบายเป็นภาษาที่ผมอ่านแล้วเข้าใจ

ข้อความหนึ่งในโครงการนี้เขียนว่า แก้ข้อบกพร่องสามข้อที่ทำให้ตัวรันชุดคำสั่งฟังตอนใช้งานจริง อีกข้อความหนึ่งเขียนว่า พิสูจน์ว่าโหมดวางแผนไม่ไปเปลี่ยนแปลงเครื่องจริง

ข้อความแบบนี้ยาวกว่า "แก้บั๊ก" อยู่สิบเท่า และใช้เวลาเขียนเพิ่มไม่ถึงยี่สิบวินาที

ผมยังเขียนเป็นภาษาไทยปนอังกฤษตามที่พูดจริง ไม่ได้พยายามเขียนให้เป็นทางการ เพราะคนที่จะอ่านมันคือผมในอนาคต และผมในอนาคตอ่านภาษาที่ตัวเองพูดได้เร็วที่สุด

กึ่งที่แยกไว้ให้ฟังได้

ระบบเก็บประวัติมีแนวคิดหนึ่งที่ชื่อว่ากึ่ง มันคือสำเนาของงานทั้งหมดที่แยกออกมาให้ทดลองได้ โดยไม่กระทบของเดิม

โครงการนี้มีกึ่งหลักสองกึ่ง

กึ่งที่ชื่อ main คือกึ่งที่ปล่อยให้ผู้ใช้ ทุกอย่างในกึ่งนี้ต้องผ่านการทดสอบครบแล้ว ห้ามเขียนทับโดยตรง

กึ่งที่ชื่อ develop คือกึ่งที่ทำงานจริงประจำวัน ของในกึ่งนี้อาจฟังได้ และมันควรฟังตรงนี้ ไม่ใช่ฟังบนเครื่องคนอื่น

ตอนแรกผมคิดว่ากฎนี้ไม่จำเป็นสำหรับงานคนเดียว เพราะผมเป็นคนเดียวที่แตะโค้ด ผมจะเผลอเขียนทับตัวเองได้ยังไง

คำตอบคือผมเผลอได้ และเคยเผลอมาแล้ว

สิ่งที่ทำให้ผมไม่ใช่ความสับสนเรื่องกิ่ง แต่เป็นวันที่รีบ ผมแก้ไขอะไรเล็ก ๆ ที่มั่นใจว่าไม่พัง แล้วส่งขึ้นกิ่งหลักโดยตรงเพราะขี้เกียจแล้วมันพัง แล้วคนที่โหลดไปใช้ในช่วงนั้นได้ของที่พังไป

กฎที่ห้ามเขียนทับกิ่งหลักโดยตรงมีไว้กันวันที่รีบ ไม่ได้มีไว้กันความสับสน

ข้อความบันทึกงานที่เขียนดี ไม่ได้มีไว้ให้คนอื่นอ่าน มีไว้ให้ตัวผู้เขียนเองในอนาคตของเจอ

เครื่องที่ทำงานแทนตอนที่ผมไม่อยู่

ส่วนที่ผมชอบที่สุดในเรื่องทั้งหมดนี้ คือสิ่งที่เกิดขึ้นอัตโนมัติทุกครั้งที่ส่งโค้ดขึ้นไป

ภาพ 4ค.1 สี่ขั้นที่เครื่องทำให้เองทุกครั้งที่ส่งโค้ดขึ้นไป ขั้นที่สองคือขั้นที่ตัดสินใจว่าโค้ดชุดนี้ผ่านหรือไม่ผ่าน ถ้าไม่ผ่านสองขั้นหลังไม่ทำงานเลย และสิ่งที่เครื่องทำแทนไม่ได้คือสิ่งที่บอกได้มากพอ ๆ กับสิ่งที่มันทำได้



สิ่งที่เครื่องทำแทนไม่ได้

การทดสอบแบบกดผ่านหน้าจอจริงไม่รันในขั้นนี้ เพราะต้องมีเครื่อง Windows ที่เปิดหน้าจอค้างไว้และมีสิทธิ์ผู้ดูแลระบบ เครื่องของบริการนี้ไม่มีทั้งสองอย่าง

ที่มา: ไฟล์ตั้งค่าใน [github/workflows](https://github.com/riipn/workflows) ของรีโปนี้

ทุกครั้งที่ผมส่งโค้ดขึ้นไป มีเครื่องเปิดขึ้นมาที่ไหนสักแห่ง ติดตั้งทุกอย่างใหม่ตั้งแต่ศูนย์ แล้วรันการทดสอบทั้งหมด

จุดที่สำคัญคือคำว่า "ตั้งแต่ศูนย์"

บนเครื่องของผม การทดสอบผ่านมาตลอด เพราะเครื่องของผมมีทุกอย่างที่ต้องมี ติดตั้งไว้เมื่อไหร่ก็จำไม่ได้ บางอย่างติดตั้งไว้ตอนทำโครงการอื่นด้วยซ้ำ

เครื่องที่เพิ่งเปิดขึ้นมาใหม่ไม่มีอะไรเลย ถ้าโปรแกรมของผมต้องการอะไรที่ไม่ได้เขียนไว้ในรายการ มันจะพังทันที

นี่คือการทดสอบที่ผมทดสอบด้วยตัวเองไม่ได้ เพราะผมไม่มีทางลัดสิ่งที่เครื่องตัวเองมีอยู่แล้ว

สิ่งที่เครื่องนั้นทำแทนไม่ได้

การทดสอบที่รันในขั้นตอนนี้มีสองชนิดจากทั้งหมดสามชนิด

ชนิดที่ไม่ได้รันคือการทดสอบแบบกดผ่านหน้าจอจริง ซึ่งมีอยู่หกลิสต์รายการในโครงการนี้

เหตุผลเขียนไว้ในหัวไฟล์ตั้งค่าเป็นภาษาอังกฤษสามบรรทัด ใจความคือการทดสอบชนิดนี้ ต้องการเครื่องที่มีหน้าจอเปิดค้างอยู่จริง ซึ่งเครื่องที่เปิดขึ้นมาชั่วคราวไม่มี

ลองนึกภาพว่าทดสอบตัวหนึ่งต้องกดปุ่มบนหน้าต่างโปรแกรม แล้วดูว่าหน้าต่างเปลี่ยนไปอย่างไร ถ้าไม่มีหน้าจอ ก็ไม่มีหน้าต่างให้กด

ยังมีการทดสอบอีกกลุ่มที่ต้องใช้สิทธิ์ผู้ดูแลระบบ ซึ่งเครื่องชั่วคราวเหล่านั้นให้ไม่ได้ ด้วยเหตุผลด้านความปลอดภัยของตัวบริการเอง

ผมเลยต้องรันสองกลุ่มนี้ด้วยมือบนเครื่องจริง แล้วบันทึกผลไว้

เทียบกับของจริง

สิ่งที่เชื่อ	ระบบทดสอบอัตโนมัติที่รันทุกครั้งที่ส่งโค้ด แปลว่าทดสอบครบทุกอย่างแล้ว
สิ่งที่วัดได้	มีการทดสอบแบบกดผ่านหน้าจอจริงอยู่ 64 รายการที่เครื่องอัตโนมัติรันให้ไม่ได้ ต้องทดสอบด้วยมือแล้วบันทึกผลแยกไว้เอง

ตรงนี่คือสิ่งที่ผมอยากให้สังเกต การมีระบบอัตโนมัติไม่ได้แปลว่าทุกอย่างอัตโนมัติ มันแปลว่าสิ่งที่ทำอัตโนมัติได้ถูกทำ โดยไม่ต้องรอผม และสิ่งที่ทำไม่ได้ถูกระบุไว้ชัดเจนว่าทำไม แทนที่จะหายไปเงียบ ๆ

การเขียนว่า "ทดสอบครบทุกอย่างโดยอัตโนมัติ" ทั้งที่มีหกลสิบสี่รายการที่ไม่ได้รัน จะเป็นคำพูดแบบเดียวกับที่ผมวิจารณ์คนอื่นไว้ตลอดทั้งเล่ม

ขั้นสุดท้ายที่แยกจากขั้นทดสอบ

เมื่อถึงเวลาปล่อยรุ่นใหม่ให้ผู้ใช้ มีขั้นตอนอีกชุดที่ทำงานคนละจังหวะกัน

ชุดนี้ทำงานเมื่อผมติดป้ายหมายเลขรุ่นเท่านั้น ไม่ได้ทำงานทุกครั้งที่ส่งโค้ด

สิ่งแรกที่ผมไม่ใช้ในการสร้างไฟล์ติดตั้ง แต่เป็นการตรวจว่าหมายเลขรุ่นที่ผมติดป้ายไว้ ตรงกับหมายเลขที่เขียนอยู่ในไฟล์ตั้งค่าสีแห่งในโปรแกรม หรือเปล่า

การตรวจข้อนี้เกิดจากข้อบกพร่องจริงที่เล่าไปในบทที่ 5 ครั้งนั้นเวอร์ชันที่แสดงบนหน้าจอไม่ตรงกับเวอร์ชันจริง ทำให้ถ้าผู้ใช้แจ้งปัญหามาโดยบอกเวอร์ชันจากหน้าจอ ผมจะไปเปิดโค้ดผิดรุ่น

หลังจากแก้ ผมไม่ได้แค่แก้ตัวเลขให้ตรง แต่ใส่การตรวจนี้ไว้ในขั้นตอนปล่อยรุ่น ถ้าไม่ตรง มันหยุด ไม่สร้างไฟล์ให้

นี่คือรูปแบบที่ผมใช้ซ้ำทั้งโครงการ เจอข้อบกพร่องหนึ่งครั้ง แล้วใส่กลไกที่ทำให้มันเกิดซ้ำไม่ได้ แทนที่จะแก้แล้วจำไว้ว่าคราวหน้าต้องระวัง

การจำไว้ว่าต้องระวัง เป็นวิธีป้องกันที่ใช้ได้จนถึงวันที่ลืม

ทำไมเรื่องพวกนี้ถึงเป็นเรื่องเดียวกัน

โพลเดอร์ ข้อความบันทึกงาน กิ่ง และขั้นตอนอัตโนมัติ ดูเหมือนเป็นเรื่องที่ไม่เกี่ยวข้องกัน

แต่ทั้งสี่เรื่องตอบคำถามเดียวกัน คำถามว่าจะรู้ได้ยังไงว่าตอนนี้ของอยู่ในสภาพไหน

โพลเดอร์บอกว่าอะไรอยู่ตรงไหน ข้อความบันทึกบอกว่าเปลี่ยนอะไรไปแล้วบ้าง กิ่งบอกว่าอันไหนพร้อมใช้อันไหนยัง และขั้นตอนอัตโนมัติบอกว่าของที่มีอยู่ตอนนี้ยังทำงานอยู่ไหม

ถ้าตอบสี่คำถามนี้ไม่ได้ ทุกการเปลี่ยนแปลงจะกลายเป็นการเสี่ยง

และในโครงการที่เครื่องมือของคุณเข้าไปแก้ระบบปฏิบัติการของคนอื่น การเสี่ยงแบบนั้นไม่ได้ตกอยู่กับคุณคนเดียว

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน โฟลเดอร์และชื่อไฟล์เป็นรายละเอียดที่ไม่กระทบความปลอดภัย

หลังอ่าน **การแบ่งโฟลเดอร์ตามสิทธิ์แต่ละระบบ คือกลไกความปลอดภัยขั้นหนึ่ง ไม่ใช่ความสะอาดของโค้ด**

- ✓ รู้ว่าทำไมข้อความบันทึกงานต้องระบุประเภทงานและส่วนที่แตะ ไม่ใช่แค่ "แก้บั๊ก"
- ✓ รู้ว่าทำไมถึงหลักห้ามถูกเขียนทับโดยตรง แม้เป็นงานคนเดียว
- ✓ รู้ว่าเครื่องอัตโนมัติทดสอบได้ไม่ครบทุกอย่าง และช่องว่างต้องถูกระบุไว้ชัด ไม่ใช่หายไปถึงจบ ๆ

บทที่ 5 — การทดสอบคือการพยายาม ทำให้ของตัวเองฟัง

คำถามที่ 09

ถ้าทดสอบทั้งหมดผ่านทุกตัว แปลว่าเครื่องมือนี้ปลอดภัย
จริงหรือแค่ยังไม่เจอวิธีทำให้ฟัง

คืนหนึ่งผมนั่งเขียนเทสต์ตัวหนึ่งที่ทำให้รู้สึกแปลก ๆ

งานของมันคือส่งคำสั่งอันตรายเข้าไปในระบบตัวเองตรง ๆ ลอง
พิมพ์คำสั่งที่ลบทั้งไดรฟ์ ลองพิมพ์คำสั่งที่ปิดไฟร์วอลล์ ลองพิมพ์คำ
สั่งแปลก ๆ ที่คนทั่วไปไม่มีทางพิมพ์เอง แล้วดูว่าเครื่องมือของผมนจะ
ทำตามไหม

ผมกดรัน แล้วรอ

ถ้าเทสต์นี้ผ่าน หมายความว่าเครื่องมือปฏิเสธคำสั่งพวกนั้น ถ้า
เทสต์นี้ไม่ผ่าน หมายความว่ามันยอมทำ ซึ่งเป็นเรื่องที่แย่กว่าบั๊ก
ทั่วไปมาก เพราะบั๊กทั่วไปทำให้โปรแกรมพัง ส่วนบั๊กแบบนี้ทำให้
เครื่องของคนอื่นพัง

นี่คือความรู้สึกที่ผมอยากเล่าในบทนี้ การเขียนโค้ดส่วนใหญ่ที่ผมทำมาตลอดชีวิต คือการเขียนให้มันทำงาน แต่การเขียนเทสต์ที่ดี คือการพยายามหาทางทำให้มันพังให้ได้ ก่อนที่คนอื่นจะเจอ

ทดสอบว่าทำงาน กับทดสอบว่าไม่ยอมทำ

มีความต่างอยู่เส้นหนึ่งที่ผมมองไม่ค่อยเห็นตอนเริ่มโครงการ

เทสต์แบบแรกคือการเช็กว่าพีเจอร์ทำงานตามที่ตั้งใจ กดปุ่มสำรวจข้อมูลแล้วดูว่ามีไฟล์สำรวจเกิดขึ้นจริง เลือกโปรไฟล์แล้วดูว่ารายการที่ควรถูกเลือกถูกเลือกจริง เทสต์แบบนี้เขียนง่าย เพราะรู้ล่วงหน้าว่าอยากเห็นอะไร

เทสต์แบบที่สองคือการเช็กว่าระบบปฏิเสธสิ่งที่ไม่ควรทำ ส่งชื่อไฟล์ที่มีอักขระแปลก ๆ เข้าไป ส่งเส้นทางที่ช้อยนอกโฟลเดอร์ที่อนุญาต ส่งคำสั่งที่ประกอบขึ้นมาให้ดูเหมือนคำสั่งปกติ แต่แอบมีคำสั่งที่สองแทรกอยู่ข้างใน

เทสต์แบบที่สองเขียนยากกว่า เพราะต้องคิดล่วงหน้าว่าคนร้ายหรือความผิดพลาดจะโผล่มาในรูปแบบไหน ต้องนั่งคิดเหมือนคนที่อยากทำให้ระบบพัง ไม่ใช่คนที่อยากให้ระบบทำงาน

ผมพบว่าตัวเองใช้เวลากับเทสต์แบบที่สองมากกว่าที่คาดไว้ตอนแรกมาก เพราะทุกครั้งที่คิดเคสใหม่ได้หนึ่งเคส มันมักจะพาไปคิดเคสที่สองและสามตามมาเสมอ

ทำไมพันทกว่าเทสต์ ถึงมากกว่าจำนวนพีเจียร์เยอะขนาดนี้

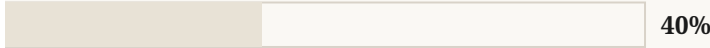
ตัวเลขรวมของการทดสอบอัตโนมัติที่รันในเครื่องปกติได้ อยู่ที่ 1,078 รายการ ผ่านทั้งหมด ไม่มีตัวไหนล้มเหลว นับเมื่อวันที่จัดพิมพ์เล่มนี้

เพราะบีกทั่วไปทำให้โปรแกรมพัง ส่วนบีกแบบนี้ ทำให้เครื่องของคนอื่นพัง

ผมเขียนวันที่กำกับตัวเลขนี้ไว้ตั้งใจ เพราะจำนวนเทสต์เป็นตัวเลขที่เพิ่มทุกสัปดาห์ ตอนที่ผมเขียนต้นฉบับบทนี้รอบแรกมันคือ 765 รายการ ตัวเลขที่ไม่บอกว่าจะวัดเมื่อไหร่ จะกลายเป็นตัวเลขผิดโดยไม่มีใครรู้ตัว

ภาพ 5.1 สัดส่วนบรรทัดที่การทดสอบเดินผ่าน วัดสองแบบ ตัวเลขทั้งโปรเจกต์รวมขึ้นหน้าจอที่ทดสอบด้วยตา ส่วนตัวเลขขึ้นแกนนับเฉพาะโค้ดที่ดึงดูดสนใจแทนผู้ใช้ เกณฑ์เจ็ดสิบที่ขีดไว้เป็นเกณฑ์ที่ผมตั้งเองตามมาตรฐาน ไม่ใช่ค่ามาตรฐานกลาง

สัดส่วนบรรทัดที่การทดสอบเดินผ่าน
ทั้งโปรเจกต์ รวมขึ้นหน้าจอ



เฉพาะขึ้นแกนที่ดึงดูดสนใจ



เกณฑ์ที่ตั้งไว้เอง 70%

ตัวเลขต่ำไม่ได้แปลว่าทดสอบน้อย มันแปลว่าขึ้นหน้าจอทดสอบด้วยตาแทน

ที่มา: วัดเองด้วย `pytest --cov` เมื่อ 2026-08-06

เทียบกับของจริง

สิ่งที่เชื่อ จำนวนเทสต์ที่มากกว่าโปรแกรมมีพีเจียร์เยอะ

สิ่งที่วัดได้ มี 1,078 เทสต์ แต่พีเจียร์หลักไม่เกินสิบอย่าง เพราะหนึ่งพีเจียร์มีหลายทางเดิน

คนที่เห็นตัวเลขนี้ครั้งแรกมักถามคำถามเดียวกัน เครื่องมือนี้มีพีเจอร์ทั้งเยอะขนาดนั้นเลยหรือ

คำตอบคือไม่ใช่ เครื่องมือนี้มีพีเจอร์ทหลักไม่เกินสิบอย่าง เหตุที่ทดสอบมากกว่าพีเจอร์ทหลายสิบเท่า เพราะหนึ่งพีเจอร์ทไม่ได้มีทางเดินเดียว

ลองนึกภาพพีเจอร์ทสำรองข้อมูลก่อนปรับแต่ง มันมีทางเดินหลักคือกดแล้วสำรองสำเร็จ มีทางเดินรองคือกดแล้วพื้นที่ดิสก์ไม่พอ กดแล้วไม่มีสิทธิ์เขียนไฟล์ กดแล้วเครื่องดับกลางทาง กดสองครั้งพร้อมกัน และกดตอนที่การสำรองครั้งก่อนยังไม่เสร็จ

แต่ละทางเดินรองพวกนี้ต้องมีทดสอบของตัวเอง เพราะแต่ละทางต้องพิสูจน์ว่าระบบไม่พังแบบเงียบ ๆ ไม่ทำงานต่อทั้งที่ไม่ควร และไม่บอกผู้ใช้งานว่าสำเร็จทั้งที่ไม่สำเร็จ

พีเจอร์ทที่แต่ละระบบจริงยังต้องมีทางเดินแบบนี้มากกว่าพีเจอร์ททั่วไป เพราะทุกจุดที่แก้ registry หรือเปิดปิด service คือจุดที่ถ้าพังแล้วพังแบบกู้คืนยาก

นี่คือเหตุผลที่ผมเลือกเปรียบเทียบให้เห็นภาพแบบนี้ พีเจอร์ทหนึ่งอย่างเหมือนถนนหนึ่งสาย ทางเดินหลักคือถนนตรงที่รถส่วนใหญ่วิ่งผ่าน แต่ถนนทุกสายมีทางลาดฉุกเฉิน มีจุดที่รถเสียต้องจอด มีทางแยกที่คนขับผิดทางได้ ถ้าอยากรู้ว่าถนนสายนั้นปลอดภัยจริง ต้องทดสอบทุกจุดพวกนั้น ไม่ใช่แค่ขับตรงไปตรงมาครั้งเดียว

ตัวเลขเดียวที่หลอกคนได้ง่ายที่สุด

พอพูดถึงการทดสอบ คนมักถามตัวเลข coverage เป็นคำถามแรก มันวัดว่าโค้ดที่เปอร์เซ็นต์ถูกทดสอบแต่ละถึง

ตัวเลขของโครงการนี้ทั้งโปรเจกต์อยู่ที่ 40%

ฟังดูน้อยมากถ้าไม่รู้ที่มา แต่ถ้าแยกดูเฉพาะชั้น core ซึ่งเป็นชั้นที่มีตรรกะความปลอดภัย และการตัดสินใจว่าจะรันอะไรได้บ้าง ตัวเลขนั้นคือ 81%

ความต่างระหว่างสองตัวเลขนี้ไม่ได้มาจากความซับซ้อน มันมาจากว่าโค้ดในโปรเจกต์นี้ไม่ได้มีความเสี่ยงเท่ากันทุกส่วน ส่วนที่เป็นหน้าจอ การจัดวางปุ่ม สีของตัวอักษร ต่อให้ไม่มีเทสต์เลยก็ไม่ได้ทำให้เครื่องของใครพัง

ส่วนที่ตรวจว่าคำสั่งนี้ปลอดภัยไหม ส่วนที่ตัดสินใจจะย้อนกลับได้หรือไม่ได้ ส่วนที่กรองคำสั่งอันตรายก่อนส่งไปรันจริง คือส่วนที่ผมทุ่มเทสตัลงไปหนักที่สุด และเป็นเหตุผลที่ตัวเลขของชั้นนั้นสูงถึง 81%

ถ้าผมรายงานแค่ตัวเลข 40% ตัวเดียว มันจะทำให้คนคิดว่าโครงการนี้ทดสอบไม่ทั่วถึง ถ้าผมรายงานแค่ 81% ตัวเดียว มันจะทำให้คนคิดว่าทุกส่วนถูกตรวจเข้มเท่ากัน ทั้งสองแบบไม่ตรงกับความจริง

ส่วนที่เหลืออยู่นอก 81% ของชั้น core คืออะไร ส่วนใหญ่คือโค้ดที่จัดการข้อผิดพลาดแปลก ๆ ที่ไม่น่าจะเกิดในทางปฏิบัติ เช่นระบบไฟล์เสียกลางทางที่อ่านค่า หรือหน่วยความจำเต็มพอดีตอนที่กำลังเขียนล็อก ผมรู้ว่ามันอยู่ตรงไหน และรู้ว่าทำไมยังไม่คุ้มเวลาที่จะเขียนเทสต์ให้ครบตอนนี้

สามข้อบกพร่องที่เล่าได้

ตลอดกระบวนการทดสอบและตรวจสอบ พบข้อบกพร่องรวม 27 ข้อ แบ่งเป็นจาก unit test 8 ข้อ จาก integration test 3 ข้อ จากการตรวจสอบความปลอดภัย 11 ข้อ และจากการรีวิวโค้ด 5 ข้อ

ในยี่สิบเจ็ดข้อนั้น มีสามข้อที่ผมอยากเล่าตรง ๆ เพราะแต่ละข้อสอนบางอย่างที่ต่างกัน

ข้อแรกคือโปรแกรมเคยทำงานต่อแม้การสำรองข้อมูลล้มเหลว ผมเล่าเรื่องนี้ไปคร่าว ๆ ในบทที่แล้ว แต่รายละเอียดที่ยังไม่ได้เล่าคือตอนที่เจอมันครั้งแรก มันไม่ได้โผล่มาเป็นข้อความ error สีแดง มันโผล่มาเป็นความเงียบ การสำรองข้อมูลล้มเหลวแบบไม่มีใครเห็น แล้วการปรับแต่งเดิรหน้าต่อไปเหมือนไม่มีอะไรเกิดขึ้น

ผมแก้ไขทันทีที่การสำรองล้มเหลว ระบบต้องหยุดทั้งกระบวนการ ไม่ใช่แค่แจ้งเตือนแล้วปล่อยให้ทำต่อ พร้อมเพิ่มเทสต์ถดถอยสี่ตัวเพื่อล็อกพฤติกรรมนี้ไว้ไม่ให้กลับไปเป็นแบบเดิมอีกในอนาคต

ข้อที่สองคือเวอร์ชันที่เขียนไว้ในไฟล์ตั้งค่า ไม่ตรงกับเวอร์ชันที่ประกาศต่อผู้ใช้ ฟังดูเป็นเรื่องเล็ก แต่ผลคือถ้าผู้ใช้รายงานปัญหามา โดยดูเวอร์ชันจากหน้าจอ ทีมพัฒนาอาจไปดูโค้ดผิดรุ่น มันเป็นความคลาดเคลื่อนที่ไม่ทำให้อะไรพังตรงหน้า แต่ทำให้การแก้ปัญหาในอนาคตหลงทางได้ง่าย

ข้อที่สามคือรายการที่อนุญาตให้ย้อนกลับได้มีบางรายการหลุดตกไป หมายความว่ามีการปรับแต่งบางตัวที่ระบบสำรองค่าไว้แล้ว แต่ตัวย้อนกลับกลับไม่รู้จักรว่ต้องย้อนตัวนั้นด้วย ถ้าไม่เจอก่อนถึงมือผู้ใช้ ผลคือคนกดย้อนกลับแล้วคิดว่าทุกอย่างกลับเป็นเดิม ทั้งที่มีบางอย่างยังค้างอยู่

สามข้อนี้มีจุดร่วมเดียวกัน ไม่มีข้อไหนเกิดจากความประมาทซัด ๆ แต่เกิดจากช่องว่างเล็ก ๆ ระหว่างสามส่วนของระบบที่ต้องเดินไปด้วยกัน แต่ไม่ได้ถูกบังคับให้เดินไปด้วยกันจริง ๆ

ห้ำสลิปห้ำการทดสอบที่ต้องทำด้วยมือ

ไม่ใช่ทุกอย่งทดสอบอัตโนมัติได้ มีการทดสอบด้วยมือ 55 รายการ
ครอบคลุม 11 หมวด ผ่านทั้งหมด

เหตุผลที่ทำอัตโนมัติไม่ได้ ไม่ใช่เพราะขี้เกียจเขียนสคริปต์ แต่
เพราะเทสต์เหล่านี้ต้องแก้ registry จริงบนเครื่องจริง ปิด service
จริง เปลี่ยนการตั้งค่าการบูตจริง

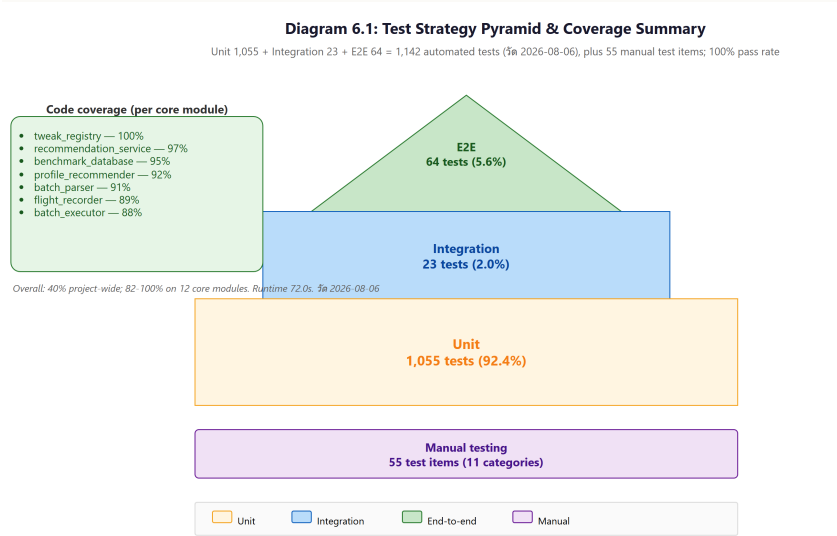
สมมติว่าอยากเขียนเทสต์อัตโนมัติที่รันซ้ำได้ทุกคืนบน CI สำหรับ
พีเจอร่ที่ปิด service ป้องกันระบบ เทสต์นั้นต้องปิด service จริงบน
เครื่องที่รันเทสต์ ถ้าเครื่องนั้นเป็นเครื่องที่ใช้ร่นงานอื่นด้วย การปิด
service ความปลอดภัยของมันซ้ำ ๆ ทุกคืนไม่ใช่เรื่องที่ทำได้อย่าง
ปลอดภัย

เครื่องที่รันเทสต์อัตโนมัติ มักเป็นเครื่องเสมือนที่ใช้ร่วมกันหลาย
โครงการ การไปแก้ค่าระดับระบบปฏิบัติการซ้ำ ๆ อาจทำให้เครื่องนั้น
เสียหาย จนงานอื่นที่ไม่เกี่ยวก็ร่นไม่ได้ตามไปด้วย

ทางเดียวที่ปลอดภัยพอ คือใช้เครื่องจริงเครื่องหนึ่งที่กันไว้เฉพาะ
สำหรับเทสต์แบบนี้ แล้วนั่งกดทดสอบทีละรายการด้วยมือ จดผลไว้
ก่อนจะคืนค่าเครื่องกลับสภาพเดิม เพื่อรอบทดสอบครั้งต่อไป

สลิปเอ็ดหมวดที่ทดสอบด้วยมือ ครอบคลุมตั้งแต่การเปิดปิด
service ป้องกันระบบ การแก้ค่าที่กระทบการบูตของเครื่อง ไปจนถึง
การทดสอบว่าเครื่องเมื่อรีสตาร์ท แล้วยังเข้าระบบได้ปกติหรือไม่ ซึ่ง
เป็นสิ่งที่อัตโนมัติทดสอบได้ยากที่สุด เพราะต้องรอเครื่องบูตขึ้นมา
ใหม่จริง

ภาพ 5.2 สัดส่วนของการทดสอบแต่ละชนิด ฐานกว้างที่สุดคือการทดสอบหน่วยย่อยซึ่งรันเร็วและรันบ่อย ยอดแคบที่สุดคือการทดสอบแบบกดผ่านหน้าจอจริงซึ่งช้าและเปราะ รูปทรงนี้ไม่ได้เกิดเอง มันคือสิ่งที่ตั้งใจให้เป็น เพราะถ้ากลับหัวเมื่อไหร่ การทดสอบทั้งชุดจะช้าจนไม่มีใครยอมรับ



ที่มา: ไดอะแกรมพีระมิดการทดสอบ ฉบับตัวเลขวัดเมื่อ 2026-08-06

ผมคิดว่าการทดสอบด้วยมือแบบนี้ไม่ได้ดีด้อยกว่าการทดสอบอัตโนมัติ มันแค่ตอบคำถามคนละแบบ อัตโนมัติตอบว่าตรรกะถูกไหมเวลารันซ้ำพันครั้ง มือตอบว่าเครื่องจริงยังเปิดติดไหม หลังจากโดนแก๊จริง



การแกะของที่ตัวเองสร้าง ไม่ใช่การทำลาย มันคือวิธีเดียวที่
จะรู้ว่ามันทนอะไรได้

ตัวเลขที่พิสูจน์ว่าออกแบบไว้ถูก

การทดสอบด้านความปลอดภัยเป็นชุดที่ผมอยากเล่าแยกจากชุดอื่น เพราะมันไม่ใช่การเช็คว่าพีเจอร์ท างาน แต่เป็นการเช็คว่าเกะระที่วางไว้ในบตที่แล้วทำงานจริงหรือเปล่า

เจ็ดเทคนิคที่ขึ้นบัญชีห้ามถาวรตั้งแต่บตที่ 3 ถูกทดสอบด้วยการพยายามส่งเข้าระบบทุกทางที่คิดได้ ผลคือผ่านเข้าระบบได้ 0 รายการ 24 service ที่ถูกกันไว้ไม่ให้ปิดไม่ว่ากรณีใด ก็ถูกทดสอบแบบเดียวกัน ผลคือถูกปิดได้ 0 รายการ

ตัวเลขศูนย์ตรงนี้สำคัญกว่าตัวเลขที่ผ่านทุกอย่างเสียอีก เพราะมันแปลว่าต่อให้ผมพยายามหาทางแหกกฎที่ตัวเองตั้งไว้เอง ก็ยังทำไม่ได้ นี่คือความต่างระหว่างความปลอดภัยที่ฟังว่าไม่มีใครลอง กับความปลอดภัยที่ทนต่อคนที่ตั้งใจลองจริง

ระบบกรองคำสั่งอันตรายด้านล่างของทุกอย่างนี้ มีรูปแบบที่ต้องจับอยู่ 39 รูปแบบ และมีเส้นทางวิกฤตที่ต้องตรวจเข้มเป็นพิเศษอยู่ 5 เส้นทาง ทุกรูปแบบและทุกเส้นทางมีเทสต์เฉพาะของตัวเอง ไม่ใช่เทสต์รวมที่หวังว่าจะครอบคลุมโดยบังเอิญ

เทียบกับของจริง

สิ่งที่เชื่อ	ตัวเลข coverage 40% แปลว่าโปรเจกต์นี้ทดสอบไม่ทั่วถึง
สิ่งที่วัดได้	40% คือทั้งโปรเจกต์ แต่ชั้น core ที่มีตระกะความปลอดภัยวัดได้ 81%

เทสที่ผ่านทุกครั้งตั้งแต่วันแรก มักไม่ได้ทดสอบอะไรเลย

เทสต์ 27 ข้อบกพร่องที่พบ แบ่งเป็น unit 8 integration 3 security 11 code review 5

ตัวเลขที่แยกตามแหล่งที่เจอ ช่วยบอกว่าจุดไหนของกระบวนการจับปัญหาได้มากที่สุด

เทสที่ผ่านทุกครั้งตั้งแต่วันแรก

มีบทเรียนหนึ่งที่ผมอยากฝากไว้ตรงนี้ก่อนไปต่อ

ระหว่างเขียนเทสต์ทั้งหมดนี้ ผมเจอเทสต์บางตัวที่ผ่านทุกครั้งไม่
ว่าจะแก้ไ่ค้ดยังงิ ตอนแรกผมดีใจ คิดว่าไ่ค้ดส่วนนั้นแข็งแรงมาก

จนวันหนึ่งผมลองแก้ไ่ค้ดให้พังโดยตั้งใจ เพื่อเช็คว่าเทสต์จะจับได้
ใหม่ แล้วพบว่าเทสต์ตัวนั้นผ่านอยู่ดี ทั้งที่ไ่ค้ดพังไปแล้ว

ปัญหาไม่ได้อยู่ที่ไ่ค้ด ปัญหาอยู่ที่เทสต์เขียนผิดตั้งแต่ต้น มันเช็กลี
งที่ไม่มีความหมาย เทสต์ตรวจว่าฟังก์ชันคืนค่าอะไรสักอย่างที่ไม่ใช่ค่า
ว่าง แต่ไม่ได้เช็คว่าค่านั้นถูกต้องหรือเปล่า ต่อให้ฟังก์ชันคืนค่าผิด
สนท ก็ยังผ่านเทสต์อยู่ดี

เทสที่ผ่านทุกครั้งตั้งแต่วันแรก มักไม่ได้ทดสอบอะไรเลย

หลังจากนั้นผมเริ่มมีวินัยข้อหนึ่ง ทุกครั้งที่เขียนเทสต์ใหม่ ต้อง
ลองทำให้ไ่ค้ดพังก่อนแล้วดูว่าเทสต์จับได้ใหม่ ถ้าเทสต์ยังผ่านอยู่ดีทั้ง
ที่ไ่ค้ดพังแล้ว แสดงว่าต้องเขียนเทสต์ใหม่ ไม่ใช่ปล่อยไว้แล้วนับว่า
ครบ

คนที่มาช่วยหาช่องโหว่

ผมไม่ได้ทำทุกอย่างคนเดียว ระหว่างที่เขียนเทสต์ด้านความปลอดภัย มีเพื่อนอีกคนที่เก่งเรื่องระบบมากกว่าผม มาช่วยดูโค้ดส่วนที่กรองคำสั่งอันตราย

**ทุกช่องที่เขาเจอในตอนนี คือช่องที่ไม่ต้องมาเจอ
ตอนมีคนใช้จริงแล้ว**

เขาไม่ได้อ่านเพื่อชม เขาอ่านเพื่อหาว่าจะแฮกกฎยังไง ลองพิมพ์คำสั่งที่ใส่ตัวอักษรพิเศษแทรกกลาง ลองส่งเส้นทางไฟล์ที่ยาวเกินขนาดที่ระบบคาดไว้ ลองเรียกฟังก์ชันข้ามลำดับที่ควรเรียก

ผมจำได้ว่าตอนแรกรู้สึกไม่สบายใจเวลาเขาเจอช่องโหว่ เหมือนถูกจับผิดต่อหน้า แต่พอผ่านไปสักสองสามรอบ ผมเปลี่ยนมุมมองไป ทุกช่องที่เขาเจอในตอนนี คือช่องที่ไม่ต้องมาเจอตอนมีคนใช้จริงแล้ว

การให้คนอื่นมาช่วยหาช่องโหว่ในงานตัวเอง เป็นเรื่องที่ทำให้ใจยากกว่าที่คิด เพราะสัญชาตญาณแรกคือการป้องกันงานที่ทำมา แต่ถ้าปล่อยให้สัญชาตญาณนั้นชนะ ช่องโหว่ก็จะไม่ถูกเจอโดยคนที่ตั้งใจช่วย แล้วไปถูกเจอโดยคนที่ตั้งใจทำร้ายแทน

เรื่องเล่าจากเทสต์ตัวที่ทำให้นอนไม่หลับคืนหนึ่ง

มีเทสต์ตัวหนึ่งที่ผมจำได้แม่นเป็นพิเศษ เพราะมันทำให้ผมนอนไม่หลับอยู่คืนหนึ่งจริง ๆ

ผมกำลังเขียนเทสต์สำหรับพีเจอร่ที่ปิด service ตามรายชื่อที่ผู้ใช้เลือก แล้วนึกขึ้นได้ว่าถ้าผู้ใช้พิมพ์ชื่อ service ที่ไม่มีอยู่ในรายชื่อที่อนุญาต แต่บังเอิญพิมพ์คล้ายกับชื่อ service ป้องกันระบบตัวหนึ่งมาก จะเกิดอะไรขึ้น

ผมลองพิมพ์ชื่อที่ต่างกันแค่ตัวอักษรเดียว แล้วกดรันเทสต์

ผลที่ได้คือระบบปฏิเสธ เพราะการตรวจสอบเทียบชื่อแบบตรงทั้งหมด ไม่ใช่แบบคล้ายกัน แต่ตอนที่รอผลตรงนั้น ใจผมเต้นเร็วขึ้นจริง เพราะถ้าผลออกมาตรงกันข้าม นั้นจะเป็นช่องโหว่ที่ร้ายแรงมาก

คืนนั้นผมกลับไปนอนแต่คิดไม่หลุด สุดท้ายลุกมาเขียนเทสต์เพิ่มอีกสิบกว่าตัว ลองสะกดคล้ายกันหลายรูปแบบ ตัวพิมพ์เล็กใหญ่ต่างกัน มีอักขระแปลกแทรก มีช่องว่างแทรก ทุกตัวผ่านหมด แต่ผมถึงจะยอมนอนก็ตอนที่เขียนจนคิดไม่ออก แล้วว่าจะแหกยังงัยอีก

สิ่งที่การทดสอบไม่ได้บอก

เส้นทางวิกฤต 5 เส้นทางและรูปแบบอันตราย 39 รูปแบบ มีเทสต์เฉพาะของตัวเองทุกตัว

::experiment

คำถาม: เจ็ดเทคนิคที่ขึ้นบัญชีห้ามถาวรและ 24 service ที่กันไว้ ผ่านเข้าระบบได้จริงไหมถ้าลองทุกทาง

วิธี: ส่งคำสั่งอันตรายเข้าระบบตัวเองตรง ๆ ผ่านทุกช่องทางที่คิดได้ พร้อมเทียบชื่อ service คล้ายกันหลายรูปแบบ

ผล: ผ่านเข้าระบบได้ 0 รายการ จาก 7 เทคนิคห้ามถาวร และปิดได้ 0 รายการ จาก 24 service ที่กันไว้

บทเรียน: ตัวเลขศูนย์สำคัญกว่าตัวเลขที่ผ่านทุกอย่าง เพราะพิสูจน์ว่าทนต่อคนที่ตั้งใจลองจริง ไม่ใช่แค่ไม่มีใครลอง

ไม่ใช่ทดสอบที่หวังว่าจะครอบคลุมโดยบังเอิญ

มีข้อจำกัดหนึ่งที่ผมต้องพูดตรง ๆ การทดสอบทั้งหมดที่เล่ามา พิสูจน์ได้ว่าระบบทำในสิ่งที่ผมตั้งใจให้ทำ และไม่ทำในสิ่งที่ผมตั้งใจห้าม

แต่การทดสอบไม่ได้พิสูจน์ว่าสิ่งที่ผมตั้งใจให้ทำนั้นถูกทั้งหมด ถ้าผมเข้าใจผิดตั้งแต่ต้นว่าอะไรควรอยู่ในระบบ เทสต์ก็จะผ่านสบาย เพราะมันแค่เช็คตามที่ผมเขียนไว้ ไม่ใช่เช็คตามความจริง

เรื่องนี้ฟังดูเป็นปัญหาเชิงทฤษฎีที่ไกลตัว จนกระทั่งวันหนึ่งการทดสอบชี้ให้เห็นว่าสิ่งที่ผมเขียนไว้ในเอกสาร และประกาศต่อสาธารณะ ไม่ตรงกับสิ่งที่ระบบทำได้จริง

ตัวเลขที่ผมเคยพูดออกไปแล้ว กลายเป็นตัวเลขที่ต้องกลับมาตรวจใหม่

ก่อนไปบทถัดไป

ลองนึกดูว่าคุณเป็นคนตรวจสอบงานของตัวเอง แล้วเจอว่าตัวเลขที่คุณเคยพูดไป แล้วต่อหน้าคนอื่น ไม่ตรงกับความจริงทั้งหมด

คุณจะไม่อึกแอ้เจียบ ๆ หรือบอกตรง ๆ

บทถัดไปคือเรื่องของวันที่ผมต้องตอบคำถามนั้นด้วยตัวเอง ไม่ใช่คำถามสมมติ

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน เทสต์เยอะแปลว่าโปรแกรมมีฟีเจอร์เยอะ และผ่านหมดแปลว่า
ปลอดภัยแน่นอน

หลังอ่าน เทสต์ส่วนใหญ่มีไว้พิสูจน์ว่ระบบปฏิเสธสิ่งที่ไม่ควรทำ ไม่ใช่แค่เช็
กว่ฟีเจอร์ทำงาน และเทสต์ที่ผ่านทุกครั้งอาจไม่ได้ทดสอบอะไร
เลย

- ✓ รู้ความต่างระหว่างเทสต์ที่เช็กว่าทำงาน กับเทสต์ที่เช็กว่าไม่ยอมทำ
- ✓ รู้ว่ coverage ตัวเดียวหลอกคนได้ ต้องแยกดูเป็นรายชั้นถึงจะเห็นความ
เสี่ยงจริง
- ✓ รู้ว่การทดสอบพิสูจน์ได้แค่ว่ระบบทำตามที่ตั้งใจ ไม่ได้พิสูจน์ว่สิ่งที่ตั้งใจ
นั้นถูกทั้งหมด

บทที่ 6 — วันที่ห้าสิบหกกลายเป็นสี่สิบ

คำถามที่ 10

ถ้าตัวเลขที่ประกาศไว้เอง ไม่ตรงกับของจริง จะเลือกแก้
เจียบ ๆ หรือบอกตรง ๆ

วันที่ 2 สิงหาคม ไม่มีใครสั่งให้ผมตรวจสอบอะไร

ไม่มีลูกค้ามาถาม ไม่มีอาจารย์ที่ปรึกษานัดประชุม ไม่มีบักกรีพอร์ต
เต็งเข้ามา เป็นวันธรรมดาที่ผมนั่งเปิดโค้ดตัวเองขึ้นมาอ่านอีกรอบ
อยากเช็กแค่ว่าตัวเลขที่ประกาศไว้ในเอกสารกับบนหน้าจอ ตรงกับ
ของจริงแค่ไหน

ผมเปิดไปด้วยความมั่นใจว่าตรงอยู่แล้ว

มันไม่ตรง

จำนวนที่เคยประกาศ

เอกสารและหน้าจอของโปรแกรมประกาศไว้ตลอดว่ามีการปรับแต่ง 56 รายการ ตัวเลขนี้โผล่ซ้ำอยู่หลายที่ ทั้งในคู่มือผู้ใช้ ในสไลด์ที่ผมใช้ พรินต์ และบนหน้าจอตอนเปิดโปรแกรมครั้งแรก

เทียบกับของจริง

สิ่งที่เชื่อ	ตัวเลขที่ประกาศไว้ในเอกสารและหน้าจอ ตรงกับของจริงในโค้ดเสมอ
สิ่งที่วัดได้	เอกสารประกาศ 56 รายการ แต่ไล่นับจากโค้ดจริงได้ 44 รายการ

ผมไล่นับใหม่ที่ละรายการ ไม่นับจากที่เขียนไว้ แต่นับจากโค้ดจริง ว่าอันไหนบ้างที่ผู้ใช้กด แล้วระบบเปลี่ยนค่าได้จริง แล้วย้อนกลับได้จริง นับได้ 44

สิบสองรายการที่หายไปไม่ได้หายไปเพราะผมลบทิ้ง มันหายไปเพราะไม่เคยควรถูกนับตั้งแต่แรกต่างหาก

ตัวเลขห้าสิบหก นับรวมอะไรที่ไม่ควรนับ

ตอนไล่ดูว่าสิบสองรายการที่หายไปคืออะไร ผมพบว่าทั้งสิบสองเป็นแค่ข้อมูลอธิบาย ไม่ใช่การปรับแต่งที่มีสัญญาณการทำงานผ่านการตรวจจริง

ภาพ 6.1 กรอบเส้นประคือจำนวนที่เอกสารเคยประกาศ แท่งทึบคือจำนวนที่นับได้จากโค้ดจริง ช่องว่างระหว่างสองเส้นนี้คือสิ่งที่ผู้อ่านเอกสารเชื่อว่ามี แต่ผู้ใช้เครื่องมือกดแล้วไม่เจอ

ที่เอกสารเคยบอก



ที่นับได้จากโค้ดจริง



ส่วนต่างสิบสองรายการ คือสิ่งที่เอกสารพูดถึงแต่ไม่มีอยู่จริง

ที่มา: ผลตรวจสอบซ้ำ D-03 เทียบกับการนับใน `tweak_registry.py`

ลองนึกภาพร้านอาหารที่มีเมนูติดไว้หน้าร้านสิบรายการ แต่ในครัวจริงทำได้แค่หกรายการ อีกสี่รายการเป็นชื่ออาหารที่เชฟเคยคิดสูตรไว้ในสมุดบันทึก แต่ยังไม่ได้ทดลองทำจริงในครัวนี้เลยแม้แต่ครั้งเดียว ถ้าลูกค้าสั่งอาหารสี่รายการนั้นตามเมนู ครัวจะทำไม่ได้ ไม่ใช่เพราะเชฟโงก แต่เพราะสมุดบันทึกกับสิ่งที่ทำได้จริงในครัว ไม่เคยแยกออกจากกันให้ชัด

รากของเรื่องนี้อยู่ที่สามแนวคิดที่ปนกันอยู่ และคนที่ปนมันก็คือผมเอง โดยไม่รู้ตัว

สามแนวคิดที่ควรแยกจากกัน

แนวคิดแรกคือบันทึกความรู้เรื่องการปรับแต่ง เป็นฐานข้อมูลที่เก็บว่าเทคนิคแต่ละตัวคืออะไร ทำงานยังไง มาจากไหน เสี่ยงระดับไหน เขียนไว้ให้คนอ่านเข้าใจ ไม่ได้ผูกกับว่าปุ่มบนหน้าจอกดได้จริงหรือเปล่า

แนวคิดที่สองคือการเป็นสมาชิกของโปรไฟล์ โปรไฟล์ทั้งสาม SAFE COMPETITIVE EXTREME แต่ละอันมีรายชื่อการปรับแต่งที่ควรอยู่ในนั้น รายชื่อชุดนี้เขียนตอนออกแบบ ว่าในทางทฤษฎีโปรไฟล์นี้ควรทำอะไรได้บ้าง

แนวคิดที่สามคือสัญญาที่ execute ได้จริงและย้อนกลับได้จริง หมายถึงมีโค้ดที่รันได้จริงในเครื่อง มีเส้นทางย้อนกลับที่ทดสอบแล้วว่าทำงาน และผ่านเกณฑ์ห้ามิตที่เล่าไปในบทที่ 3

ปัญหาคือตัวเลข 56 ที่ประกาศไว้ นับจากแนวคิดที่หนึ่งกับสอง ไม่ได้ตรวจสอบว่าทุกรายการผ่านแนวคิดที่สามจริง

พูดอีกแบบ ตัวเลขนั้นนับสิ่งที่ผมรู้จักและตั้งใจให้มี ไม่ใช่สิ่งที่ผมพิสูจน์แล้วว่าทำงาน

ทำไมสามแนวคิดนี้ถึงปนกันได้ตั้งแต่แรก

ผมนั่งคิดย้อนว่ามันปนกันตอนไหน แล้วพบว่าไม่มีจุดเดียวที่ทำให้มันสะสมมาที่ละนิดจากการทำงานปกตินี้แหละ

ตอนเริ่มออกแบบระบบ ผมเขียนบันทึกความรู้ก่อน เพราะอยากเห็นภาพรวมของเทคนิคที่สำรวจมาในบทที่ 1 กับ 2 อันไหนผ่านเกณฑ์ห้ามิต ผมใส่ลงบันทึกนั้นหมด

จากนั้นตอนออกแบบโปรไฟล์ ผมหยิบรายชื่อจากบันทึกความรู้มาจัดกลุ่มว่าอันไหนควรอยู่ SAFE อันไหนควรอยู่ COMPETITIVE ขึ้นนี้ยังเป็นการออกแบบบนกระดาษ ยังไม่มีโค้ดที่รันได้จริงสักบรรทัด

พอมาถึงขั้นเขียนโค้ดจริง ผมเขียนทีละส่วนตามความสำคัญ อันที่เสียงสูงและใช้บ่อย ผมเขียนก่อน อันที่ซับซ้อนกว่า ผมทิ้งไว้ทำทีหลัง

ปัญหาคือหน้าจอกับเอกสารดังตัวเลขมาจากบันทึกความรู้ และรายชื่อโปรไฟล์ ไม่ได้ดึงจากรายการที่เขียนโค้ดจริงเสร็จแล้ว มันเลยประกาศตัวเลขของสิ่งที่ตั้งใจจะมี ไม่ใช่ของที่มีอยู่จริงตอนนั้น

ผมเคยเล่าไว้ในบทที่ 2 ว่าเจอปัญหานี้ในโครงการของคนอื่น ตอนที่คำอธิบายกับโค้ดแยกกันโต เพราะไม่มีอะไรบังคับให้เดินไปด้วยกัน ผมเขียนไว้ว่าจะกลับมาเรื่องนี้อีกทีตอนมันเกิดกับผมเอง

นี่คือจุดนั้น

ผลกระทบเต็ม ๆ

ผลที่ตรงที่สุดวัดได้จากการให้ระบบลองแปลงรายการในปุ่มลัดทุกชุด เป็นการปรับแต่งจริงที่รันได้

การทดลองที่ 02

คำถาม	Quick Action ทั้งเก้าชุดที่อ้างอิงบันทึกความรู้และรายชื่อโปรไฟล์ resolve ผ่านจริงกี่ชุด
วิธี	ให้ระบบแปลงชื่อรายการในปุ่มลัดแต่ละชุดเป็นการปรับแต่งจริงที่รันได้ ตามสัญญาที่ทดสอบแล้ว
ผล	resolve ผ่าน 0 ชุด จากทั้งหมด 9 ชุด เพราะมีรายการที่ไม่มีสัญญาจริงรองรับป้อนอยู่ทุกชุด
บทเรียน	จำนวนที่ประกาศจากบันทึกความรู้และรายชื่อโปรไฟล์ ไม่เท่ากับจำนวนที่ resolve ผ่านสัญญาจริง ต้องนับจากแหล่งเดียวกันเท่านั้น

ตัวเลขที่ลดลงเพราะเจ้าของมันตรวจเอง เชื่อถือได้กว่าตัวเลขที่ไม่มีใครเคยตรวจ

ผมไม่อยากเสี่ยงที่จะบอกผลกระทบทั้งหมด เพราะถ้าเล่าครั้งเดียวก็ไม่ต่างจากที่ผมทำผิดมาตั้งแต่ต้น

ผลที่ตรงที่สุดคือ Quick Action ทั้งเก้าชุด resolve ไม่ผ่านเลยแม้แต่ชุดเดียว

Quick Action คือปุ่มลัดที่รวมการปรับแต่งหลายรายการไว้ในปุ่มเดียว ออกแบบไว้เพื่อให้ผู้ใช้ที่ไม่อยากเลือกทีละรายการกดปุ่มเดียวแล้วได้ครบชุด ทั้งเก้าชุดนี้อ้างอิงรายการจากบันทึกความรู้ และรายชื่อโปรไฟล์ ซึ่งมีรายการที่ไม่มีสัญญาณจริงรองรับนอนอยู่

พอระบบพยายาม resolve คือแปลงชื่อรายการในปุ่มให้เป็นการปรับแต่งจริงที่รันได้ มันเจอรายการที่ไม่มีอยู่จริงในทุกชุด ทำให้ resolve ล้มเหลวทั้งเก้าชุด

พูดตรง ๆ คือพีเจอร์ปุ่มลัดทั้งหมด ที่ผมตั้งใจให้เป็นจุดขายของเครื่องมือ ใช้งานไม่ได้จริงตั้งแต่เริ่มมีการทดสอบ

สิ่งที่ผมคิดตอนนั่งมองหน้าจอ นั้น

ผมนั่งอยู่กับตัวเลขนั้นสักพักก่อนจะทำอะไรต่อ

**ถ้าผมโกหก ผมจะรู้ว่าต้องแก้ไขอะไร แค่
หยุดโกหก**

**แต่ถ้าผมเชื่อผิด ปัญหาไม่ได้อยู่ที่คำพูด
มันอยู่ที่วิธีที่ผมตรวจสอบตัวเอง**

สิ่งที่รู้สึกไม่ใช่ความตกใจว่โค้ดพัง โค้ดพังเป็นเรื่องปกติ ผมแก้บั๊กมาแล้วยี่สิบเจ็ดตัวในโครงการนี้ และไม่มีตัวไหนทำให้รู้สึกแบบนี้

สิ่งที่รู้สึกคือผมเคยยืนพูดต่อหน้าคนอื่นเรื่องตัวเลขนี้

ผมเคยเปิดสไลด์ที่มีเลขห้าสิบหกอยู่บนนั้น เคยอธิบายว่ปุ่มลัดแก้ไขคือสิ่งที่ทำให้เครื่องมือนี้ใช้ง่ายกว่าตัวอื่น และตอนนั้นผมเชื่อทุกคำที่พูด

นั่นคือส่วนที่ทำให้ไม่สบายใจที่สุด ไม่ใช่การที่ผมพูดสิ่งที่ไม่จริง แต่การที่ผมพูดสิ่งที่ไม่จริงโดยเชื่อว่ามันจริง

เพราะถ้าผมโกหก ผมจะรู้ว่าต้องแก้อะไร แค่หยุดโกหก

แต่ถ้าผมเชื่อผิด ปัญหาไม่ได้ยู่ที่คำพูด มันยู่ที่วิธีที่ผมตรวจสอบตัวเอง แปลว่อาจมีเรื่องอื่นที่ผมยังเชื่อผิดยู่ตอนนั้น และผมยังไม่รู้ว่าเรื่องไหน

ผมใช้เวลาสองวันถัดมาไล่ตรวจตัวเลขทุกตัวที่ผมเคยประกาศไว้ ไม่ใช่เพราะสงสัยเป็นเรื่อง ๆ แต่เพราะไม่มีเหตุผลจะเชื่อว่าตัวเลขนี้เป็นตัวเดียวที่ผิด

โชคคิตที่ตัวอื่นตรง แต่ตอนเริ่มไล่ ผมไม่รู้

แต่ระบบไม่เคยไม่ปลอดภัย

สิ่งที่เกินจริงในกรณีนี้คือคำพูด ไม่ใช่พฤติกรรม เอกสารพูดเกินกว่าที่ระบบทำได้จริง

::reality

เชื่อว่า: ตัวเลขที่ประกาศผิดพลาดว่าระบบไม่ปลอดภัยด้วย

วัดได้ว่า: ระบบ fail-closed จับได้ทันทีที่รายการไม่มีสัญญาณจริง เจ็ดเทคนิค
ห้ามการผ่านเข้าระบบได้ 0 รายการ 24 service ที่ป้องกันไว้ถูกปิดได้ 0
รายการ

แต่ระบบเองไม่เคยทำเกินกว่าที่มันควรทำ เพราะ ออกแบบเป็น fail-closed มาตั้งแต่บทที่ 4

ตรงนี้เป็นจุดที่ผมอยากให้อ่านซ้ำ ๆ เพราะเป็นเส้นแบ่งที่สำคัญที่สุด
ของบทนี้

ระบบไม่เคยรับการปรับแต่งที่ไม่มีสัญญาณจริงรองรับ ไม่มีครั้งใดที่
ผู้ใช้กดปุ่ม แล้วเครื่องถูกแก้ค่าโดยไม่มีการตรวจสอบ ไม่มีครั้งใดที่
การย้อนกลับพัง เพราะรายการที่ไม่มีอยู่จริง

เหตุผลคือระบบถูกออกแบบเป็น fail-closed ตั้งแต่บทที่ 4 คือถ้า
รายการไหนไม่ผ่านการตรวจสอบครบทุกชั้น มันจะไม่รัน ไม่ใช้รันไป
ก่อนแล้วหวังว่าจะไม่มีปัญหา

พอ Quick Action พยายาม resolve รายการที่ไม่มีสัญญาณจริง
กลไก fail-closed จับได้ทันทีและปฏิเสธ ผลคือปุ่มกดไม่ได้ ไม่ใช่ปุ่ม
กดได้แล้วพัง

ลองเทียบกับสายไฟในบ้านที่มีเบรกเกอร์ ถ้าสายไฟบางเส้นเดิน
ผิดตั้งแต่ตอนสร้างบ้าน เบรกเกอร์ที่ดีจะตัดไฟทันทีที่ตรวจพบความ
ผิดปกติ ไฟจะดับ ใช้งานไม่ได้ แต่บ้านจะไม่ไหม้ ส่วนบ้านที่ไม่มีเบรก
เกอร์ สายไฟเดินผิดแบบเดียวกันอาจไม่มีอะไรเกิดขึ้นเลยเป็นปี
จนกว่าจะเกิดไฟไหม้จริง

สิ่งที่เกินจริงในกรณีนี้คือคำพูด ไม่ใช่พฤติกรรม เอกสารพูดเกินกว่าที่ระบบทำได้จริง แต่ระบบเองไม่เคยทำเกินกว่าที่มันควรทำ

ผมยืนยันแบบนี้ได้เพราะการทดสอบด้านความปลอดภัย ที่เล่าไปในบทที่แล้ว เจ็ดเทคนิคห้ามถาวรผ่านเข้าระบบได้ 0 รายการ 24 service ที่ป้องกันไว้ถูกปิดได้ 0 รายการ ตัวเลขพวกนี้ไม่เปลี่ยนไปตามการค้นพบเรื่องห้าสิบหกกับสี่สิบสี่ เพราะมันเป็นชั้นป้องกันคนละชั้น ที่ไม่ได้พึ่งความถูกต้องของตัวเลขรวม

เขียนลงเล่ม หรือแก้เจียบ ๆ

หลังจากรู้ผลแล้ว ผมมีทางเลือกอยู่ตรงหน้าสองทาง

***ถ้าผมแก้เจียบ ๆ ครั้งนี้ คนอ่านจะไม่มีทางรู้ว่า
โครงการที่ประกาศตัวเลขสวยงามตลอดทาง
อาจมีจุดที่ตัวเลขไม่ตรงแบบนี้ซ่อนอยู่***

ทางแรกคือแก้เจียบ ๆ เปลี่ยนตัวเลขในเอกสารและหน้าจอจาก 56 เป็น 44 คอมมิตเข้าไปเหมือนการแก้บั๊กทั่วไป ไม่มีใครนอกทีมจะรู้ว่าเคยมีตัวเลข 56 อยู่เลย เพราะไม่มีใครสั่งให้ผมตรวจ และไม่มีใครนอกจากผมที่รู้ว่าต้องตรวจ

ทางที่สองคือเขียนเรื่องนี้ลงในหนังสือที่ผมกำลังเขียนอยู่ ให้คนอ่านเห็นทั้งกระบวนการค้นพบ และตัวเลขที่เปลี่ยน

ผมเลือกทางที่สอง และอยากอธิบายว่าทำไม

เหตุผลแรกคือ หนังสือเล่มนี้ตั้งใจเล่าเรื่องการแยกของจริงออกจากคำโฆษณา ถ้าผมเชื่อว่าตัวเองพูดเกินจริงแล้วเจ็บใจ ทุกบทก่อนหน้าที่ผมเล่าเรื่องคนอื่นพูดเกินจริง จะกลายเป็นการชี้นิ้ว โดยไม่ยอมมองตัวเอง

เหตุผลที่สองคือถ้าผมแก้เจ็บ ๆ ครั้งนี้ คนอ่านจะไม่มีทางรู้ว่าโครงการที่ประกาศตัวเลขสวยงามตลอดทาง อาจมีจุดที่ตัวเลขไม่ตรงแบบนี้ซ่อนอยู่ โดยไม่มีใครตรวจ การเขียนออกมาคือการบอกว่าวิธีตรวจสอบที่ผมใช้ ใช้ได้จริง เพราะมันจับความผิดของตัวเองได้

ตัวเลขที่เชื่อได้มากกว่า

มีประโยคหนึ่งที่ผมคิดขึ้นมาระหว่างเขียนบทนี้ และอยากให้มันยืนอยู่ตรงนี้ด้วยตัวเอง

สี่สิบห้าคือจำนวนที่ผ่านเกณฑ์ห้ามติดบนกระดาด สี่สิบสี่คือจำนวนที่มีสัญญาณทำงานจริงตรวจแล้ว

ห้าสิบหกไม่เกี่ยวกับทั้งสองตัวนี้ เพราะนับจากบันทึกความรู้คุณละชุดกับรายการที่ทำจริง

ตัวเลขที่ลดลงเพราะเจ้าของมันตรวจเอง เชื่อถือได้กว่าตัวเลขที่ไม่มีใครเคยตรวจ

ตัวเลข 56 ที่เคยประกาศไว้ ฟังดูดีกว่า 44 แน่หนอ มันมากกว่ามันเป็นตัวเลขที่ขายง่ายกว่าในสไลด์ แต่มันไม่มีใครตรวจสอบมันเลย จนกระทั่งวันที่ 2 สิงหาคม

ตัวเลข 44 ที่เหลืออยู่ตอนนี้ น้อยกว่า ฟังดูด้อยกว่าในสไลด์ แต่ทุกตัวในสี่สิบสี่รายการนั้น ผ่านการตรวจสอบสามแนวคิดครบแล้วจริง มีบันทึกความรู้รองรับ อยู่ในโปรไฟล์ที่ถูกต้อง และมีสัญญาณที่ execute

ได้จริงกับย้อนกลับได้จริง

ถ้าต้องเลือกระหว่างตัวเลขที่ใหญ่กว่า แต่ไม่มีใครยืนยัน กับตัวเลขที่เล็กกว่า แต่ผ่านการตรวจอย่างจริงจังจากคนที่ม่แรงจูงใจจะไม่พบนั่น ผมเลือกตัวเลขหลังทุกครั้ง

แล้วสี่สิบห้าที่เล่าไว้ตอนต้นเล่มละ

ผู้อ่านที่ตามตัวเลขมาตลอดน่าจะสะดุดตรงนี้ เพราะในบทที่ 1 กับบทที่ 3 ผมบอกว่ามีสี่สิบห้าเทคนิคที่ผ่านเกณฑ์ห้ามิติ แต่บทนี้ผมพูดถึงสี่สิบสี่

สองตัวเลขนี้ไม่ใช่ตัวเดียวกัน และผมควรอธิบายให้ชัดตั้งแต่ต้น

สี่สิบห้าคือจำนวนเทคนิคที่ผ่านการให้คะแนนตามเกณฑ์ห้ามิติ เป็นการตัดสินบนกระดาษว่าเทคนิคไหนควรค่าแก่การเอามาทำ

สี่สิบสี่คือจำนวนที่ผมทำงานมีสัญญาการทำงานที่ตรวจสอบ แล้วจริงในโปรแกรม

หนึ่งรายการที่ต่างกันคือเทคนิคที่ผ่านเกณฑ์ แต่พอลงมือทำจริงแล้วผมหาวิธีย้อนกลับที่เชื่อถือได้ไม่เจอ มันแก้ค่าที่ระบบเขียนทับตัวเองได้ในภายหลัง แปลว่าค่าที่ผมบันทึกไว้ก่อนแก้ อาจไม่ใช่ค่าที่ควรคืนกลับไป แล้วเมื่อถึงเวลาย้อน

ผมจึงไม่ใส่มัน เพราะเกณฑ์ข้อที่ห้าของผมคือย้อนกลับได้ และถ้าย้อนกลับได้ไม่จริง ก็ไม่ควรผ่าน

ส่วนตัวเลข 56 ไม่เกี่ยวกับทั้งสี่สิบห้าและสี่สิบสี่ มันเป็นตัวเลขที่นับจากรายการในบันทึกความรู้ คนละชุดกับรายการที่ทำจริง และนั่นคือความสับสนทั้งหมดที่บทนี้เล่ามา

สิ่งที่แก้ไปแล้วและสิ่งที่ยังต้องทำ

หลังพบปัญหา ผมแก้สามส่วนพร้อมกัน

ส่วนแรกคือแก้ตัวเลขในเอกสาร และหน้าจอทุกจุดที่เคยเขียนว่า 56 ให้เป็น 44 ใส่ตรวจทุกไฟล์ที่อ้างถึงตัวเลขนี้ ไม่ใช่แค่จุดที่เจอโดยบังเอิญ

ส่วนที่สองคือแก้กระบวนการ ที่ทำให้หน้าจอติงตัวเลขจากบันทึกความรู้ และรายชื่อโปรไฟล์โดยตรง เปลี่ยนให้ดึงจากรายการที่ resolve ผ่านสัญญาจริงเท่านั้น เพื่อไม่ให้เกิดช่องว่างแบบนี้ซ้ำในอนาคต ถ้ามีการปรับแต่งใหม่เพิ่มเข้ามา

ส่วนที่สามคือ Quick Action ทั้งเก้าชุด ต้องปรับให้อ้างอิงเฉพาะรายการที่มีอยู่จริง ในสลิปสักรายการนี้ บางชุดก็ต้องตัดรายการที่ไม่มีสัญญาจริงออกไป ทำให้บางปุ่มลัดทำได้น้อยกว่าที่เคยตั้งใจไว้

งานส่วนที่สามนี้ยังไม่เสร็จสมบูรณ์ทั้งเก้าชุด ตอนที่ผมเขียนบทนี้ และผมเลือกจะบอกตรงนี้เลยว่ายังไม่เสร็จ แทนที่จะรอให้เสร็จก่อนแล้วเขียนย้อนหลังให้ดูเรียบร้อย

ตอนที่บอกทีมงานเป็นครั้งแรก

ผมไม่ได้เก็บเรื่องนี้ไว้กับตัวเองนาน วันรุ่งขึ้นผมเล่าให้เพื่อนที่ช่วยดูงานฟังตรง ๆ ว่าเจออะไร

เขานิ่งไปพักหนึ่งก่อนถาม คำถามแรกไม่ใช่ว่าใครทำผิด แต่เป็นคำถามว่ามีใครเคยกดปุ่มที่พังไปแล้วหรือยัง ผมตอบว่าไม่มี เพราะระบบทดสอบยังไม่เปิดให้คนนอกใช้ในชวงนั้น

เขาถึงค่อยผ่อนคลาย แล้วพูดประโยคหนึ่งที่ผมจำมาถึงตอนเขียนบทนี้ เขาบอกว่าถ้าเรื่องนี้เกิดตอนมีคนใช้จริงหลักร้อยคนแล้ว การแก้จะยากกว่านี้มาก เพราะต้องอธิบายว่าทำไมปุ่มที่เคยกดได้ถึงกดไม่ได้อีกแล้ว

ผมคิดว่าเวลาที่พบเรื่องนี้เป็นเรื่องที่โชคดี ไม่ใช่เพราะผมวางแผนให้เจตตอนี่ แต่เพราะผมเลือกจะตรวจสอบตัวเอง ก่อนที่จะมีใครมาถามแทน ถ้าผมรอให้มีคนถามก่อน ผมอาจจะเลือกทางแก้เจียบ ๆ แทนที่จะเขียนออกมาแบบนี้ก็ได้ เพราะแรงกดดันตอนนั้นจะต่างกันมาก

ทำไมผมไม่โทษเครื่องมือหรือกระบวนการ

มีคนถามผมทีหลังว่าทำไมไม่โทษว่าเป็น เพราะเครื่องมือติดตามงานไม่ดี หรือกระบวนการพัฒนาหลวมเกินไป

ผมคิดว่าคำถามนั้นวางผิดที่ ต่อให้มีเครื่องมือติดตามงานที่ดีที่สุดในโลก ถ้าคนที่กรอกข้อมูลเข้าไปไม่แยกสามแนวคิดออกจากกัน ตั้งแต่ในหัว เครื่องมือก็จะบันทึกความสับสนนั้นไว้อย่างเป็นระบบเท่านั้นเอง

ปัญหาของผมไม่ใช่ขาดเครื่องมือ ปัญหาคือผมไม่เคยเขียนออกมาชัด ๆ ว่าสามแนวคิดนี้ต่างกัน จนกว่าจะมานั่งไล่นับตัวเลขด้วยมือในวันที่ 2 สิงหาคม การแก้ที่ถูกต้องจึงไม่ใช่การซื้อเครื่องมือใหม่ แต่คือการเขียนกฎให้ชัดว่าตัวเลขที่ประกาศต้องมาจากรายการที่ resolve ผ่านสัญญาจริงเท่านั้น แล้วบังคับให้ทุกจุดที่แสดงตัวเลขดึงจากแหล่งเดียวกันนั้น

บทเรียนที่กว้างกว่าตัวเลขเดียว

ถ้าถามว่าเรื่องนี้สอนอะไรผมมากที่สุด คำตอบไม่ใช่เรื่องต้องนับให้ถูก คำตอบคือการบนแนวคิดที่ดูเหมือนกันเข้าด้วยกัน เป็นเรื่องที่เกิดขึ้นได้ง่ายมาก โดยไม่มีใครตั้งใจ

**ตัวเลขที่ประกาศออกไปจะค่อย ๆ เพี้ยนจาก
ความจริงไปเรื่อย ๆ โดยไม่มีใครรู้ตัว จนกว่าจะมี
คนลงไปนับใหม่ด้วยมือ**

บันทึกความรู้ รายชื่อสมาชิกโปรไฟล์ และสัญญาที่ทำงานได้จริง ทั้งสามอย่างนี้ล้วนเกี่ยวกับ "การปรับแต่ง" เหมือนกัน ใช้คำเดียวกัน อยู่ในหัวข้อเดียวกัน แต่ตอบคำถามคนละคำถาม บันทึกความรู้ตอบว่านี่คืออะไร รายชื่อโปรไฟล์ตอบว่าควรอยู่กลุ่มไหน สัญญาจริงตอบว่าทำได้จริงหรือยัง

ผมคิดว่าปัญหาแบบนี้ไม่ได้จำกัดอยู่แค่โครงการซอฟต์แวร์ มันเกิดได้กับทุกงานที่มีชั้นวางแผน ชั้นออกแบบ และชั้นทำจริง แยกกันตามเวลา ถ้าไม่มีจุดที่บังคับให้สามชั้นนี้มาเช็กกันเอง ตัวเลขที่ประกาศออกไปจะค่อย ๆ เพี้ยนจากความจริงไปเรื่อย ๆ โดยไม่มีใครรู้ตัว จนกว่าจะมีคนลงไปนับใหม่ด้วยมือแบบที่ผมทำในวันที่ 2 สิงหาคม

ก่อนไปบทถัดไป

ความซื่อสัตย์แบบเดียวกันที่ทำให้ผมเขียนบทนี้ ต้องใช้กับเรื่องอื่นในโครงการด้วย ไม่ใช่แค่ตัวเลขที่ตรวจแล้วพบว่าผิด แต่รวมถึงเรื่องที่ยังไม่มีคำตอบเลยด้วยซ้ำ

มีคำถามหนึ่งที่ผมยังตอบไม่ได้จนถึงตอนนี้ และมันเป็นคำถามที่คนอ่านทุกคนอยากรู้มากที่สุด

เครื่องมือนี้ทำให้เร็วขึ้นจริงไหม

บทถัดไปคือเรื่องของคำถามนั้น และเหตุผลที่ผมยังไม่มีข้อมูลจริงมาตอบ

คำถามถึงคุณ

ลองนึกถึงตัวเลขหนึ่งที่คุณเคยพูดออกไปโดยเชื่อมั่นว่ามันจริง แล้วไม่มีใครสั่งให้คุณตรวจสอบมันเลย

ถ้าวันนี้คุณลองไล่นับใหม่ด้วยมือ คุณมั่นใจแค่ไหนว่าตัวเลขนั้นจะยังตรงเหมือนวันที่คุณพูดครั้งแรก

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน ตัวเลขที่เอกสารประกาศไว้ ควรตรงกับของจริงในระบบเสมอ เพราะเป็นคนเดียวกันที่เขียนทั้งสอง

หลังอ่าน ตัวเลขที่ไม่มีจุดบังคับให้เช็กกับของจริง จะเพี้ยนไปเรื่อย ๆ โดยไม่มีใครรู้ตัว จนกว่าจะมีคนลงไปนับใหม่ด้วยมือ

- ✓ รู้ว่าบันทึกความรู้ รายชื่อโปรไฟล์ และสัญญาที่ execute ได้จริง เป็นสามแนวคิดที่ต้องแยกกัน ไม่ใช่แนวคิดเดียว
- ✓ รู้ว่าตัวเลข 56 กับ 44 ต่างกันเพราะแหล่งที่นับต่างกัน และทำไม 44 น่าเชื่อกว่า 56
- ✓ รู้ว่าเอกสารพูดเกินจริงได้ แต่ระบบที่ออกแบบเป็น fail-closed จะไม่ทำเกินกว่าที่มันควรทำ

บทที่ 7 — สิ่งที่ยังพิสูจน์ไม่ได้

คำถามที่ 11

ถ้ายังไม่มีข้อมูลจากผู้เล่นจริงเลยแม้แต่คนเดียว เราจะรู้ได้อย่างไรว่าการปรับแต่งนี้ช่วยได้จริง

คำถามที่ผมกลัวที่สุดตอนเล่าโครงการนี้ให้คนฟัง เป็นคำถามที่สั้นที่สุด

"แล้วมันเร็วขึ้นจริงไหม"

คำตอบที่ซื่อสัตย์ที่สุด ณ วันที่เขียนหนังสือเล่มนี้คือ ผมยังไม่รู้ ไม่ใช่ว่าผมไม่มีตัวเลขอะไรเลย ผมวัดของผมเองมาแล้ว และตัวเลขที่ได้ก็ดีขึ้น แต่ผมรู้ว่าตัวเลขที่ผมวัดบนเครื่องผมเอง มีค่าเท่ากับตัวเลข ที่คนทำเครื่องมืออื่นวัดบนเครื่องตัวเอง ซึ่งบทที่ 2 ผมบอกไปแล้วว่ามันเชื่อไม่ได้

ผมจะเถียงเรื่องนี้กับตัวเองไม่ได้ ถ้าเกณฑ์ที่ผมใช้ตัดคนอื่นออก แล้วผมยกเว้นให้ตัวเอง เกณฑ์นั้นก็ไม่ใช่เกณฑ์

ทำไมการวัดว่าเร็วขึ้นถึงยากกว่าที่คิด

ลองนึกภาพว่าคุณอยากรู้ว่า กาแฟร้านใหม่อร่อยกว่าร้านเดิมไหม

วิธีที่ง่ายที่สุดคือไปกินร้านใหม่วันนี้ แล้วเทียบกับความจำว่าร้านเดิมเป็นยังไง

ปัญหาคือวันนี้คุณหลับเต็มอิ่มกว่า อากาศเย็นกว่า คุณกินตอนเช้า ไม่ใช่ตอนบ่าย และคุณรู้อยู่แล้วว่านี่คือร้านใหม่ที่คนบอกที่ดี

ถ้าคุณตอบว่าอร่อยกว่า คุณไม่รู้เพราะกาแฟหรือเพราะอย่างอื่น

การวัดประสิทธิภาพคอมพิวเตอร์เจอปัญหาเดียวกัน แต่มีตัวแปรมากกว่ากาแฟหลายเท่า

เครื่องที่รีสตาร์ทใหม่ต่างจากเครื่องที่เปิดมาแปดชั่วโมง เครื่องที่อุณหภูมิ 45 องศาต่างจากเครื่องที่ 75 องศาเพราะซีพียูลดความเร็วตัวเองเมื่อร้อน เกมที่อัปเดตเมื่อวานต่างจากเวอร์ชันเมื่อสัปดาห์ก่อน และไดรเวอร์การ์ดจอที่อัปเดตอัตโนมัติตอนกลางคืน ก็เปลี่ยนผลได้

ถ้าคุณวัดก่อนปรับตอนเช้า แล้ววัดหลังปรับตอนเย็น ความต่างที่ได้อาจไม่เกี่ยวกับการปรับแต่งเลย

ตัวเลขเดียวที่บอกอะไรไม่ได้

มีอีกเรื่องที่ผมเข้าใจผิดตอนแรก คือผมคิดว่าเฟรมเรตเฉลี่ยเป็นตัวเลขที่บอกทุกอย่าง

สมมติเกมหนึ่งให้เฟรมเรตเฉลี่ย 200 เฟรมต่อวินาที ฟังดูดีมาก

แต่ตัวเลขเฉลี่ยซ่อนอะไรไว้เยอะ ถ้าในหนึ่งวินาทีนั้น 195 เฟรมมาถึงเร็วสม่ำเสมอ แล้วมีอยู่ 5 เฟรมที่มาช้ากว่าปกติสิบเท่า ค่าเฉลี่ยยังสวยอยู่ แต่คนเล่นจะรู้สึกภาพสะดุด

ในเกมยิงที่ตัดสินกันด้วยเลี้ยววินาที การสะดุดห่าครั้งต่อวินาทีสำคัญกว่าค่าเฉลี่ยที่สูง

เรื่องนี้ทำให้ผมต้องวัดค่าอีกชุดหนึ่ง คือกลุ่มเฟรมที่ช้าที่สุด ถ้ามีเฟรมทั้งหมดร้อยเฟรม ผมสนใจว่าเฟรมที่ช้าที่สุดหนึ่งเฟรมนั้นช้าแค่ไหน

การปรับแต่งที่ทำให้ค่าเฉลี่ยดีขึ้น แต่ทำให้เฟรมที่ช้าที่สุดแย่งคือการปรับแต่งที่ทำให้เกมรู้สึกแย่ง ทั้งที่ตัวเลขบนหน้าจอดีขึ้น

ผมจึงเก็บทั้งค่าเฉลี่ยและกลุ่มที่ช้าที่สุด เพราะถ้าเก็บแค่ตัวเดียวก็ตอบคำถามที่คนเล่นสนใจจริง ๆ ไม่ได้

ผู้เล่นไม่ใช่เครื่องจักร

มีตัวแปรอีกชุดที่ผมประเมินต่ำไปตอนแรก คือตัวผู้เล่นเอง

คนเล่นเกมช่วงเข้ากับช่วงตึกทำคะแนนไม่เท่ากัน คนที่เพิ่งเล่นไปสามรอบจะแม่นกว่ารอบแรกเพราะมือชิน และคนที่เล่นไปแปดรอบจะเริ่มแย่งเพราะเหนื่อย

นี่แปลว่าถ้าผมให้คนเล่นบนเครื่องที่ไม่ได้ปรับก่อน แล้วให้เล่นบนเครื่องที่ปรับแล้วทีหลัง ผลที่ดีขึ้นอาจมาจากการที่เขาชินมือแล้ว ไม่ใช่จากการปรับแต่ง

และถ้าผมสลับให้เล่นเครื่องที่ปรับแล้วก่อน ผลก็จะเอียงไปทางตรงข้าม

การไม่พบความแตกต่าง กับการพบว่าไม่มีความแตกต่าง เป็นคนละเรื่องกัน

ปัญหานี้แก้ได้ ไม่ใช่ด้วยการหาลำดับที่ถูกต้อง แต่ด้วยการยอมรับว่า ไม่มีลำดับที่ถูกต้อง แล้วสลับลำดับระหว่างคน

คนที่หนึ่งเริ่มเครื่อง A คนที่สองเริ่มเครื่อง B คนที่สามเริ่ม A คนที่สี่เริ่ม B ไล่แบบนี้ไป

พอเอาผลมารวมกัน ผลของการขึ้นมือจะกระจายเท่า ๆ กันทั้งสองฝั่ง แล้วมันจะหักกันเอง วิธีนี้เรียกว่าการถ่วงดุลลำดับ

เหมือนการชิมกาแฟสองแก้วโดยให้คนครึ่งหนึ่งชิมแก้วซ้ายก่อน อีกครึ่งชิมแก้วขวา ก่อน ถ้าแก้วแรกที่ชิมได้เปรียบเสมอ การสลับจะทำให้ความได้เปรียบนั้นตกกับทั้งสองแก้วเท่ากัน

แผนแรกที่ใช้ไม่ได้

ตอนออกแบบการทดลองครั้งแรก ผมคิดว่าจะทำแบบนี้

ให้ผู้เล่นคนหนึ่งเล่นบนเครื่องเดียว ไล่จากเครื่องที่ไม่ปรับอะไร แล้วปรับเป็นโปรไฟล์แรก เล่นอีก แล้วปรับเป็นโปรไฟล์ที่สอง เล่นอีก แล้วปรับเป็นโปรไฟล์ที่สาม เล่นอีก

ฟังดูดี เพราะได้เทียบทั้งสามโปรไฟล์ในคนเดียวกัน

ผมเขียนแผนนี้ไปได้หลายหน้า ก่อนจะเจอว่ามันมีปัญหาที่แก้ไม่ได้เลย

โปรไฟล์ทั้งสามของผมเป็นแบบสะสม โปรไฟล์แรกมีสิบสามรายการ โปรไฟล์ที่สองมีสามสิบแปดรายการ รวมสิบสามรายการแรกไว้ด้วย และโปรไฟล์ที่สามมีสี่สิบสี่รายการ ซึ่งรวมทั้งหมด

เหมือนชั้นเสื้อผ้าที่ผมเล้าไว้ในบทยี่ 4 คือถอดตัวในสุดโดยไม่ถอดตัวนอกก่อนไม่ได้

แปลว่าลำดับของมมถูกบังคับให้เป็นแบบเดียว มจะให้ใครเล่นโปรไฟล์ที่สองก่อนโปรไฟล์แรก ไม่ได้ในทางตรรกะ เพราะโปรไฟล์ที่สองมีโปรไฟล์แรกอยู่ในนั้นแล้ว

การถ่วงดุลลำดับจึงเป็นไปไม่ได้ ไม่ใช่เพราะผมขี้เกียจ แต่เพราะโครงสร้างของสิ่งที่ผมสร้างไม่อนุญาต

ผลคือถ้าคนเล่นดีขึ้นในเงื่อนไขที่สี่ ผมจะบอกไม่ได้เลยว่าเพราะโปรไฟล์ดีขึ้น หรือเพราะเขาเล่นมาสามรอบแล้ว สองอย่างนี้ปนกันจนแยกไม่ออก

ตอนนั้นผมนั่งมองแผนที่เขียนไปแล้วอยู่พักหนึ่ง แล้วลบทิ้ง

แผนที่สอง

แผนที่ใหม่ตัดปัญหานั้นออกด้วยวิธีที่ตรงไปตรงมา คือใช้สองเครื่องแทนเครื่องเดียว

เครื่อง A เป็น Windows ตามค่าเริ่มต้น ไม่ติดตั้งอะไรเลย

เครื่อง B ติดตั้งโปรไฟล์ที่แรงที่สุดไว้ล่วงหน้า แล้วไม่แตะอีก

สองเครื่องนี้อยู่ในสภาพเดิมตลอดการทดลอง ไม่มีการปรับกลับ ไปกลับมาระหว่างคน ผมเลยสลับลำดับได้จริง คนรหัสสี่เริ่มเครื่อง A คนรหัสคู่เริ่มเครื่อง B

ผมเพิ่มปัจจัยที่สองเข้าไปด้วย คือระดับความยากของบอทในเกมสามระดับ

เหตุผลที่ผมเพิ่มไม่ใช่เพื่อให้ดูซับซ้อน แต่เพราะผมสงสัยว่า ผลของการปรับแต่งอาจไม่เท่ากันในทุกสถานการณ์ ถ้างานง่าย เครื่องอาจไม่ได้ทำงานหนักพอที่จะเห็นความต่าง ถ้างานหนัก ความต่างอาจชัดเจนขึ้น

คำถามนี้ตอบได้ก็ต่อเมื่อวัดทั้งสองระดับ

สองเครื่องคุณสามระดับความยาก ได้หกเงื่อนไข เล่นสามรอบต่อเงื่อนไข เป็นสิบแปดรอบต่อคน ใช้เวลาประมาณหนึ่งชั่วโมง

การทดลองที่ 03

คำถาม	การปรับแต่งช่วยผู้เล่นจริงได้แค่ไหน และผลต่างกันไหมตามระดับความยากของงาน
วิธี	แผนสองเครื่อง (ปรับ กับ ไม่ปรับ) คุณสามระดับความยากบอท รวมหกเงื่อนไข เล่นสามรอบต่อเงื่อนไข สลับลำดับเครื่องเริ่มต้นระหว่างคนรหัสคู่กับคู้
ผล	เก็บได้ 0 จาก 10 คนที่วางแผนไว้ ยังไม่เริ่มเก็บข้อมูลจริง
บทเรียน	ลอกแผนวิเคราะห์และแผนถ่วงดุลลำดับไว้ก่อนเก็บข้อมูลเพื่อกันตัวเองเลือกวิธีที่ให้ผลตามที่หวังไว้ทีหลัง

ทำไมต้องสามรอบ ไม่ใช่รอบเดียว

การเล่นรอบเดียวต่อเงื่อนไขจะเร็วกว่ามาก และผมก็อยากทำแบบนั้น

ปัญหาคือผลของการเล่นเกมแปรปรวนสูงมากตามธรรมชาติ รอบหนึ่งคุณอาจเล็งแม่นผิดปกติ อีกรอบมือลั่น สองรอบนี้ต่างกันได้มากกว่าผลของการปรับแต่งที่ผมกำลังพยายามวัด หลายเท่า

ถ้าผมเก็บรอบเดียว ตัวเลขที่ได้อาจเป็นแค่ความบังเอิญของรอบนั้น

การเก็บสามรอบแล้วเฉลี่ยไม่ได้ทำให้ความแปรปรวนหายไป แต่มันทำให้ความบังเอิญมีน้ำหนักน้อยลง เหมือนการชั่งน้ำหนักสามครั้งแล้วเฉลี่ย แทนที่จะเชื่อตัวเลขจากการชั่งครั้งเดียวนบนตาชั่งที่สั่น

ทำไมสามไม่ใช่สิบ เพราะสิบแปดรอบก็ใช้เวลาหนึ่งชั่วโมงแล้ว ถ้าเพิ่มเป็นสามสิบรอบ ผู้เล่นจะเหนื่อยจนผลเพี้ยนจากความล้าแทน

ตัวเลขสามจึงไม่ใช่ตัวเลขที่ดีที่สุดในทางสถิติ มันคือตัวเลขที่ดีที่สุดเมื่อคิดรวมกับข้อจำกัด ว่าคนเราเล่นเกมต่อเนื่องได้นานแค่ไหน ก่อนจะเริ่มแย่งลง

สิ่งที่แผนนี้แลกไป

แผนสองเครื่องแก้ปัญหาเรื่องลำดับได้ แต่มันมีราคา

ราคานั่นคือผมเทียบได้แค่สองสภาพ คือเครื่องที่ไม่ปรับกับเครื่องที่ปรับเต็มที่ ผมไม่ได้วัดว่าโปรไฟล์กลางให้ผลเท่าไร

น่าเสียดายอยู่ เพราะโปรไฟล์กลางคือตัวที่ผมคิดว่าคนส่วนใหญ่ควรใช้จริง

ผมเลือกแบบนี้เพราะคิดว่าคำตอบที่เชื่อได้เรื่องสองสภาพ มีค่ามากกว่าคำตอบที่เชื่อไม่ได้เรื่องสี่สภาพ แต่ผมเขียนข้อจำกัดนี้ไว้ตรงๆ ไม่ได้ปิด

ถ้ามีคนถามว่าโปรไฟล์กลางได้ผลแค่ไหน คำตอบคือผมยังไม่มีข้อมูล



คนสองคนวิ่งแข่งกันได้ ถ้าสนามเดียวกัน

สิบคน มาจากไหน

ตัวเลขสิบคนไม่ได้มาจากการเดา แต่ก็ไม่ได้มาจากที่ที่มั่นคงนัก เหมือนกัน

มีวิธีคำนวณอยู่ว่าถ้าอยากตรวจจับความแตกต่างขนาดหนึ่ง ด้วยความมั่นใจระดับหนึ่ง ต้องใช้ผู้เข้าร่วมกี่คน ผมใส่ตัวเลขที่ต้องการเข้าไป แล้วได้คำตอบว่าอย่างน้อยแปดคน ผมเผื่อเป็นสิบเผื่อมีคนถอนตัว

ปัญหาอยู่ที่ตัวเลขที่ผมใส่เข้าไป

การคำนวณนั้นต้องบอกก่อนว่า "ความแตกต่างขนาดหนึ่ง" ที่ว่านั้นใหญ่แค่ไหน ตรงนี้ผมยังไม่รู้ เพราะยังไม่มีใครวัด ผมก็ต้องเดาจากงานที่ใกล้เคียง

ผมเลือกเดาว่าผลจะใหญ่ เป็นการเดาที่ทำให้ใช้คนน้อยลง

ถ้าผลจริงมีขนาดกลางหรือเล็ก การทดลองของผมจะตรวจไม่พบ และผมจะได้ข้อสรุปว่า "ไม่พบความแตกต่าง" ทั้งที่ความแตกต่างอาจมีอยู่จริง

นี่คือข้อจำกัดที่ผมต้องเขียนไว้ให้ชัดเมื่อรายงานผล ไม่ใช่ซ่อนไว้ในเชิงอรรถ

การไม่พบความแตกต่าง กับการพบว่าไม่มีความแตกต่าง เป็นคนละเรื่องกัน และสิบคนบอกได้แค่อย่างแรก

เทียบกับของจริง

สิ่งที่เชื่อ ถ้าออกแบบการทดลองรอบคอบพอ ก็แปลว่างานวิจัยนั้นมีผลลัพธ์ที่เชื่อได้แล้ว

สิ่งที่วัดได้ ยังเก็บข้อมูลจริงได้ 0 จาก 10 คนที่วางแผนไว้ ตัวเลขแปดคนที่คำนวณได้คือขั้นต่ำจากการเดาขนาดผล ไม่ใช่ผลที่วัดจริง

ตัวเลขสิบคนมาจากการคำนวณขนาดตัวอย่างที่ต้องเดาขนาดผลก่อน ไม่ใช่ตัวเลขที่ยืนยันได้ว่าเพียงพอจริง

วัดสิ่งที่คนโกหกไม่ได้

มีการตัดสินใจอีกข้อ ที่ผมคิดว่าสำคัญที่สุดในการออกแบบทั้งหมด ผมไม่ได้วัดแค่เฟรมเรต

เฟรมเรตเป็นตัวเลขที่ผู้เล่นเห็นบนจอ ตรงนี้แหละที่เป็นปัญหา เพราะพอเขาเห็นว่าตัวเลขสูงขึ้น เขารู้สึกว่าเล่นได้ดีขึ้น และถ้าเขารู้ว่าเครื่องนี้คือเครื่องที่ปรับแล้ว เขาอาจตอบแบบสอบถามในทางที่ดีขึ้นด้วย

ผมจึงวัดค่าอีกตัวที่ชื่อ DPC latency

ค่านี้วัดว่าระบบใช้เวลานานเท่าไร ในการจัดการงานระดับล่างที่ต้องทำให้เสร็จก่อนงานอื่น เป็นค่าที่วัดจากภายในระบบปฏิบัติการโดยตรง

จุดสำคัญคือ ผู้เล่นรู้สึกถึงค่านี้ไม่ได้ เขามองไม่เห็น เขาบอกไม่ได้ว่ามันสูงหรือต่ำ และเขาแกล้งทำเป็นรู้สึกก็ไม่ได้

ถ้าค่านี้ดีขึ้น มันดีขึ้นเพราะระบบเปลี่ยนจริง ไม่ใช่เพราะใครคาดหวังอะไร

นี่คือคำตอบที่ผมเตรียมไว้ สำหรับคนที่จะถามว่าผลที่ได้เป็นแค่ความรู้สึกหรือเปล่า ถ้าตัวเลขที่คนรู้สึกไม่ได้ก็ยิ่งดีขึ้น คำอธิบายว่าเป็นความคาดหวังก็ใช้ไม่ได้

การลือกคำตอบก่อนเห็นข้อมูล

มีอีกอย่างที่ผมทำ และเป็นอย่างที่ผมคิดว่าคนนอกวงวิจัยมักไม่รู้ว่มันสำคัญ

ผมเขียนแผนวิเคราะห์ทั้งหมดให้เสร็จ ก่อนเก็บข้อมูลแม้แต่คนเดียว
แผนนั้นระบุว่าจะใช้วิธีทางสถิติตัวไหน ในเงื่อนไขอะไร ถ้าข้อมูลมีลักษณะแบบหนึ่งจะใช้วิธีไหน ถ้ามีลักษณะอีกแบบจะเปลี่ยนไปใช้วิธีไหน และเกณฑ์ที่จะบอกว่าผลนั้นมีความหมายอยู่ที่เท่าไร

ทำไมต้องลือกไว้ก่อน

เพราะถ้าไม่ลือก ผมจะเห็นข้อมูลก่อนแล้วค่อยเลือกวิธี และมนุษย์ทุกคนรวมถึงผมด้วย มีแนวโน้มจะเลือกวิธีที่ให้ผลที่เราหวังไว้
เรื่องนี้ไม่ต้องมีเจตนาโกงเลยก็เกิดขึ้นได้ คุณลองวิธีหนึ่ง ผลไม่ขัด คุณคิดว่า "เฮ้ วิธีนี้อาจไม่เหมาะกับข้อมูลแบบนี้" แล้วลองอีกวิธี ผลขัดขึ้น คุณก็ใช้วิธีที่สอง

ทุกขั้นตอนฟังดูมีเหตุผล แต่ผลรวมคือคุณเลือกคำตอบที่อยากได้ แล้วหาวิธีที่พาไปถึงคำตอบนั้น

การลือกแผนไว้ก่อนคือการมัดมือตัวเองไว้ ตอนที่ยังไม่มีแรงจูงใจจะโกง เพื่อกันตัวเองในตอนที่จะมี

การเลือกวิธีวัดหลังเห็นผลแล้ว คือการเลือกคำตอบที่อยากได้

สถานะจริง ณ วันที่เขียน

ทุกอย่างที่ผมเล่าไปในบทนี้คือการออกแบบ

การเก็บข้อมูลจริงยังไม่เริ่ม ศูนย์จากสิบคน

เทียบกับของจริง

สิ่งที่เชื่อ การรอเก็บข้อมูลให้ครบก่อนเขียนหนังสือ จะทำให้เล่มนี้
น่าเชื่อถือกว่า

สิ่งที่วัดได้ สิ่งพร้อมจริงตอนนี้คือเครื่องมือวัดสามตัวที่ติดตั้งและ
ทดสอบแล้ว ส่วนผู้เข้าร่วมยังอยู่ที่ 0 จาก 10 คน

ภาพ 7.1 สองสิ่งที่คนมักคิดว่าเสร็จพร้อมกัน แยกบนคือเครื่องมือกับการทดสอบ
เสร็จและวัดได้แล้ว แยกล่างคือข้อมูลจากผู้เล่นจริง ยังว่างทั้งแถบ การเขียนแถบ
ล่างให้เต็มตอนนี้ทำได้ง่ายมาก และนั่นคือเหตุผลที่ต้องปล่อยให้มันว่าง

เครื่องมือกับการทดสอบ

10 จาก 10



เขียนเสร็จ ทดสอบผ่าน วัดได้

ข้อมูลจากผู้ใช้จริง

0 จาก 10



ยังไม่ได้เริ่มเก็บ

การเขียนว่าเก็บได้สิบคนตอนที่ยังไม่ได้เก็บ ทำให้ทุกอย่างที่เหลือน่าสงสัย

ที่มา: DECISIONS-LOCKED — จำนวนผู้เข้าร่วมที่เก็บได้จริง

สิ่งที่พร้อมแล้วคือเครื่องมือวัดสามตัวที่ติดตั้ง และทดสอบแล้ว
แบบสอบถาม เอกสารขอความยินยอมที่ระบุสิทธิในการถอนตัว ห้อง
ปฏิบัติการที่จองไว้ เครื่องสองเครื่องที่ตั้งค่าไว้แล้ว และแผนวิเคราะห์
ที่ล็อกไว้

สิ่งที่ยังไม่มีคือผู้เข้าร่วม

ผมคิดอยู่นานว่าจะเขียนหนังสือเล่มนี้ตอนไหน หรือรอให้เก็บข้อมูลครบก่อน

เหตุผลที่เลือกเขียนตอนนี้มีสองข้อ

ข้อแรก เรื่องที่ผมอยากเล่าคือกระบวนการ ไม่ใช่ผลลัพธ์ ถ้าผลออกมาว่าการปรับแต่งช่วยให้สามเปอร์เซ็นต์ หรือช่วยไม่ได้เลย วิธีคิดที่พาไปถึงคำตอบนั้นก็ยิ่งเหมือนเดิม และผมคิดว่าวิธีคิดมีค่ากว่าตัวเลข

ข้อสอง ถ้าผมรอให้ผลออกก่อน ผมจะเขียนย้อนหลัง และการเขียนย้อนหลังมันหลอกคนอ่านได้ง่ายมาก โดยไม่ต้องโกหกเลย แม้แต่คำเดียว

คนที่รู้คำตอบแล้วเขียนย้อนกลับไป จะเลือกเล่าเฉพาะทางที่พาไปถึงคำตอบ ทางที่เดินผิดจะหายไปจากเรื่อง ความลังเลจะหายไป และผู้อ่านจะได้ภาพของคนที่เดินตรงไปข้างหน้าตั้งแต่ต้น

ไม่มีใครเดินตรงแบบนั้นจริงหรอก

เตรียมคำตอบสำหรับผลที่ไม่สวย

คำถามที่ผมถามตัวเองก่อนล็อกแผนคือ ถ้าผลออกมาว่าต่างกันไม่นัยสำคัญ ผมจะทำอย่างไร

คำตอบที่ผมเขียนไว้และจะทำตามคือ รายงานตรงแบบนั้น

ผลที่ไม่มีนัยสำคัญไม่ใช่ความล้มเหลว มันเป็นคำตอบชนิดหนึ่ง มันบอกว่าในบริบทนี้ กับเครื่องคู่นี้ กับผู้เล่นกลุ่มนี้ การปรับแต่งระดับนี้ไม่ให้ผลที่วัดได้

มีค่ากว่าการไม่มีใครวัดเลย และมีค่ากว่าคำอ้างลอย ๆ ว่าเร็วขึ้นสิบห้าเปอร์เซ็นต์

และถ้าผลออกมาแบบนั้น สิ่งที่เหลืออยู่ก็ยิ่งเหลืออยู่ คือเกณฑ์คัดกรอง ที่แยกความเชื่อผิดหักข้อออกจากเทคนิคที่มีหลักฐาน และระบบที่ทดสอบแล้วว่าไม่ยอมให้ปิดสิ่งที่ไม่ควรปิด

สองอย่างนั้นไม่ได้ขึ้นกับว่า เฟรมเรตจะดีขึ้นกี่เปอร์เซ็นต์

ก่อนไปบทสุดท้าย

ผมใช้เจ็ดบทเล่าเรื่องการปรับแต่งคอมพิวเตอร์ แต่สิ่งที่ผมอยากให้ติดไปกับผู้อ่าน ไม่ใช่เรื่องคอมพิวเตอร์

มันคือลำดับคำถามที่ผมใช้ตลอดทั้งโครงการ ใช้ได้กับอะไรก็ได้ที่มีคนมาบอกคุณว่าดี

บทสุดท้ายคือการสรุปคำถามเหล่านั้น ให้อยู่ในรูปที่คุณเอาไปใช้พรุ่งนี้ได้ กับคำแนะนำขั้นถัดไปที่คุณจะทำ

**การลือกแผนไว้ก่อนคือการมัดมือตัวเองไว้ ตอน
ที่ยังไม่มีแรงจูงใจจะโง่ เพื่อกันตัวเองในตอนที่จะมี**

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน การออกแบบการทดลองที่ดี น่าจะแปลว่างานนี้พร้อมสรุปผลได้แล้ว

หลังอ่าน การออกแบบเสร็จ กับการมีข้อมูลจริง เป็นสองขั้นที่ต่างกัน บทนี้เสร็จแค่ขั้นแรก ยังเก็บข้อมูลได้ 0 จาก 10 คน

- ✓ รู้ว่าทำไมการวัดประสิทธิภาพคอมพิวเตอร์ยากกว่าที่คิด และทำไมเฉลี่ยอย่างเดียวไม่พอ
- ✓ รู้จักการถ่วงดุลลำดับ และทำไมโปรไฟล์แบบสะสมทำให้ใช้วิธีนี้ไม่ได้
- ✓ รู้ว่าทำไมต้องล็อกแผนวิเคราะห์ก่อนเห็นข้อมูล และทำไมผลที่ไม่มีนัยสำคัญก็มีค่า

บทที่ 8 — วิธีอ่านคำแนะนำที่คุณจะเจอ พรุ่งนี้

คำถามที่ 12

ครั้งหน้าที่คุณเจอคำแนะนำที่ฟังดูดีมีคนกดถูกใจเยอะ
คุณจะถามอะไรก่อนลงมือทำตาม

เพื่อนผมที่เครื่องเปิดไม่ติดในบทแรก ตอนนี้อยู่เล่นเกมอยู่

ผมถามเขาเมื่อไม่นานมานี้ว่า หลังจากคินันเขาเปลี่ยนอะไรไหม
เขาบอกว่าเปลี่ยน เขาไม่รันสคริปต์จากคลิป์อีกแล้ว

แล้วเขาก็เล่าต่อว่า เดือนก่อนเขาเจอโพสต์หนึ่งที่บอกวิธีทำให้เกม
ลื่นขึ้น มีคนกดถูกใจเยอะมาก เขาอยากลอง แต่ไม่รู้จะดูยังไงว่าอันนี้
ปลอดภัยหรือเปล่า

เขาเลยไม่ทำอะไรเลย

ผมคิดว่าคำตอบนั้นดีกว่าเดิม แต่ยังไม่ใช่ว่าคำตอบที่ดีที่สุด เพราะ
การไม่ทำอะไรเลย ก็แปลว่าคุณเสียโอกาสดูคำแนะนำที่ดีจริงไปด้วย
บทนี้คือความพยายามตอบคำถามของเขา

หาคำถามที่ใช้ได้กับทุกคำแนะนำ

ผมสรุปสิ่งที่ทำมาทั้งหมดเป็นคำถามห้าข้อ ห้าข้อนี้คือเกณฑ์ห้ามิติในบทที่ 3 ที่แปลเป็นภาษาที่ใช้กับอะไรก็ได้

คำถามข้อแรก มันแลกอะไรไป

ถ้าคำแนะนำบอกแต่สิ่งที่คุณได้ ไม่บอกสิ่งที่คุณเสีย นั่นไม่ใช่คำแนะนำที่ครบ อาจไม่ใช่เพราะคนแนะนำตั้งใจปิด แต่อาจเพราะเขาเองก็ไม่วู้ ซึ่งอันตรายพอกัน

ทุกการเปลี่ยนแปลงมีสิ่งที่แลก ถ้าหาไม่เจอ แปลว่ายังหาไม่พอ

คำถามข้อสอง เพราะอะไรมันถึงได้ผล

คำอธิบายที่ดีบอกกลไก ไม่ใช่แค่ผลลัพธ์ ถ้าคำตอบคือ "มันทำให้เร็วขึ้น" นั่นคือการทวนคำถาม ไม่ใช่คำตอบ

คำอธิบายที่บอกกลไกได้ จะตรวจสอบได้ด้วย ส่วนคำอธิบายที่บอกแค่ผล จะเถียงไม่ได้เลยว่าถูกหรือผิด

คำถามข้อสาม ใครวัด และวัดยังไง

ตัวเลขที่คนขายของวัดเอง บนเครื่องตัวเอง ครั้งเดียว ไม่ใช่หลักฐาน มันคือคำโฆษณาที่มีตัวเลข

คำถามที่ควรถามคือวัดกี่ครั้ง เทียบกับอะไร และควบคุมอะไรไว้บ้าง

คำถามข้อสี่ ถ้าไม่ชอบ ย้อนกลับยังงัย

ข้อนี้ทดสอบได้ง่ายที่สุด และคนถามน้อยที่สุด ถ้าคำตอบคือ "ก็ตั้งค่ากลับเป็นเหมือนเดิม" ให้ถามต่อว่าเหมือนเดิมคืออะไร ใครจัดไว้ และถ้าจำไม่ได้จะทำยังงัย

เครื่องมือที่ย้อนกลับได้จริงจะตอบข้อนี้ได้ทันที โดยไม่ต้องคิด

คำถามข้อห้า คนที่แนะนำ ยอมบอกใหม่ว่ามันใช้ไม่ได้ตอนไหน

ข้อนี้เป็นข้อที่ผมคิดว่าบออะไรได้มากที่สุด

ลองใช้ห้าคำถามกับของจริง

ทฤษฎีฟังดูง่าย ลองใช้กับตัวอย่างจริงดู

สมมติคุณเจอโพสต์ที่บอกว่าให้ปิด service ตัวหนึ่งของ Windows แล้วเกมจะลื่นขึ้น มีคนคอมเมนต์ยืนยันเป็นร้อย

คำถามข้อแรก มันแลกอะไรไป ถ้าโพสต์ไม่บอก คุณต้องหาเอง วิธีที่เร็วที่สุดคือค้นชื่อ service นั้นในเอกสารของผู้ผลิตระบบปฏิบัติการที่จะบอกว่ามันมีหน้าที่อะไร และถ้าปิดแล้วอะไรจะไม่ทำงาน

ถ้าคำตอบคือมันจัดการเรื่องการพิมพ์ และคุณไม่มีเครื่องพิมพ์ การแลกนั้นก็สมเหตุสมผล ถ้าคำตอบคือมันเกี่ยวกับการอัปเดตความปลอดภัย การแลกนั้นเป็นอีกเรื่องหนึ่ง

คำถามข้อสอง เพราะอะไรมันถึงได้ผล ถ้าอธิบายบอกว่า service นี้กินซีพียูตลอดเวลา คุณตรวจได้เลยว่ามันกินจริงไหมบนเครื่องคุณ ด้วยเครื่องมือที่ติดมากับ Windows อยู่แล้ว

หลายครั้งคำตอบคือมันกินไม่ถึงหนึ่งเปอร์เซ็นต์ แปลว่าปิดไปก็ไม่ได้อะไร

คำถามข้อสาม ใครวัด ถ้าโพสต์บอกว่าเฟรมเรตเพิ่มจากหนึ่งร้อยสี่สิบ เป็นหนึ่งร้อยหกสิบห้า ให้ถามว่าวัดยังไง ถ้าไม่มีคำตอบ ตัวเลขนั้นก็แค่ความรู้สึกที่มีเลขกำกับ

คำถามข้อสี่ ย้อนกลับยังงัย ข้อนี้มีคำตอบชัดสำหรับกรณีนี้ คือเปิด service กลับ แต่คุณต้องรู้ว่าเดิมมันตั้งค่าเป็นอะไร เพราะ service มีสถานะมากกว่าแค่เปิดกับปิด

คำถามข้อห้า มีเงื่อนไขใหม่ ถ้าทั้งโพลด์ไม่มีประโยชน์ที่ขึ้นต้นว่า ยกเว้นเลยสักประโยค นั่นคือสัญญาณ

ห้าคำถามนี้ใช้เวลาประมาณสิบนาที น้อยกว่าเวลาที่ต้องใช้คู่ เครื่องมาก

เมื่อคำตอบคือไม่รู้

มีกรณีที่คุณถามครบห้าข้อแล้วยังตอบไม่ได้ เกิดขึ้นบ่อยกว่าที่คิด

ตอนนั้นคุณมีทางเลือกสามทาง

ทางแรกคือไม่ทำ ปลอดภัยที่สุด แต่แลกด้วยการเสียโอกาส ถ้ามันเป็นคำแนะนำที่ดีจริง

ทางที่สองคือทำ แต่ทำแบบที่ย้อนกลับได้ จดค่าเดิมไว้ก่อน เปลี่ยนทีละอย่าง ไม่เปลี่ยนสับอย่างพร้อมกัน เพราะถ้าเปลี่ยนพร้อมกันแล้วมีปัญหา คุณจะรู้ว่าอันไหนเป็นตัวการ

ทางที่สามคือรอ ถ้าคำแนะนำนั้นดีจริง อีกสามเดือนจะมีคนอธิบายกลไกของมันออกมา ถ้ามันไม่จริง อีกสามเดือนก็จะมีคนบอกว่าไม่เวิร์ก

ผมใช้ทางที่สองกับเรื่องที่ย้อนกลับได้ง่าย และใช้ทางที่สามกับเรื่องที่แตะระบบลึก

สิ่งที่ผมเลิกทำคือทางที่สี่ ทางที่ผมเคยใช้บ่อยที่สุด คือทำตาม เพราะคนอื่นบอกว่าดี แล้วไม่ทำอะไรไว้เลย



คำถามที่ถูกถาม เปิดสิ่งที่คำตอบสำเร็จรูปปิดไว้

ทำไมการยอมรับว่าไม่รู้ ถึงเป็นสัญญาณที่ดี

ปกติเราคิดว่าคนที่มั่นใจคือคนที่รู้จริง

ในเรื่องทางเทคนิค ผมคิดว่าตรงข้าม

คนที่รู้เรื่องอะไรลึกจริง ๆ จะรู้ขอบเขตของสิ่งที่ตัวเองรู้ เขาจะบอกได้ว่าวิธีนี้ใช้ได้กับเครื่องแบบไหน ใช้ไม่ได้กับแบบไหน และเคยมีกรณีไหนบ้างที่มันไม่เวิร์ก

ส่วนคนที่รู้ผิวเผินจะพูดเป็นสูตรสำเร็จ เพราะเขาไม่เคยเจอกรณีที่ยังไม่พ้อง หรือเจอแล้วแต่ไม่รู้ว่ที่พ้องเพราะอะไร

เวลาผมอ่านคำแนะนำอะไร ผมจึงมองหาประโยคที่ขึ้นต้นว่า "ยกเว้นถ้า" หรือ "อันนี้ใช้ไม่ได้กับ"

ถ้าไม่มีประโยคแบบนั้นเลยทั้งบทความ ผมจะระวังเป็นพิเศษ

คนที่บอกคุณได้ว่าเครื่องมือของเขาทำอะไรไม่ได้ คือคนที่รู้จริงว่มันทำอะไรได้

**คนที่รู้เรื่องอะไรลึกจริง ๆ จะรู้ขอบเขตของสิ่งที่
ตัวเองรู้**

สัญญาณเตือนที่เจอบ่อย

จากการอ่านคำแนะนำมาหลายร้อยชิ้น ผมเจอรูปแบบที่ซ้ำกันจนใช้เป็นสัญญาณเตือนได้

สัญญาณแรกคือตัวเลขที่กลมเกินไป ถ้าเจอ "เร็วขึ้น 30%" หรือ "ลดความหน่วงลงครึ่งหนึ่ง" โดยไม่มีเงื่อนไขกำกับ ให้สงสัยไว้ก่อน ผลของการปรับแต่งจริงมักออกมาเป็นตัวเลขที่ไม่สวย และมักต่างกันไปตามเครื่อง

เทียบกับของจริง

สิ่งที่เชื่อ	ตัวเลขที่กลมสวยอย่าง "เร็วขึ้น 30%" ฟังดูน่าเชื่อถือกว่าตัวเลขที่ดูรก
สิ่งที่วัดได้	คำแนะนำเรื่องคอมพิวเตอร์มีอายุ สิ่งที่ถูกในปี 2015 อาจผิดในปี 2025 เพราะระบบปฏิบัติการเปลี่ยนไปแล้ว ตัวเลขที่ไม่มีวันกำกับก็มีปัญหาแบบเดียวกัน

สัญญาณที่สองคือการอ้างว่าใช้ได้กับทุกคน คำแนะนำที่ดีจะบอกว่าเหมาะกับใคร แปลว่ามีคนที่ไม่เหมาะสมด้วย

สัญญาณที่สามคือความเร่งด่วน ถ้าคำแนะนำบอกให้ทำเดี๋ยวนี้ ก่อนจะสาย นั่นเป็นเทคนิคการขาย ไม่ใช่ข้อมูลทางเทคนิค การตั้งค่าคอมพิวเตอร์ไม่มีอะไรที่รอกถึงพรั้งนี้ไม่ได้

สัญญาณที่สี่คือการไม่มีวันที่ คำแนะนำเรื่องคอมพิวเตอร์มีอายุ สิ่งที่ถูกในปี 2015 อาจผิดในปี 2025 เพราะระบบปฏิบัติการเปลี่ยนไปแล้ว ถ้าไม่รู้ว่าเขียนเมื่อไหร่ ก็ไม่รู้ว่ายังใช้ได้ไหม

เรื่อง defrag ไดรฟ์ SSD ที่ผมเล่าในบทที่ 1 เป็นตัวอย่างที่ดีที่สุดของข้อนี้ มันเคยถูกต้อง แล้วเทคโนโลยีเปลี่ยน แต่คำแนะนำไม่ได้เปลี่ยนตาม

สัญญาณเตือนสี่ข้อนี้ไม่ได้พิสูจน์ว่าคำแนะนำนั้นผิด มันแค่บอกว่าควรถามหาคำถามในหัวข้อก่อนหน้าให้ครบก่อนเชื่อ

สิ่งที่หนังสือเล่มนี้ยังตอบไม่ได้

ผมอยากปิดเล่มด้วยการบอกว่าอะไรที่ผมยังไม่รู้ เพราะถ้าผมจบด้วยการสรุปว่าทุกอย่างเรียบร้อยแล้ว ผมก็จะทำสิ่งเดียวกับที่ผมเตือนไว้ตลอดเจ็ดบท

ข้อแรก ผมยังไม่รู้ว่าการปรับแต่งที่ผ่านเกณฑ์ทั้งสี่สิบรายการ ช่วยผู้เล่นจริงได้แค่ไหน เพราะยังไม่ได้เก็บข้อมูล

ข้อสอง เกณฑ์ห้ามิตที่ผมใช้ ผมเป็นคนกำหนดน้ำหนักเอง ยังไม่มีใครมาตรวจว่าน้ำหนักนั้นเหมาะสมไหม ถ้าคนอื่นให้น้ำหนักต่างออกไป รายการที่ผ่านอาจไม่เหมือนเดิม

ข้อสาม ตัวกรองคำสั่งอันตรายของผมทำงานด้วยการเทียบรูปแบบที่รู้จัก มันจับสิ่งที่ผมนึกออกได้ แต่จับสิ่งที่ผมนึกไม่ออกไม่ได้ ถ้ามีคนตั้งใจเล็งมัน ผมยังไม่ได้ทดสอบว่าจะเป็นยังไง

ข้อสี่ ผมทดสอบบนฮาร์ดแวร์ชุดหนึ่ง กับเกมเกมเดียว ถ้าเปลี่ยนเครื่องหรือเปลี่ยนเกม ผมไม่รู้ว่าจะเหมือนกันไหม

สี่ข้อนี้ไม่ใช่คำขอโทษ มันคือขอบเขตของสิ่งที่ผมพูดได้ และผมคิดว่าความรู้ขอบเขตของตัวเองมีค่ากว่าการเคลมเกินขอบเขต แล้วถูกจับได้ทีหลัง

ถ้าจะทำต่อจากตรงนี้

มีคนถามผมว่าถ้าอยากทำอะไรแบบนี้เองบ้าง ควรเริ่มตรงไหน

คำตอบของผมไม่ใช่การเริ่มจากเครื่องมือ แต่เริ่มจากการจด

ลองจดว่าเครื่องคุณตอนนี้ตั้งค่าอะไรไว้บ้างที่ไม่ใช่ค่าเริ่มต้น และคุณรู้ไหมว่าแต่ละอันมาจากไหน

ตอนผมทำแบบนี้กับเครื่องตัวเอง ผมพบว่ามียูเอจิดอย่างที่ผมตั้งไว้เมื่อหลายปีก่อนโดยทำตามคนอื่น สองในนั้นผมหาเหตุผลไม่เจอเลย่ว่าทำไมถึงตั้ง และอีกหนึ่งอันเป็นค่าที่ควรจะย้อนกลับตั้ง แต่ตอนเปลี่ยนการ์ดจอแล้ว แต่ผมลืม

การจดใช้เวลาไม่ถึงชั่วโมง และมันเปลี่ยนความสัมพันธ์ระหว่างผมกับเครื่องของผมพอสมควร จากของที่มีคนอื่นตั้งค่าให้ กลายเป็นของที่ผมรู้ว่าข้างในมีอะไร

ขั้นต่อไปคือเวลาจะเปลี่ยนอะไร ให้จดค่าเดิมก่อนเสมอ ไม่ต้องใช้เครื่องมืออะไร สมุดหรือไฟล์ข้อความก็พอ

ฟังดูเป็นคำแนะนำที่น่าเบื่อมาก แต่นี่คือสิ่งเดียวกับที่ผมใช้เวลาหลายเดือนสร้างขึ้นเป็นซอฟต์แวร์ ต่างกันแค่ซอฟต์แวร์ทำให้อัตโนมัติ

หลักการเบื้องหลังมันเหมือนกัน คือรู้ว่าเปลี่ยนอะไรไป และย้อนกลับได้

เรื่องที่ไม่ได้เกี่ยวกับคอมพิวเตอร์

ตอนเริ่มเขียนหนังสือเล่มนี้ ผมคิดว่ากำลังเขียนเรื่องการปรับแต่งคอมพิวเตอร์

พอเขียนไปถึงบทที่ 6 ผมเริ่มคิดว่ามันไม่ใช่

บทที่ 6 คือบทที่ผมเล่าว่าการตรวจสอบภายในพบว่าตัวเลขที่ผมประกาศไป แล้วสูงเกินจริง และผมลดมันลงเอง จากห้าสิบหกเหลือสี่สิบสี่

ตอนนั้นไม่มีใครรู้ ไม่มีใครกำลังตรวจ ไม่มีใครจะจับได้ถ้าผมไม่บอก และการแก้ตัวเลขลงหมายความว่าตัวเลขที่ผมโชว์ได้จะน้อยลง

ผมนั่งคิดอยู่พักหนึ่งจริง ๆ ว่าจะแก้เจ็บ ๆ หรือจะเขียนลงไป

สิ่งที่ทำให้ตัดสินใจได้ไม่ใช่เรื่องจริยธรรมอะไรที่ยิ่งใหญ่ มันเป็นเรื่องที่ง่ายกว่านั้นมาก

ถ้าผมแก้เจ็บ ๆ แล้ววันหนึ่งมีคนไปนับเอง แล้วพบว่าตัวเลขไม่ตรงกับที่ผมเคยพูด ทุกตัวเลขที่เหลือในงานนี้จะถูกตั้งคำถามหมด รวมถึงตัวเลขที่ถูกต้อง

ความน่าเชื่อถือมันไม่ได้แยกเป็นส่วน ๆ มันพัวพันทั้งก้อน

ส่วนถ้าผมเขียนลงไปเอง สิ่งที่เกี่ยวข้องคือตัวเลขที่ดูดีน้อยลงสิบสองรายการ สิ่งที่ได้คือคนอ่านรู้ว่าตัวเลขที่เหลือผ่านการตรวจมาแล้วจริง

ผมคิดว่านี่คือการแลกที่คุ้ม และผมคิดว่ามันคุ้มในเรื่องอื่นด้วย ไม่ใช่แค่ซอฟต์แวร์

คำถามสุดท้าย

หนังสือเล่มนี้เริ่มด้วยคำถามว่า เราจะรู้ได้ยังไงว่าสิ่งที่เราเชื่อว่าเป็นจริงมันจริง

ผมไม่มีคำตอบที่ครบ แต่ผมมีสิ่งที่ใกล้เคียงที่สุดเท่าที่หามาได้

วิธีที่ผมใช้คือเขียนเกณฑ์ก่อนดูคำตอบ วัดสิ่งที่โกหกไม่ได้ ลือกวิธีวิเคราะห์ก่อนเห็นข้อมูล และยอมให้ตัวเลขของตัวเองลดลงเมื่อการตรวจสอบชี้แบบนั้น

ทั้งสองอย่างนี้มีจุดร่วมเดียวกัน คือมันคือการมัดมือตัวเองไว้ล่วงหน้า ในตอนที่ยังไม่มีแรงจูงใจจะโง่ง เพื่อกันตัวเองในตอนที่จะมี เพราะคนที่คุณต้องระวังที่สุดเวลาตรวจสอบอะไรสักอย่าง ไม่ใช่คนที่พยายามหลอกคุณ คือตัวคุณเองที่อยากให้คำตอบออกมาเป็นแบบที่หวังไว้

คำถามถึงคุณ

มีอะไรอยู่บนเครื่องของคุณตอนนี้ ที่คุณตั้งค่าไว้ตามคนอื่นบอก แล้วไม่รู้ด้วยซ้ำว่าทำไมถึงตั้ง
ลองนึกดูว่าคุณเชื่อเรื่องไหนอยู่ตอนนี้ โดยที่ยังไม่เคยวัดด้วยตัวเองเลยสักครั้ง

ครั้งหน้าที่คุณเจอคำแนะนำที่ฟังดูดี ลองถามหาคำถามในบทนี้ดู และถ้าคุณเป็นคนที่กำลังจะแนะนำอะไรใครสักอย่าง ลองถามตัวเองด้วยคำถามเดียวกัน

คำถามที่อยากที่สุดมักจะเป็นข้อสุดท้าย

คำถามที่ยากที่สุดไม่ใช่คำถามที่ถามคนอื่น

คือคำถามที่ถามตัวเองตอนอยากเชื่อใน
สิ่งที่ยังไม่ได้วัด

ถ้าคุณอ่านมาถึงตรงนี้แล้วเริ่มถามคำถามพวกนี้กับตัวเองบ้าง เล่มนี้
ก็ทำงานของมันเสร็จแล้ว

อ่านบทนี้แล้วเปลี่ยนอะไร

ก่อนอ่าน การเชื่อคำแนะนำเรื่องคอมพิวเตอร์ ก็แค่ต้องหาคนที่ดูน่าเชื่อถือ

หลังอ่าน มีห้าคำถามที่ใช้ตรวจคำแนะนำได้ทุกชิ้น และคนที่รู้จริงคือคนที่
บอกได้ว่าวิธีของเขาใช้ไม่ได้ตอนไหน

- ✓ จำห้าคำถามที่ใช้ได้กับคำแนะนำทุกชิ้นที่เจอ ตั้งแต่แลกอะไรไปถึงย้อนกลับ
ยังไ้
- ✓ รู้จักสัญญาณเตือนสี่แบบที่เจอบ่อยในโพสต์แนะนำการปรับแต่ง
- ✓ รู้ขอบเขตที่หนังสือเล่มนี้ยังตอบไม่ได้ และวิธีเริ่มจดสิ่งที่ตั้งค่าไว้บนเครื่องตัว
เอง

อภิธานศัพท์

คำที่ใช้ซ้ำในเล่มนี้ เรียงตามลำดับที่ปรากฏครั้งแรก ไม่ใช่ตามตัวอักษร เพราะถ้าคุณกลับมาเปิดหน้านี้ คุณน่าจะกำลังหาคำที่เพิ่งอ่านเจอ

คำเกี่ยวกับเครื่องและระบบ

ความหน่วง เวลาที่ผ่านไประหว่างการกระทำกับผลที่เห็น เช่นจากตอนคลิกเมาส์ถึงตอนที่ภาพบนจอเปลี่ยน ในเล่มนี้หมายถึงความหน่วงที่เกิดจากระบบปฏิบัติการเป็นหลัก ไม่ใช่ความหน่วงของเครือข่าย

DPC latency ความหน่วงในการจัดการงานเร่งด่วนระดับล่างของ Windows วัดจากภายในระบบปฏิบัติการโดยตรง จุดสำคัญคือผู้ใช้รู้สึกถึงค่านี้ไม่ได้ จึงใช้เป็นตัววัดที่ปลอดภัยจากความคาดหวังของผู้เล่น

registry ฐานข้อมูลกลางที่ Windows เก็บการตั้งค่าเกือบทั้งหมดไว้ การปรับแต่งส่วนใหญ่ในเล่มนี้ คือการเปลี่ยนค่าในนี้

service โปรแกรมที่ทำงานอยู่เบื้องหลังโดยผู้ใช้ไม่เห็น บางตัวจำเป็น บางตัวไม่จำเป็นกับทุกคน เล่มนี้ป้องกันไว้ยี่สิบสี่ตัวที่ห้ามปิดไม่ว่ากรณีใด

VBS กลไกความปลอดภัยของ Windows ที่ใช้การจำลองระบบเพื่อแยกส่วนสำคัญออกจากส่วนที่เหลือ การปิดมันทำให้เครื่องเร็วขึ้นได้ และลดความปลอดภัยลงด้วย เป็นการปรับแต่งความเสี่ยงสูงเพียงรายการเดียวในสี่สิบสี่รายการ

เฟรมไทม์ เวลาที่ใช้วาดภาพหนึ่งเฟรม ต่างจากเฟรมเรตตรงที่เฟรมเรตเป็นค่าเฉลี่ยต่อวินาที ส่วนเฟรมไทม์ดูทีละเฟรม จึงเห็นการสะดุดที่ค่าเฉลี่ยซ่อนไว้

คำเกี่ยวกับเครื่องมือที่สร้าง

การปรับแต่ง การเปลี่ยนค่าหนึ่งอย่างในระบบเพื่อหวังผลด้านประสิทธิภาพ ในเล่มนี้นับเฉพาะรายการ ที่มีสัญญาณการทำงานผ่านการตรวจสอบแล้ว ซึ่งมีสี่สิบสี่รายการ

โปรไฟล์ ชุดการปรับแต่งที่จัดกลุ่มไว้ให้เลือกใช้ทีเดียว มีสามระดับและเป็นแบบสะสม คือระดับที่แรงกว่ารวมระดับที่เบากว่าไว้ด้วย

การสำรองข้อมูล การบันทึกค่าเดิมก่อนเปลี่ยน เพื่อให้ย้อนกลับได้ในเล่มนี้เป็นขั้นบังคับ ถ้าสำรองไม่สำเร็จ โปรแกรมจะหยุด ไม่ทำต่อ

ย้อนกลับ การคืนค่าที่เปลี่ยนไปแล้วให้กลับเป็นเหมือนเดิม ต่างจากจุดคืนค่าระบบตรงที่ ย้อนกลับทีละรายการได้ ไม่ต้องย้อนทั้งเครื่อง

Flight Recorder ส่วนที่บันทึกทุกการเปลี่ยนแปลงพร้อมค่าก่อนและหลัง แล้วสร้างสคริปต์ย้อนกลับให้อัตโนมัติ ชื่อมาจากกล่องดำในเครื่องบิน ที่บันทึกไว้เพื่อวันที่ต้องย้อนดูว่าเกิดอะไรขึ้น

fail-closed หลักการออกแบบที่ว่าเมื่อระบบไม่แน่ใจ ให้ปฏิเสธไว้ก่อน ไม่ใช่ปล่อยให้ผ่าน เป็นเหตุผลที่ระบบยังปลอดภัยตลอดแม้ตัวเลขที่ประกาศจะเกินจริง

คำเกี่ยวกับการวัดผล

การถ่วงดุลลำดับ การสลับลำดับที่ผู้เข้าร่วมแต่ละคนเจอเงื่อนไข เพื่อให้ผลของลำดับกระจายเท่ากันทุกฝั่ง แล้วหักล้างกันเองเมื่อรวมผล

ตัวแปรแทรกซ้อน สิ่งที่เปลี่ยนไปพร้อมกับสิ่งที่เราตั้งใจเปลี่ยน จนแยกไม่ออกจากผลมาจากอะไร เช่นการวัดก่อนปรับตอนเช้าแล้ววัดหลังปรับตอนเย็น อุณหภูมิเครื่องเป็นตัวแปรแทรกซ้อน

การพิสูจน์เท็จไม่ได้ คุณสมบัติของข้ออ้างที่ไม่มีทางถูกพิสูจน์ว่าผิด ไม่ว่าผลจะออกมาแบบไหน ข้ออ้างแบบนี้ก็ไม่มีทางถูกพิสูจน์ว่าถูกเหมือนกัน

การล็อกแผนวิเคราะห์ การเขียนวิธีวิเคราะห์ทั้งหมดให้เสร็จก่อนเห็นข้อมูล เพื่อกันตัวเองไม่ให้เลือกวิธี ที่ให้ผลตามที่หวังหลังเห็นข้อมูลแล้ว

คำถามชวนคุย

หนังสือเล่มนี้เถียงเรื่องที่เถียงต่อได้ คำถามชุดนี้เขียนไว้สำหรับกลุ่มอ่านหนังสือ วรสนทนาในที่ทำงาน หรือคนที่อ่านคนเดียวแล้วอยากคิดต่อ

ไม่มีคำถามข้อไหนที่มีคำตอบถูกข้อเดียว ถ้ามี ผมคงไม่ใส่ไว้

เรื่องหลักฐานกับความเชื่อ

หนึ่ง ในบทที่ 2 ผมเขียนว่าเครื่องมือเหล่านั้นไม่ได้โกหก มันแค่ไม่เคยถูกถาม คุณเห็นด้วยไหมว่าการไม่บอกสิ่งที่ผู้ใช้ไม่ได้ถาม ต่างจากการปกปิด แล้วเส้นแบ่งอยู่ตรงไหน

สอง ความเชื่อหกข้อที่ผมตัดออก ผมตัดเพราะไม่มีใครพิสูจน์ได้ว่ามันได้ผล ไม่ใช่เพราะพิสูจน์ได้ว่าไม่ได้ผล คุณคิดว่าสองเหตุผลนี้ควรได้รับการปฏิบัติต่างกันไหม

สาม ลองนึกถึงคำแนะนำที่คุณเชื่อมั่นานโดยไม่เคยตรวจ มันผ่านคำถามห้าข้อในบทที่ 8 ก็ข้อ

เรื่องความปลอดภัยกับความเร็ว

สี่ การปรับแต่งความเสี่ยงสูงรายการเดียวในเล่มนี้คือการปิด VBS ซึ่งทำให้เร็วขึ้นจริง และปลอดภัยน้อยลงจริง ถ้าคุณเป็นคนออกแบบ คุณจะใส่มันไว้ไหม และถ้าใส่ คุณจะทำยังไงให้ผู้รู้ตัวว่ากำลังแลกอะไร

ห้า ผมขึ้นบัญชีห้ามถาวรไว้เจ็ดรายการ ที่โปรแกรมจะไม่ทำให้แม่ผู้ใช้ขอเอง การตัดสินใจแทนผู้ใช้แบบนี้ถูกต้องไหม หรือเป็นการดูถูกว่าเขาตัดสินใจเองไม่ได้

หก ระบบห้าชั้นมีต้นทุน ทั้งเวลาเขียน เวลารัน และความซับซ้อน ถ้าคุณต้องตัดออกหนึ่งชั้น คุณจะตัดชั้นไหน เพราะอะไร

เรื่องการตรวจสอบตัวเอง

เจ็ด ในบทที่ 6 ไม่มีใครจะจับได้ถ้าผมแก้เจียบ ๆ คุณคิดว่าอะไรทำให้คนเลือกบอก ทั้งที่การไม่บอกง่ายกว่าและได้ประโยชน์มากกว่า

แปด ผมเขียนว่าความน่าเชื่อถือไม่ได้แยกเป็นส่วน ๆ มันพังทั้งก้อน คุณเคยเจอกรณีที่ความผิดพลาดจุดเดียว ทำให้คุณสงสัยทุกอย่างที่คนนั้นเคยพูดไหม แล้วมันยุติธรรมหรือเปล่า

เก้า ถ้าคุณเป็นคนตรวจงานตัวเอง ในองค์กรที่ตัวเลขสวยมีผลต่อการประเมิน อะไรจะทำให้คุณยอมรายงานตัวเลขที่ลดลง

เรื่องการวัดผล

สิบ ผมล็อกแผนวิเคราะห์ไว้ก่อนเก็บข้อมูล เพื่อกันตัวเองไม่ให้เลือกวิธีที่ให้ผลที่หวัง คุณคิดว่ามีวิธีอื่นที่กันเรื่องนี้ได้อีกไหม

ลิปเอ็ด ลิบคนน้อยเกินไปสำหรับการสรุปอะไรที่หนักแน่น แต่การรอให้ได้สามลิปคน อาจแปลว่าไม่ได้ทำเลย คุณคิดว่าควรลงมือทดลอง ทั้งที่รู้อยู่แล้วว่ากลุ่มตัวอย่างเล็กเกินไป หรือควรรอจนกว่าจะทำได้เต็มรูปแบบ

ลิปสอง หนังสือเล่มนี้เขียนเสร็จก่อนการทดลองจะเริ่ม คุณคิดว่านั่นทำให้มันน่าเชื่อถือน้อยลง หรือมากขึ้น

กิตติกรรมประกาศ

หนังสือเล่มนี้เขียนจากงานที่ทำคนเดียวเป็นส่วนใหญ่ แต่ไม่มีบทไหนที่เกิดขึ้นได้โดยไม่มีคนอื่น

ขอบคุณเพื่อนที่เครื่องเปิดไม่ติดในคืนนั้น ถ้าเขาไม่ส่งข้อความมาตอนตีสองครึ่ง ผมคงยังคิดว่าเรื่องนี้เป็นปัญหาทางเทคนิค ไม่ใช่ปัญหาเรื่องข้อมูลที่คุณได้รับไม่ครบ

ขอบคุณคนที่เขียนเครื่องมือทั้งยี่สิบสามโครงการ ที่ผมอ่านโค้ดงานของพวกเขาเปิดให้อ่านฟรีทั้งที่ไม่มีใครบังคับ

เหมือนร้านอาหารที่เปิดครัวให้ลูกค้ามองเห็น ร้านแบบนี้ถูกวิจารณ์ง่ายกว่า ร้านที่ปิดประตูครัวสนิท แต่ก็ยังเป็นร้านเดียวที่เราตรวจสอบได้จริง

เครื่องมือที่ผมวิจารณ์ไม่ได้เลย คือเครื่องมือที่ปิดโค้ด

ขอบคุณอาจารย์ที่ปรึกษาที่ถามคำถามที่ผมตอบไม่ได้ในตอนนั้น คำถามนั้นทำให้โครงการเปลี่ยนทิศ และกลายเป็นเหตุผลที่หนังสือเล่มนี้มีอยู่

ขอบคุณคนที่อ่านต้นฉบับก่อนใครแล้วบอกว่าบทที่ 6 ยังไม่หนักพอ ตอนแรกผมไม่เห็นด้วย แล้วกลับไปอ่านอีกรอบก็พบว่าเขาพูดถูก บทนั้นจึงยาวขึ้นและตรงขึ้น

และขอบคุณคนที่จะเป็นผู้เข้าร่วมการทดลองสิบคน ซึ่งตอนที่เขียนบรรทัดนี้ยังไม่รู้ว่าเป็นใคร งานส่วนที่สำคัญที่สุดของโครงการนี้ยังรอพวกเขาอยู่

มีคนถามผมว่า "แล้วรู้ได้ยังไงว่าที่พูดมาจริง"

ผมตอบไม่ได้ ไม่ใช่เพราะสิ่งที่พูดไม่จริง แต่เพราะผมไม่มีวิธีพิสูจน์ ที่ต่างจากวิธีที่คนอื่นใช้ วิธีที่ผมเพิ่งพิจารณาไปเมื่อครู่ว่าเชื่อไม่ได้

เล่มนี้เริ่มจากคำถามนั้น แล้วพาไปจบที่วันที่ผมนั่งอยู่หน้าจอ แล้วพบว่าตัวเลขที่ตัวเองประกาศไว้ทุกที ไม่ตรงกับของจริง

- อ่านโค้ดยี่สิบสามโครงการ แล้วพบว่าเกือบครึ่งไม่บอกผู้ใช้เลยว่าปุ่มไหนอันตราย
- สร้างเกณฑ์ห้ามิติที่ตัดคำแนะนำสองร้อยข้อเหลือสี่สิบห้า แล้วเกณฑ์นั้นย้อนกลับมาตัดงานตัวเอง
- เขียนการทดสอบพันกว่าตัวเพื่อพยายามทำให้ของตัวเองพัง แล้วมันพังจริง
- วันที่ห้าสิบหกกลายเป็นสี่สิบสี่ กับการเลือกว่าจะแก้ไขบ ั หรือเขียนลงเล่ม

เขียนสำหรับคนที่เคยกดปุ่มปรับแต่งโดยไม่รู้ว่ามันทำอะไร คนที่ต้องตัดสินใจว่าจะเชื่อคำแนะนำไหน และคนที่กำลังทำโครงการแล้วต้องพิสูจน์อะไรสักอย่าง

ไม่ต้องรู้เรื่องระบบปฏิบัติการ สถิติ หรือการเขียนโปรแกรมมาก่อน ทุกอย่างที่ต้องใช้อธิบายด้วยการเปรียบเทียบกับของที่เห็นได้ในชีวิตประจำวัน